Project Phase 2- Preliminary Analysis

Team Popcorn - Amritha, Ahana, Keerthana

05/3/2026

Contents

So we have three tasks in the MST_Data folder **2**

Dataset Description **2**

VISUALISATION **58**

NORMALITY TESTING **67**

STATISTICAL METHODS **84**

——————————————— **84**

REACTION TIME ANALYSIS **84**

——————————————— **84**

1. Does the pattern make sense? **88**

Your plot shows: 88

2. One thing to double-check (important) **89**

Fix (for interpretation clarity): 89

3. Are effect sizes reasonable? **89**

4. What would be "wrong" output? **90**

5. Interpretation you can use (report-ready) **90**

6. One improvement (optional but recommended) **90**

Final verdict **90**

REPEATED MEASURES ANOVA **90**

Post HOC Test using EMM 94

One-Sample t-tests (Boundary vs Baseline) 96

Paired t-test (Pre vs Post) **108**

Lure Bin Analysis **112**

Quartile Analysis **121**

```
knitr::opts_chunk$set(cache = FALSE)
```

So we have three tasks in the MST_Data folder

Both_item_task/ both_data/ # Data files for Experiment 1 item_only/ item_only_data/ # Data files for Experiment 2 task_only/ task_only_data/ # Data files for Experiment 3

Dataset Description

The dataset used in this study is organized into three experimental conditions of the Mnemonic Similarity Task (MST): **Both Item + Task**, **Item-only**, and **Task-only**. Each condition is stored in a separate directory within the `MST_Data` folder.

Within each condition, the data is structured at the participant level. For every participant, there are two CSV files corresponding to the two phases of the experiment:

- a **task file**, representing the encoding (study) phase
- a **test file**, representing the retrieval (memory test) phase

The files follow a consistent naming convention of the form: `<subjectID>_MST_<phase>_<date>_<timestamp>.csv`, where the subject ID uniquely identifies each participant and the phase indicates whether the file corresponds to the encoding or test stage.

The task files primarily contain information related to stimulus presentation and encoding responses, including image paths, trial indices, and key responses recorded during the study phase.

The test files contain trial-level behavioral data used for analysis, including:

stimulus identity (e.g., scene or object images) stimulus condition (e.g., target, lure, foil — inferred from stimulus position/type) participant responses (e.g., old, similar, new) reaction times associated with each response

From these test-phase variables, key behavioral measures are computed:

LDI (Lure Discrimination Index), reflecting the ability to distinguish similar items REC (Recognition Score), reflecting general recognition performance Reaction Time (RT), capturing response latency

The dataset is structured such that each participant contributes multiple trials, enabling within-subject analysis of memory performance across different stimulus types.

Experiment Condition: 1 Both (item + task boundary) 2 Item-only 3 Task-only # Preprocessing

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
base_path <- "../MST_Data"

clean_task_file <- function(file_path) {

  cat("\n==============================\n")
  cat("Processing:", basename(file_path), "\n")

  df <- suppressMessages(
    read_csv(file_path, show_col_types = FALSE, name_repair = "unique")
  )

  cat("Initial rows:", nrow(df), "\n")

  # --------------------------------
  # Extract participant ID
  # --------------------------------
  participant_id <- str_extract(basename(file_path), "^\\d{5}")

  # --------------------------------
  # Ensure required columns exist
  # --------------------------------
  required_cols <- c("image_path", "trials.thisN")

  missing_cols <- setdiff(required_cols, names(df))

  if(length(missing_cols) > 0){
    stop(paste("Missing columns:", paste(missing_cols, collapse=", ")))
  }

  # --------------------------------
  # Create RT columns if missing
  # --------------------------------
  if(!"trials.key_resp_9.rt" %in% names(df)) df$trials.key_resp_9.rt <- NA
  if(!"trials.key_resp_8.rt" %in% names(df)) df$trials.key_resp_8.rt <- NA

  # --------------------------------
  # CLEAN
  # --------------------------------
  df_clean <- df %>%
```

```r
    # remove non-trial rows
    filter(!is.na(trials.thisN)) %>%

    # remove empty image rows
    filter(!is.na(image_path), image_path != "") %>%

    # remove practice
    filter(!str_detect(image_path, "practice")) %>%

    mutate(
      participant_id = participant_id,
      trial_number = trials.thisN + 1,

      # numeric safety
      rt_9 = suppressWarnings(as.numeric(trials.key_resp_9.rt)),
      rt_8 = suppressWarnings(as.numeric(trials.key_resp_8.rt)),

      # RT correction
      rt = case_when(
        !is.na(rt_9) ~ rt_9,
        !is.na(rt_8) ~ 3 + rt_8,
        TRUE ~ NA_real_
      ),

      image_name = basename(image_path)
    ) %>%

    select(participant_id, trial_number, image_path, image_name, rt)

  # -------------------------------
  # DEBUG PRINTS
  # -------------------------------
  cat("Rows after cleaning:", nrow(df_clean), "\n")
  cat("Unique images:", length(unique(df_clean$image_name)), "\n")
  cat("RT summary:\n")
  print(summary(df_clean$rt))

  return(df_clean)
}

# TEST PREPROCESSER
clean_test_file <- function(file_path) {

  cat("\n=============================\n")
  cat("Processing TEST:", basename(file_path), "\n")

  df <- suppressMessages(
    read_csv(file_path, show_col_types = FALSE, name_repair = "unique")
  )

  participant_id <- str_extract(basename(file_path), "^\\d{5}")

  df_clean <- df %>%
```

```r
    filter(!is.na(trials.thisN)) %>%

    filter(!is.na(image_path), image_path != "") %>%

    mutate(
      participant_id = participant_id,
      trial_number = trials.thisN + 1,
      image_name = basename(image_path),

      # classify stimulus
      stimulus_type = case_when(
        str_detect(tolower(image_path), "^foil") ~ "Foil",
        str_detect(image_name, "b\\.jpg$") ~ "Lure",
        str_detect(image_name, "a\\.jpg$") ~ "Target",
        TRUE ~ NA_character_
      ),

      # response mapping
      response = case_when(
        trials.key_resp_3.keys == "o" ~ "Old",
        trials.key_resp_3.keys == "s" ~ "Similar",
        trials.key_resp_3.keys == "n" ~ "New",
        TRUE ~ NA_character_
      )
    ) %>%

    select(participant_id, trial_number, image_path, image_name, stimulus_type, response)

  # DEBUG
  cat("Rows:", nrow(df_clean), "\n")
  cat("Stimulus counts:\n")
  print(table(df_clean$stimulus_type))

  return(df_clean)
}

# LOAD ALL EXPEERIMENTS

load_experiment <- function(task_dir, test_dir, label) {

  cat("\n==============================\n")
  cat("LOADING EXPERIMENT:", label, "\n")

  task_files <- list.files(task_dir, pattern = "task.*\\.csv$", full.names = TRUE)
  test_files <- list.files(test_dir, pattern = "test.*\\.csv$", full.names = TRUE)

  cat("Task files:", length(task_files), "\n")
  cat("Test files:", length(test_files), "\n")

  task_data <- map_dfr(task_files, clean_task_file)
  test_data <- map_dfr(test_files, clean_test_file)

  task_data$experiment <- label
```

```r
  test_data$experiment <- label

  return(list(task = task_data, test = test_data))
}

# RUN PREPROCESSING FOR ALL THREE EXPERIMENTS

exp1 <- load_experiment(
  "../MST_Data/Both_item_task/both_data",
  "../MST_Data/Both_item_task/both_data",
  "both"
)
```

```
##
## ==============================
## LOADING EXPERIMENT: both
## Task files: 52
## Test files: 52
##
## ==============================
## Processing: 00015_MST_task_2025-11-08_11h36.39.538.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.764   1.247   1.604   1.756   2.094   6.630
##
## ==============================
## Processing: 00016_MST_task_2025-11-08_17h23.18.303.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5795  0.9818  1.2303  1.3901  1.7021  5.4214
##
## ==============================
## Processing: 00017_MST_task_2025-11-08_19h45.48.093.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5158  1.1258  1.6407  2.1604  2.8202  7.8410       1
##
## ==============================
## Processing: 00018_MST_task_2025-11-09_10h52.32.840.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.4007  0.7693  1.0461  1.2912  1.6681  5.8094
##
## ==============================
## Processing: 00019_MST_task_2025-11-09_15h07.19.518.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5438  0.9498  1.3690  1.7595  2.0897  6.9205
##
## ==============================
## Processing: 00020_MST_task_2025-11-09_17h59.31.021.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4729  0.9038  1.3011  1.5052  1.9273  5.5420
##
## ==============================
## Processing: 00021_MST_task_2025-11-10_14h42.14.279.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5582  0.9139  1.3180  1.8409  2.1955  8.1655
##
## ==============================
## Processing: 00022_MST_task_2025-11-11_04h49.33.010.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4963  1.1159  1.6026  1.8512  2.3606  7.9743
##
## ==============================
## Processing: 00023_MST_task_2025-11-14_15h38.10.062.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3689  1.6777  3.2108  2.7800  3.3874 17.9845
##
## ==============================
## Processing: 00024_MST_task_2025-11-14_18h21.40.405.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.5287  0.8408  1.2492  1.5714  2.0138  5.2116
##
## ==============================
## Processing: 00025_MST_task_2025-11-14_21h47.43.146.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5528  0.8662  1.1140  1.2681  1.5029  4.6700
##
## ==============================
## Processing: 00026_MST_task_2025-11-15_16h03.48.687.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5623  0.9737  1.3206  1.4423  1.7490  3.7383
##
## ==============================
## Processing: 00027_MST_task_2025-11-17_14h45.33.830.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4709  0.7649  1.0187  1.2016  1.4111  6.9029
##
## ==============================
## Processing: 00028_MST_task_2025-11-17_17h09.59.427.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6207  0.9531  1.2139  1.4281  1.6454  8.5223       1
##
## ==============================
## Processing: 00029_MST_task_2025-11-17_19h57.21.052.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5961  1.2012  1.6478  1.7380  2.1964  4.8947
##
## ==============================
## Processing: 00030_MST_task_2025-11-18_16h15.07.638.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
```

```
##  0.7662  1.7605  2.5078  2.5678  3.3157  5.5514       1
##
## ==============================
## Processing: 00031_MST_task_2025-11-21_15h47.15.320.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5579  1.1241  1.4806  1.7417  2.1963  4.8077
##
## ==============================
## Processing: 00032_MST_task_2026-01-16_10h16.07.864.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5647  1.0637  1.5947  1.8306  2.2502  6.5565
##
## ==============================
## Processing: 00033_MST_task_2026-01-16_11h18.29.757.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3975  0.7016  0.9694  1.1573  1.4294  4.0232
##
## ==============================
## Processing: 00033_MST_task_2026-01-16_12h22.39.096.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6062  1.2539  1.9320  2.2789  3.4154  6.0673
##
## ==============================
## Processing: 00034_MST_task_2026-01-16_11h55.32.155.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   0.785   1.220   1.743   1.969   2.440   8.503       1
##
## ==============================
## Processing: 00036_MST_task_2026-01-16_15h17.10.885.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.5209  0.8828  1.2069  1.3978  1.7852  5.6305
##
## ==============================
## Processing: 00037_MST_task_2026-01-16_21h31.12.620.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6565  1.4709  2.2482  2.3432  3.1195  7.8100       1
##
## ==============================
## Processing: 00038_MST_task_2026-01-16_22h21.40.058.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5295  0.7833  1.1002  1.4074  1.5214 14.4026       1
##
## ==============================
## Processing: 00039_MST_task_2026-01-17_10h46.08.097.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4666  0.8667  1.2119  1.4441  1.7238  4.7639
##
## ==============================
## Processing: 00040_MST_task_2026-01-17_11h26.33.901.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5513  0.9985  1.3561  1.4962  1.8639  4.6831
##
## ==============================
## Processing: 00041_MST_task_2026-01-17_12h11.08.959.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7141  1.1062  1.6431  1.8277  2.3177  6.3833
##
## ==============================
## Processing: 00042_MST_task_2026-01-17_15h05.40.791.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.1347  1.2667  2.1804  2.3473  3.3373  9.7890
##
## ==============================
## Processing: 00043_MST_task_2026-01-17_15h59.33.115.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7944  1.3765  1.9466  2.1854  2.6411  6.8975
##
## ==============================
## Processing: 00044_MST_task_2026-01-17_16h35.46.136.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.742   1.233   1.644   1.845   2.382   4.694
##
## ==============================
## Processing: 00045_MST_task_2026-01-17_17h37.26.166.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6302  1.1267  1.5852  1.8243  2.3175  4.7913       1
##
## ==============================
## Processing: 00046_MST_task_2026-01-17_18h21.17.024.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6559  1.0365  1.4891  1.5476  1.9329  3.5184       1
##
## ==============================
## Processing: 00047_MST_task_2026-01-18_10h42.03.071.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5212  0.9538  1.2525  1.5394  1.8791  5.5513
##
## ==============================
## Processing: 00049_MST_task_2026-01-18_12h17.50.288.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.5384  0.9236  1.2287  1.5162  1.9577  5.0367
##
## ==============================
## Processing: 00050_MST_task_2026-01-18_14h46.37.542.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5673  0.9323  1.2200  1.4690  1.8132  4.7854       1
##
## ==============================
## Processing: 00051_MST_task_2026-01-18_15h58.48.144.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6571  1.2526  1.5867  1.6715  2.0334  4.3554
##
## ==============================
## Processing: 00052_MST_task_2026-01-18_16h38.10.006.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.07752 1.64400 2.31227 2.25816 2.81444 7.02289
##
## ==============================
## Processing: 00053_MST_task_2026-01-18_17h18.33.436.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6063  0.9245  1.2332  1.5465  1.9559  4.9718       1
##
## ==============================
## Processing: 00054_MST_task_2026-01-18_18h01.08.629.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5761  0.9376  1.2128  1.4344  1.7203  5.2924       1
##
## ==============================
## Processing: 00055_MST_task_2026-01-18_18h37.48.341.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
```

```
##  0.3356  0.9395  1.3971  1.6723  2.0606  6.9263       1
##
## ==============================
## Processing: 00056_MST_task_2026-01-18_19h23.59.153.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5755  0.9075  1.2049  1.4025  1.7549  4.4341
##
## ==============================
## Processing: 00057_MST_task_2026-01-18_21h46.54.870.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6358  1.2178  1.5711  1.7109  2.0423  5.2838
##
## ==============================
## Processing: 00058_MST_task_2026-01-19_10h33.30.310.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.486   2.857   3.531   4.020   4.476  18.527
##
## ==============================
## Processing: 00059_MST_task_2026-01-19_14h57.32.554.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6229  1.0246  1.2816  1.4637  1.8067  4.5727
##
## ==============================
## Processing: 00060_MST_task_2026-01-19_16h09.40.790.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5682  0.7993  1.0209  1.1298  1.3476  2.7879
##
## ==============================
## Processing: 00061_MST_task_2026-01-19_16h57.47.717.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.6594  1.0135  1.2828  1.5118  1.8289  4.1027
##
## ==============================
## Processing: 00062_MST_task_2026-01-19_17h33.39.502.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5807  1.1515  1.6695  2.0151  2.6408  6.6273       1
##
## ==============================
## Processing: 00063_MST_task_2026-01-19_18h15.20.709.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6046  1.1642  1.5341  1.5898  1.8989  3.7681
##
## ==============================
## Processing: 00064_MST_task_2026-01-19_21h51.27.317.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6733  0.9835  1.3204  1.4456  1.6963  7.3288
##
## ==============================
## Processing: 00065_MST_task_2026-01-20_10h45.49.248.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6445  1.2383  1.6544  1.8857  2.2749  5.4217
##
## ==============================
## Processing: 00066_MST_task_2026-01-20_12h00.59.817.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5385  0.8248  1.0019  1.0829  1.2557  2.6655
##
## ==============================
## Processing: 00067_MST_task_2026-01-25_15h01.19.462.csv
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.6122  1.1935  1.7141  2.0133  2.4033  6.6003
##
## ==============================
## Processing TEST: 00015_MST_test_2025-11-08_12h04.25.183.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00016_MST_test_2025-11-08_17h48.07.404.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00017_MST_test_2025-11-08_20h16.49.187.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00018_MST_test_2025-11-09_11h24.21.434.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00019_MST_test_2025-11-09_15h47.52.826.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00020_MST_test_2025-11-09_18h37.23.782.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00021_MST_test_2025-11-10_15h11.04.599.csv
## Rows: 150
## Stimulus counts:
```

```
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00022_MST_test_2025-11-11_05h14.33.217.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00023_MST_test_2025-11-14_16h06.48.021.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00024_MST_test_2025-11-14_18h50.28.414.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00025_MST_test_2025-11-14_22h13.48.402.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00026_MST_test_2025-11-15_16h29.12.970.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00027_MST_test_2025-11-17_15h14.33.025.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00028_MST_test_2025-11-17_17h48.12.274.csv
```

```
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00029_MST_test_2025-11-17_20h24.42.007.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00030_MST_test_2025-11-18_16h42.15.702.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00031_MST_test_2025-11-21_16h33.17.994.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00032_MST_test_2026-01-16_11h00.42.172.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00033_MST_test_2026-01-16_11h44.00.919.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00033_MST_test_2026-01-16_12h51.02.580.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
```

```
## ==============================
## Processing TEST: 00034_MST_test_2026-01-16_12h27.35.390.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00036_MST_test_2026-01-16_15h43.24.043.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00037_MST_test_2026-01-16_22h02.46.955.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00038_MST_test_2026-01-16_23h03.09.394.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00039_MST_test_2026-01-17_11h12.14.726.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00040_MST_test_2026-01-17_12h02.35.455.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00041_MST_test_2026-01-17_12h38.24.102.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
```

```
##     30     60     60
##
## ==============================
## Processing TEST: 00042_MST_test_2026-01-17_15h54.18.926.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00043_MST_test_2026-01-17_16h46.47.651.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00044_MST_test_2026-01-17_17h07.24.402.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00045_MST_test_2026-01-17_18h03.54.425.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00046_MST_test_2026-01-17_18h47.41.863.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00047_MST_test_2026-01-18_11h21.32.587.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00049_MST_test_2026-01-18_12h55.08.788.csv
## Rows: 150
## Stimulus counts:
```

```
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00050_MST_test_2026-01-18_15h27.16.543.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00051_MST_test_2026-01-18_16h25.44.190.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00052_MST_test_2026-01-18_17h03.51.417.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00053_MST_test_2026-01-18_17h47.05.654.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00054_MST_test_2026-01-18_18h26.51.352.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00055_MST_test_2026-01-18_19h02.33.344.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00056_MST_test_2026-01-18_19h48.57.810.csv
```

```
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00057_MST_test_2026-01-18_22h26.01.980.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00058_MST_test_2026-01-19_11h41.42.159.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00059_MST_test_2026-01-19_15h22.33.265.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00060_MST_test_2026-01-19_17h39.04.439.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00061_MST_test_2026-01-19_17h22.37.232.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00062_MST_test_2026-01-19_18h03.17.511.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
```

```
## ==============================
## Processing TEST: 00063_MST_test_2026-01-19_18h40.38.986.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00064_MST_test_2026-01-19_22h17.30.709.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00065_MST_test_2026-01-20_11h58.49.978.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00066_MST_test_2026-01-20_12h26.21.761.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00067_MST_test_2026-01-25_15h37.10.370.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
```

```r
exp2 <- load_experiment(
  "../MST_Data/item_only/item_only_data",
  "../MST_Data/item_only/item_only_data",
  "item_only"
)
```

```
##
## ==============================
## LOADING EXPERIMENT: item_only
## Task files: 56
## Test files: 56
##
## ==============================
## Processing: 00001_MST_task_2025-11-08_16h36.47.075.csv
```

```
## Initial rows: 297
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5153  0.7977  1.0120  1.3762  1.5277  7.8496
##
## ==============================
## Processing: 00002_MST_task_2025-11-08_18h50.56.769.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
## 0.03741 2.28004 2.73506 2.80715 3.18742 7.66508       2
##
## ==============================
## Processing: 00003_MST_task_2025-11-08_22h19.46.477.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3735  0.9994  1.5137  1.7544  2.2112  7.3758
##
## ==============================
## Processing: 00004_MST_task_2025-11-09_14h29.19.772.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5878  0.9291  1.1999  1.4504  1.8007  4.1831
##
## ==============================
## Processing: 00005_MST_task_2025-11-09_16h53.39.484.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4651  0.8018  1.0328  1.2172  1.5234  3.0970
##
## ==============================
## Processing: 00006_MST_task_2025-11-09_19h25.43.020.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4529  0.8721  1.2039  1.5064  2.0066  5.1423
##
## ==============================
## Processing: 00007_MST_task_2025-11-20_16h42.37.692.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3485  3.3169  3.4866  3.8155  4.2790  7.8838
##
## ==============================
## Processing: 00008_MST_task_2025-11-10_16h35.03.177.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7327  1.6381  2.6925  3.1084  4.0454 13.4479
##
## ==============================
## Processing: 00009_MST_task_2025-11-11_06h02.51.248.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.4543  0.9149  1.3460  1.5203  1.9511  8.3889       2
##
## ==============================
## Processing: 00010_MST_task_2025-11-14_12h10.04.262.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5018  0.8432  1.1106  1.3102  1.6234  4.9179
##
## ==============================
## Processing: 00011_MST_task_2025-11-14_21h11.30.103.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3923  0.8206  1.4085  1.7065  2.0638 10.3256
##
## ==============================
## Processing: 00012_MST_task_2025-11-15_10h50.12.984.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.8457  1.6957  2.0788  2.0814  2.3952  5.3271       2
##
## ==============================
## Processing: 00013_MST_task_2025-11-15_17h25.43.188.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5754  1.1634  1.6684  1.9714  2.3783  9.0837
##
## ==============================
## Processing: 00014_MST_task_2025-11-17_15h22.12.288.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5373  0.8863  1.1286  1.3074  1.5584  4.1845
##
## ==============================
## Processing: 00015_MST_task_2025-11-17_18h05.33.650.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.7534  1.2123  1.6633  1.8845  2.1480 10.7979       3
##
## ==============================
## Processing: 00016_MST_task_2025-11-18_15h18.28.519.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5740  0.8062  1.0220  1.2232  1.4991  3.6943
##
## ==============================
## Processing: 00017_MST_task_2026-01-20_12h09.25.940.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6551  1.2097  1.6449  1.7345  2.1798  3.8996
##
## ==============================
## Processing: 00018_MST_task_2026-01-20_14h57.56.200.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5314  0.8879  1.3767  1.4799  1.9410  3.7343
##
## ==============================
## Processing: 00019_MST_task_2026-01-20_15h19.21.867.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5962  1.1430  1.5830  1.7808  2.1462  9.6101       1
##
## ==============================
## Processing: 00020_MST_task_2026-01-20_16h14.02.894.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5415  0.9524  1.2923  1.4984  1.8426  5.3882
##
## ==============================
## Processing: 00021_MST_task_2026-01-20_15h54.11.031.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6463  1.2628  1.8033  2.0745  2.6125  5.9827       1
##
## ==============================
## Processing: 00022_MST_task_2026-01-20_16h50.51.424.csv
## Initial rows: 295
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.025   2.276   2.981   2.967   3.484   6.217
##
## ==============================
## Processing: 00023_MST_task_2026-01-20_17h20.47.244.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6047  0.8864  1.1734  1.4177  1.6693  4.6603
##
## ==============================
## Processing: 00024_MST_task_2026-01-20_17h45.28.652.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5442  0.8750  1.2153  1.3962  1.7328  4.3142
##
## ==============================
## Processing: 00025_MST_task_2026-01-20_19h03.31.875.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5968  0.9503  1.2674  1.5365  1.8756  4.7040
##
## ==============================
## Processing: 00026_MST_task_2026-01-20_19h20.16.777.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5751  0.9600  1.1956  1.3106  1.6442  2.8726
##
## ==============================
## Processing: 00027_MST_task_2026-01-20_19h47.53.224.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3616  0.7327  0.9392  1.1411  1.3417  4.0313
##
## ==============================
## Processing: 00028_MST_task_2026-01-20_21h43.31.019.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6706  1.1152  1.5127  1.7850  2.2959  4.6690
##
## ==============================
## Processing: 00029_MST_task_2026-01-21_11h19.06.023.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5457  1.0141  1.3719  1.6131  2.1226  4.0442
##
## ==============================
## Processing: 00030_MST_task_2026-01-21_11h23.46.765.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6066  0.9979  1.2831  1.4395  1.7965  5.0134       1
##
## ==============================
## Processing: 00031_MST_task_2026-01-21_14h49.42.484.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.581   1.102   1.662   1.993   2.355   7.001
##
## ==============================
## Processing: 00032_MST_task_2026-01-22_10h59.30.259.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6007  0.8774  1.0715  1.2244  1.4657  3.0155
##
## ==============================
## Processing: 00033_MST_task_2026-01-22_11h03.27.713.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.04275 1.47082 2.02510 2.05088 2.62423 4.43832
##
## ==============================
## Processing: 00034_MST_task_2026-01-22_17h23.05.207.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6033  1.1179  1.5429  1.7866  2.2448  5.6081
##
## ==============================
## Processing: 00035_MST_task_2026-01-22_20h53.05.508.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6307  1.1160  1.9310  2.1548  3.3251  5.2396
##
## ==============================
## Processing: 00036_MST_task_2026-01-23_16h03.45.360.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6175  1.1427  1.7981  1.8054  2.3911  4.4543
##
## ==============================
## Processing: 00037_MST_task_2026-01-23_16h05.43.404.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5520  0.8911  1.2564  1.6938  2.1093  8.7671       2
##
## ==============================
## Processing: 00038_MST_task_2026-01-23_16h36.46.044.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4886  0.6661  0.8223  1.0218  1.2004  4.1921
##
## ==============================
## Processing: 00039_MST_task_2026-01-23_17h02.47.129.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.4743  0.8196  1.1521  1.4217  1.6455  4.7808       1
##
## ==============================
## Processing: 00040_MST_task_2026-01-23_17h20.31.026.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5360  0.8811  1.1251  1.1784  1.3995  2.2590
##
## ==============================
## Processing: 00041_MST_task_2026-01-23_18h51.27.276.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.625   1.303   1.762   2.051   2.487   6.489
##
## ==============================
## Processing: 00042_MST_task_2026-01-23_21h18.31.909.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5668  1.0577  1.4606  1.5255  1.8933  4.2830
##
## ==============================
## Processing: 00043_MST_task_2026-01-23_21h19.14.514.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5462  0.8020  1.1390  1.3505  1.7292  4.3861       1
##
## ==============================
## Processing: 00044_MST_task_2026-01-24_10h56.04.295.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6754  1.3654  1.7623  1.9463  2.3904  6.7030       2
##
## ==============================
## Processing: 00045_MST_task_2026-01-24_11h29.01.648.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.538   0.805   1.014   1.231   1.569   3.656
##
## ==============================
## Processing: 00046_MST_task_2026-01-24_12h31.51.771.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5375  0.8253  1.1190  1.5074  1.7037  8.4833
##
## ==============================
## Processing: 00047_MST_task_2026-01-24_14h59.12.126.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6932  1.0283  1.5194  1.7506  2.2462  4.5093
##
## ==============================
## Processing: 00048_MST_task_2026-01-24_15h11.48.563.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6451  1.9187  3.2723  2.8198  3.4930  8.1009
##
## ==============================
## Processing: 00049_MST_task_2026-01-25_11h05.12.347.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6169  1.1874  1.7199  1.9378  2.3378 10.1816       1
##
## ==============================
## Processing: 00050_MST_task_2026-01-25_11h21.39.278.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##  0.04449  3.32701  3.47464  3.85988  3.95429 10.13405
##
## ==============================
## Processing: 00051_MST_task_2026-01-25_12h15.06.091.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.4981  0.9933  1.3107  1.5686  1.9104  7.6743       3
##
## ==============================
## Processing: 00052_MST_task_2026-01-25_12h44.06.631.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.2210  0.7779  1.2361  1.6095  2.3198  7.1487
##
## ==============================
## Processing: 00053_MST_task_2026-01-25_14h59.52.341.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7262  1.6140  2.0332  2.1689  2.6173  6.4417
##
## ==============================
## Processing: 00054_MST_task_2026-01-25_17h08.49.872.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   0.638   1.184   1.767   2.063   2.807   7.733       2
##
## ==============================
## Processing: 00055_MST_task_2025-11-21_16h42.03.893.csv
```

```
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5200  0.8787  1.1465  1.4247  1.8596  5.5670
##
## ==============================
## Processing: 00056_MST_task_2026-01-21_17h29.13.240.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4466  0.9596  1.3514  1.5741  1.9967  5.1689
##
## ==============================
## Processing TEST: 00001_MST_test_2025-11-08_17h07.29.968.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00002_MST_test_2025-11-08_19h19.48.470.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00003_MST_test_2025-11-08_22h47.34.525.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00004_MST_test_2025-11-09_14h57.36.500.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00005_MST_test_2025-11-09_17h45.24.630.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
```

```
##     30     60     60
##
## ==============================
## Processing TEST: 00006_MST_test_2025-11-09_19h53.31.018.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00007_MST_test_2025-11-20_17h14.07.925.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00008_MST_test_2025-11-10_17h06.27.688.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00009_MST_test_2025-11-11_06h27.17.578.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00010_MST_test_2025-11-14_18h08.55.599.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00011_MST_test_2025-11-14_21h38.18.508.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00012_MST_test_2025-11-15_11h16.55.438.csv
## Rows: 150
## Stimulus counts:
```

```
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00013_MST_test_2025-11-15_17h55.05.651.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00014_MST_test_2025-11-17_16h00.28.058.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00015_MST_test_2025-11-17_18h33.40.554.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00016_MST_test_2025-11-18_15h57.27.139.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00017_MST_test_2026-01-20_12h47.41.536.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00018_MST_test_2026-01-20_15h46.27.618.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00019_MST_test_2026-01-20_15h44.09.765.csv
```

```
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00020_MST_test_2026-01-20_16h39.03.637.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00021_MST_test_2026-01-20_17h10.36.612.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00022_MST_test_2026-01-20_17h30.21.472.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00023_MST_test_2026-01-20_17h47.38.675.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00024_MST_test_2026-01-20_18h09.57.673.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00025_MST_test_2026-01-20_19h32.59.931.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
```

```
## ==============================
## Processing TEST: 00026_MST_test_2026-01-20_19h45.56.914.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00027_MST_test_2026-01-20_20h11.54.094.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00028_MST_test_2026-01-20_22h09.36.854.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00029_MST_test_2026-01-21_11h42.53.652.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00030_MST_test_2026-01-21_11h51.29.080.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00031_MST_test_2026-01-21_15h25.55.946.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00032_MST_test_2026-01-22_11h26.12.622.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
```

```
##     30     60     60
##
## ==============================
## Processing TEST: 00033_MST_test_2026-01-22_11h28.01.006.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00034_MST_test_2026-01-22_17h48.05.945.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00035_MST_test_2026-01-22_21h18.33.812.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00036_MST_test_2026-01-23_16h28.22.558.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00037_MST_test_2026-01-23_16h33.26.226.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00038_MST_test_2026-01-23_17h00.07.805.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00039_MST_test_2026-01-23_17h27.51.140.csv
## Rows: 150
## Stimulus counts:
```

```
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00040_MST_test_2026-01-23_17h44.12.035.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00041_MST_test_2026-01-23_19h17.51.467.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00042_MST_test_2026-01-23_22h06.21.633.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00043_MST_test_2026-01-23_22h08.45.877.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00044_MST_test_2026-01-24_11h25.58.400.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00045_MST_test_2026-01-24_11h54.00.474.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00046_MST_test_2026-01-24_12h57.45.954.csv
```

```
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00047_MST_test_2026-01-24_15h24.10.813.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00048_MST_test_2026-01-24_15h36.35.684.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00049_MST_test_2026-01-25_11h31.50.122.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00050_MST_test_2026-01-25_12h06.10.875.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00051_MST_test_2026-01-25_12h41.05.232.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00052_MST_test_2026-01-25_13h08.47.205.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
```

```
## ==============================
## Processing TEST: 00053_MST_test_2026-01-25_15h25.49.781.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00054_MST_test_2026-01-25_17h34.48.535.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00055_MST_test_2025-11-21_17h08.33.541.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00056_MST_test_2026-01-21_17h53.13.920.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
```

```r
exp3 <- load_experiment(
  "../MST_Data/task_only/task_only_data",
  "../MST_Data/task_only/task_only_data",
  "task_only"
)
```

```
##
## ==============================
## LOADING EXPERIMENT: task_only
## Task files: 53
## Test files: 53
##
## ==============================
## Processing: 00019_MST_task_2025-11-08_12h25.21.130.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7666  1.5048  1.8866  1.9692  2.3447  4.4536
##
## ==============================
```

```
## Processing: 00020_MST_task_2025-11-08_18h04.13.718.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7905  1.3065  1.7921  1.8969  2.3371  5.4022
##
## ============================
## Processing: 00021_MST_task_2025-11-08_21h31.55.785.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.7974  1.4840  2.0036  2.2741  2.7488  8.4050       1
##
## ============================
## Processing: 00022_MST_task_2025-11-09_11h49.42.417.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6965  1.1821  1.4553  1.6205  1.9489  6.4875
##
## ============================
## Processing: 00023_MST_task_2025-11-09_16h10.25.535.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.5634  1.3633  1.9563  2.0907  2.5627  8.7918       2
##
## ============================
## Processing: 00024_MST_task_2025-11-09_18h45.42.464.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5344  0.9319  1.4974  2.0674  2.5489 22.1454
##
## ============================
## Processing: 00025_MST_task_2025-11-10_15h37.05.281.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.8368  1.7305  2.1453  2.2338  2.7436  5.3791       1
##
## ============================
```

```
## Processing: 00026_MST_task_2025-11-11_05h26.46.426.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.8117  1.7454  2.2619  2.4240  2.7721  6.7327
##
## ============================
## Processing: 00027_MST_task_2025-11-14_16h15.34.724.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.360   3.395   3.571   3.562   3.753   9.231
##
## ============================
## Processing: 00028_MST_task_2025-11-14_19h03.32.428.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6279  0.9738  1.2172  1.3456  1.5997  5.0433
##
## ============================
## Processing: 00029_MST_task_2025-11-15_11h29.12.994.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7595  1.2883  1.7210  1.9222  2.3875  6.4804
##
## ============================
## Processing: 00030_MST_task_2025-11-15_18h08.36.625.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   1.005   2.002   2.495   2.674   3.198   6.551       2
##
## ============================
## Processing: 00032_MST_task_2026-01-16_11h31.50.282.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6801  1.1865  1.5882  1.6710  2.0650  9.6307
##
## ============================
```

```
## Processing: 00033_MST_task_2025-11-18_14h44.49.862.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6139  0.9782  1.2901  1.6160  1.8788  7.6041
##
## ============================
## Processing: 00034_MST_task_2026-01-16_12h47.55.482.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6896  1.4374  1.8880  2.0172  2.3894  8.5712
##
## ============================
## Processing: 00035_MST_task_2026-01-16_14h49.08.296.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.7845  1.6100  2.4147  2.8430  3.6645 10.2844       1
##
## ============================
## Processing: 00036_MST_task_2026-01-16_22h07.58.999.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6295  1.0957  1.4766  1.6027  1.9102  4.6155       1
##
## ============================
## Processing: 00037_MST_task_2026-01-17_10h41.49.026.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6069  0.9364  1.2173  1.3636  1.6293  5.0346
##
## ============================
## Processing: 00038_MST_task_2026-01-17_11h13.54.018.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   1.065   1.934   2.515   2.607   3.217   7.566       1
##
## ============================
```

```
## Processing: 00039_MST_task_2026-01-17_11h27.57.476.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6501  1.2194  1.6876  1.8387  2.2674  6.1330
##
## ============================
## Processing: 00040_MST_task_2026-01-17_15h03.47.022.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7477  1.2806  1.6609  1.7639  2.1105  7.6776
##
## ============================
## Processing: 00041_MST_task_2026-01-17_15h38.16.978.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6274  1.0086  1.4195  1.5709  1.9083  4.3817       1
##
## ============================
## Processing: 00042_MST_task_2026-01-17_16h29.52.993.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6315  1.6741  2.3426  2.5639  3.2905 10.0683       2
##
## ============================
## Processing: 00043_MST_task_2026-01-17_16h59.39.804.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7669  1.4679  2.0876  2.2601  2.8839  4.7689
##
## ============================
## Processing: 00044_MST_task_2026-01-17_17h50.12.146.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.2414  1.5877  2.1693  2.4301  3.1122  6.6106
##
## ============================
```

```
## Processing: 00045_MST_task_2026-01-18_10h35.39.680.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7989  1.3949  1.7569  1.8963  2.2703  4.6392
##
## ============================
## Processing: 00046_MST_task_2026-01-18_11h15.02.395.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7636  1.9085  2.6034  3.1264  4.0184 11.8064
##
## ============================
## Processing: 00047_MST_task_2026-01-18_12h01.27.453.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.8262  1.2560  1.7208  2.1079  2.5211  8.6137
##
## ============================
## Processing: 00048_MST_task_2026-01-18_12h44.08.666.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6704  1.2355  1.6959  1.8846  2.3272  4.9909
##
## ============================
## Processing: 00049_MST_task_2026-01-18_15h14.33.413.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   1.144   2.541   3.276   2.998   3.366   4.648       2
##
## ============================
## Processing: 00050_MST_task_2026-01-18_16h12.54.357.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.8941  1.4390  1.8672  1.9403  2.2132  5.1410
##
## ============================
```

```
## Processing: 00051_MST_task_2026-01-18_16h47.53.463.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.9383  2.1013  3.1296  3.2046  3.9843  7.8579       1
##
## ============================
## Processing: 00052_MST_task_2026-01-18_17h28.15.143.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.9911  2.9015  3.2570  3.2251  3.3731 10.8347       1
##
## ============================
## Processing: 00053_MST_task_2026-01-18_18h34.57.020.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.8772  1.6380  2.1474  2.3876  2.8791  8.8562
##
## ============================
## Processing: 00054_MST_task_2026-01-18_18h59.11.419.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.4389  1.0336  1.6579  2.1623  2.7240 26.5536       4
##
## ============================
## Processing: 00055_MST_task_2026-01-18_21h39.30.446.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6033  1.4427  2.1167  2.5497  3.5607  7.0714       1
##
## ============================
## Processing: 00056_MST_task_2026-01-19_10h30.41.083.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.725   1.258   1.554   1.580   1.804   3.123
##
## ============================
```

```
## Processing: 00057_MST_task_2026-01-19_11h17.15.155.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5594  1.1123  1.5140  1.5707  1.8597  5.7598
##
## ============================
## Processing: 00058_MST_task_2026-01-19_15h01.44.304.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7033  1.3648  1.7492  1.9751  2.4423  6.1402
##
## ============================
## Processing: 00059_MST_task_2026-01-19_16h24.22.614.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.09717 3.24563 3.28087 3.35781 3.38361 5.86337
##
## ============================
## Processing: 00060_MST_task_2026-01-19_17h07.13.071.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.576   1.070   1.546   1.833   2.286   5.714
##
## ============================
## Processing: 00061_MST_task_2026-01-19_17h44.07.535.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.8428  1.4920  2.0434  2.2421  2.7645  6.3939       4
##
## ============================
## Processing: 00062_MST_task_2026-01-19_18h25.43.917.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6571  1.2105  1.6547  1.9172  2.3390  8.2587
##
## ============================
```

```
## Processing: 00063_MST_task_2026-01-19_21h58.30.655.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6938  1.0833  1.4003  1.4453  1.7080  2.9537
##
## ============================
## Processing: 00064_MST_task_2026-01-20_10h40.57.117.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.019   1.987   2.522   2.703   3.229   6.447
##
## ============================
## Processing: 00065_MST_task_2026-01-20_11h49.32.082.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.6687  1.2136  1.5889  1.8031  2.1893  7.2914       1
##
## ============================
## Processing: 00066_MST_task_2026-01-16_10h19.28.668.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.4931  0.8906  1.1911  1.3622  1.6219  6.0745
##
## ============================
## Processing: 00067_MST_task_2025-11-17_18h52.18.098.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.091   2.133   2.983   2.990   3.533   9.461
##
## ============================
## Processing: 00068_MST_task_2026-01-16_22h41.10.264.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7807  1.5544  1.8883  1.9407  2.2882  2.9674
##
## ============================
```

```
## Processing: 00069_MST_task_2026-01-25_15h44.35.264.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6992  1.5914  2.1894  2.4567  3.0810 10.6523
##
## ============================
## Processing: 00070_MST_task_2026-01-16_12h36.17.636.csv
## Initial rows: 295
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6265  1.1012  1.3615  1.4967  1.7597  3.0388
##
## ============================
## Processing: 00071_MST_task_2025-11-17_18h52.18.098.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.091   2.133   2.983   2.990   3.533   9.461
##
## ============================
## Processing: 00072_MST_task_2026-01-18_11h30.21.482.csv
## Initial rows: 296
## Rows after cleaning: 280
## Unique images: 280
## RT summary:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   0.777   2.147   3.083   2.831   3.393   5.253       2
##
## ============================
## Processing TEST: 00019_MST_test_2025-11-08_12h49.14.340.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00020_MST_test_2025-11-08_18h29.24.405.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ============================
## Processing TEST: 00021_MST_test_2025-11-08_21h59.32.852.csv
## Rows: 150
```

```
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00022_MST_test_2025-11-09_12h15.12.382.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00023_MST_test_2025-11-09_16h38.05.927.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00024_MST_test_2025-11-09_19h14.07.408.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00025_MST_test_2025-11-10_16h04.32.749.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00026_MST_test_2025-11-11_05h50.27.008.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00027_MST_test_2025-11-14_17h03.55.071.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
```

```
## Processing TEST: 00028_MST_test_2025-11-14_19h28.58.680.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00029_MST_test_2025-11-15_11h52.06.081.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00030_MST_test_2025-11-15_18h38.22.342.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00032_MST_test_2026-01-16_12h17.18.732.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00033_MST_test_2025-11-18_15h10.05.046.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00034_MST_test_2026-01-16_13h12.28.236.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00035_MST_test_2026-01-16_15h26.14.383.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
```

```
##
## ==============================
## Processing TEST: 00036_MST_test_2026-01-16_22h32.11.646.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00037_MST_test_2026-01-17_11h06.07.125.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00038_MST_test_2026-01-17_11h39.48.495.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00039_MST_test_2026-01-17_12h10.16.109.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00040_MST_test_2026-01-17_15h41.52.844.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00041_MST_test_2026-01-17_16h10.49.006.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00042_MST_test_2026-01-17_16h56.53.653.csv
## Rows: 150
## Stimulus counts:
##
```

```
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00043_MST_test_2026-01-17_17h25.10.870.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00044_MST_test_2026-01-17_18h25.01.445.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00045_MST_test_2026-01-18_11h00.57.957.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00046_MST_test_2026-01-18_11h48.54.013.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00047_MST_test_2026-01-18_12h29.07.522.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00048_MST_test_2026-01-18_13h09.21.414.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00049_MST_test_2026-01-18_15h51.26.110.csv
## Rows: 150
```

```
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00050_MST_test_2026-01-18_16h37.27.121.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00051_MST_test_2026-01-18_17h15.14.453.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00052_MST_test_2026-01-18_18h26.03.402.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00053_MST_test_2026-01-18_19h05.55.236.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00054_MST_test_2026-01-18_19h26.53.815.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00055_MST_test_2026-01-18_22h10.40.713.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
```

```
## Processing TEST: 00056_MST_test_2026-01-19_11h07.07.462.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00057_MST_test_2026-01-19_11h48.50.532.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00058_MST_test_2026-01-19_15h27.36.538.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00059_MST_test_2026-01-19_16h49.56.647.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00060_MST_test_2026-01-19_17h34.35.123.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00061_MST_test_2026-01-19_18h08.07.496.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00062_MST_test_2026-01-19_19h46.43.275.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
```

```
##
## ==============================
## Processing TEST: 00063_MST_test_2026-01-19_22h22.35.848.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00064_MST_test_2026-01-20_11h50.51.345.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00065_MST_test_2026-01-20_12h13.42.600.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00066_MST_test_2026-01-16_11h24.31.717.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00067_MST_test_2025-11-17_19h45.14.102.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00068_MST_test_2026-01-16_23h19.04.686.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00069_MST_test_2026-01-25_16h10.30.795.csv
## Rows: 150
## Stimulus counts:
##
```

```
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00070_MST_test_2026-01-16_13h00.56.755.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00071_MST_test_2025-11-17_19h45.14.102.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
##
## ==============================
## Processing TEST: 00072_MST_test_2026-01-18_11h57.30.393.csv
## Rows: 150
## Stimulus counts:
##
##   Foil   Lure Target
##     30     60     60
```

```r
# Combine
task_all <- bind_rows(exp1$task, exp2$task, exp3$task)
test_all <- bind_rows(exp1$test, exp2$test, exp3$test)

cat("\nFINAL DATA SUMMARY\n")
```

```
##
## FINAL DATA SUMMARY
```

```r
cat("Total task rows:", nrow(task_all), "\n")
```

```
## Total task rows: 45080
```

```r
cat("Total test rows:", nrow(test_all), "\n")
```

```
## Total test rows: 24150
```

```r
# SANITY CHECK
head(task_all)
```

```
## # A tibble: 6 x 6
##   participant_id trial_number image_path          image_name       rt experiment
##   <chr>                 <dbl> <chr>               <chr>         <dbl> <chr>
## 1 00015                     1 "Objects\\060a.jpg" "Objects\\06~  6.63 both
```

```
## 2 00015                     2 "Objects\\105a.jpg" "Objects\\10~  2.50 both
## 3 00015                     3 "Objects\\112a.jpg" "Objects\\11~  2.13 both
## 4 00015                     4 "Objects\\150a.jpg" "Objects\\15~  1.37 both
## 5 00015                     5 "Objects\\006a.jpg" "Objects\\00~  2.09 both
## 6 00015                     6 "Objects\\122a.jpg" "Objects\\12~  2.13 both
```

```r
summary(task_all$rt)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.     NA's
##  0.03741  1.09604  1.61911  1.90480  2.41883 26.55363       65
```

```r
table(test_all$stimulus_type)
```

```
##
##   Foil   Lure Target
##   4830   9660   9660
```

VISUALISATION

HOW TO USE THESE (IMPORTANT) Check these before moving forward: Reaction Time Right skew? → expected Extreme outliers? → consider trimming/log transform Response Distribution Balanced responses? Too many "Old"? → bias issue Stimulus Types Should roughly match: Accuracy / Hit Rate Should be above chance (~0.5) If not → data quality issue LDI / REC Should be > 0 (as per MST theory)

```r
cat("\n================ VISUALIZATION PIPELINE ================\n")
```

```
##
## ================ VISUALIZATION PIPELINE ================
```

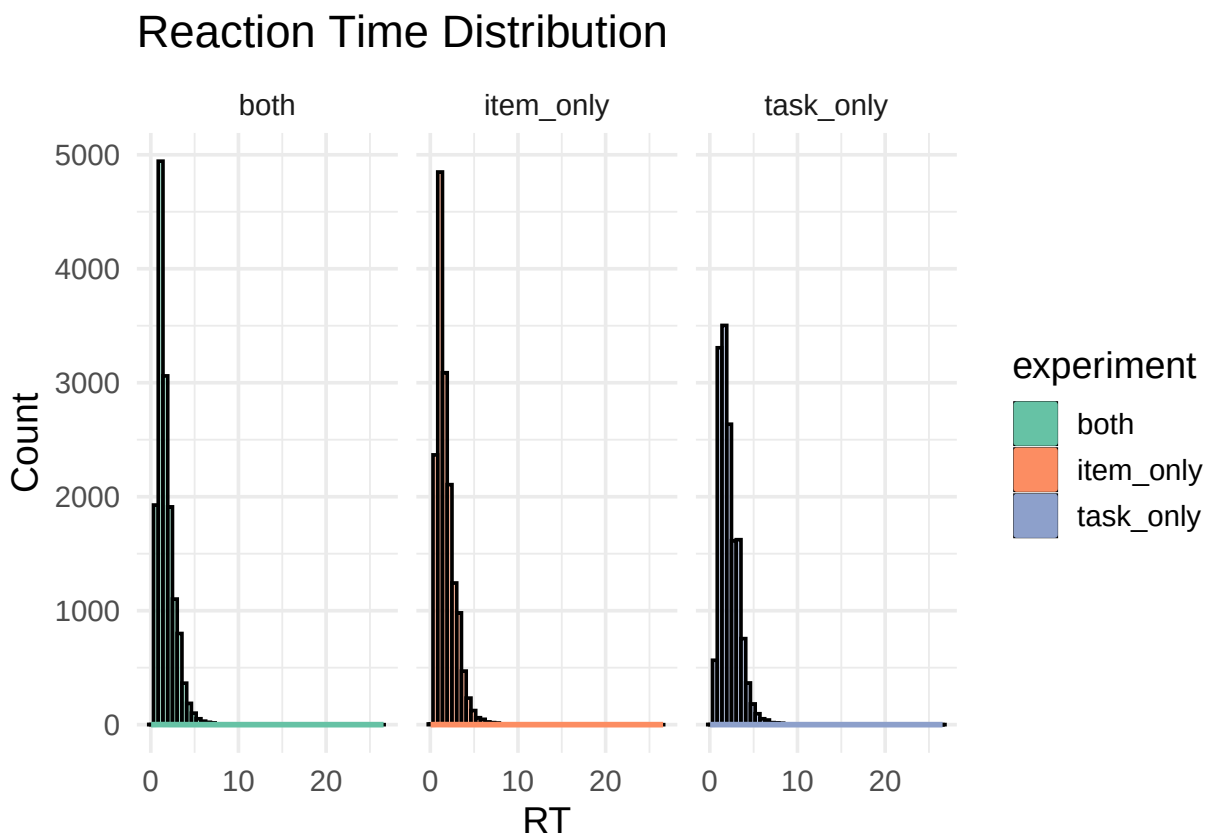
```r
# ----------------------------------------------------------
# 1. FIX STIMULUS CLASSIFICATION (IMPORTANT FIX)
# ----------------------------------------------------------
test_all_fixed <- test_all %>%
  mutate(
    stimulus_type = case_when(
      str_detect(tolower(image_path), "^foil") ~ "Foil",
      str_detect(image_name, "a\\.jpg$") ~ "Target",
      str_detect(image_name, "b\\.jpg$") ~ "Lure",
      TRUE ~ NA_character_
    )
  )

cat("\nStimulus distribution AFTER FIX:\n")
```

```
##
## Stimulus distribution AFTER FIX:
```

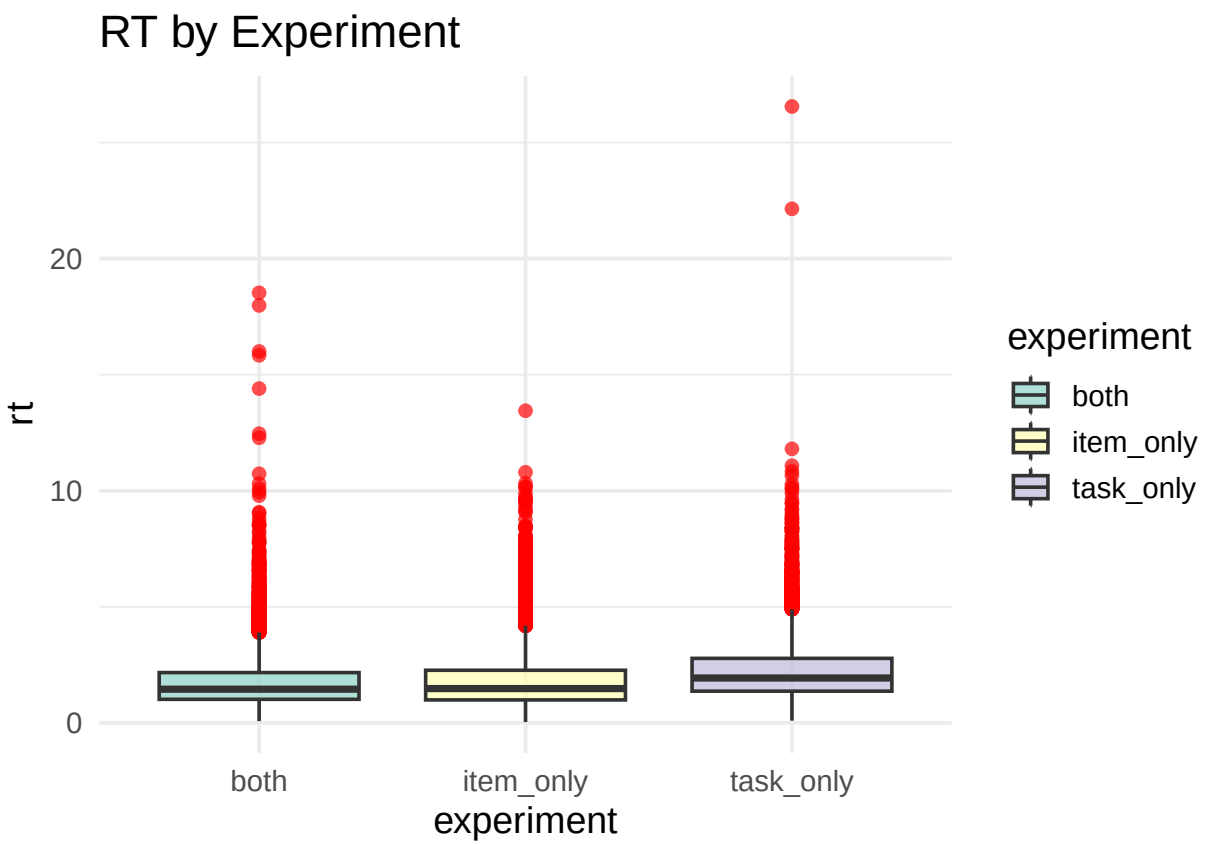
```r
print(table(test_all_fixed$stimulus_type, useNA = "always"))
```

```
##
##   Foil   Lure Target   <NA>
##   4830   9660   9660      0
```

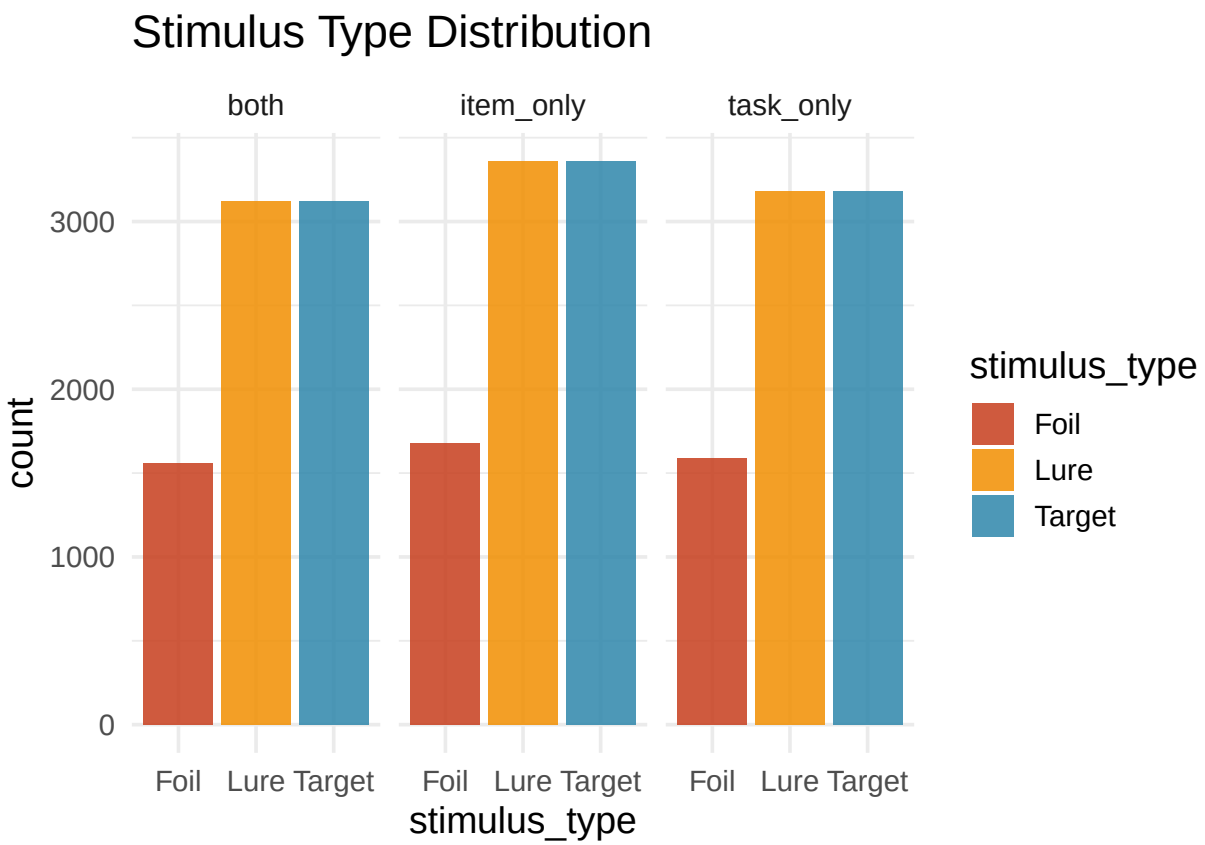
```r
# ----------------------------------------------------------
# 2. CLEAN DATA (REMOVE INVALID)
# ----------------------------------------------------------
task_clean <- task_all %>%
  filter(!is.na(rt), is.finite(rt))

test_clean <- test_all_fixed %>%
  filter(!is.na(stimulus_type))

# ----------------------------------------------------------
# Custom Color Palette
# ----------------------------------------------------------
palette_main <- c(
  "Target" = "#2E86AB",
  "Lure"   = "#F18F01",
  "Foil"   = "#C73E1D"
)

palette_resp <- c(
  "Old" = "#1B998B",
  "Similar" = "#ED217C",
  "New" = "#2D3047"
)

# ----------------------------------------------------------
# 3. REACTION TIME DISTRIBUTION
# ----------------------------------------------------------
ggplot(task_clean, aes(x = rt, fill = experiment)) +
  geom_histogram(bins = 50, alpha = 0.6, color = "black") +
  geom_density(aes(color = experiment), linewidth = 1) +
  facet_wrap(~experiment) +
  scale_fill_brewer(palette = "Set2") +
  scale_color_brewer(palette = "Set2") +
  labs(title = "Reaction Time Distribution", x = "RT", y = "Count") +
  theme_minimal(base_size = 14)
```



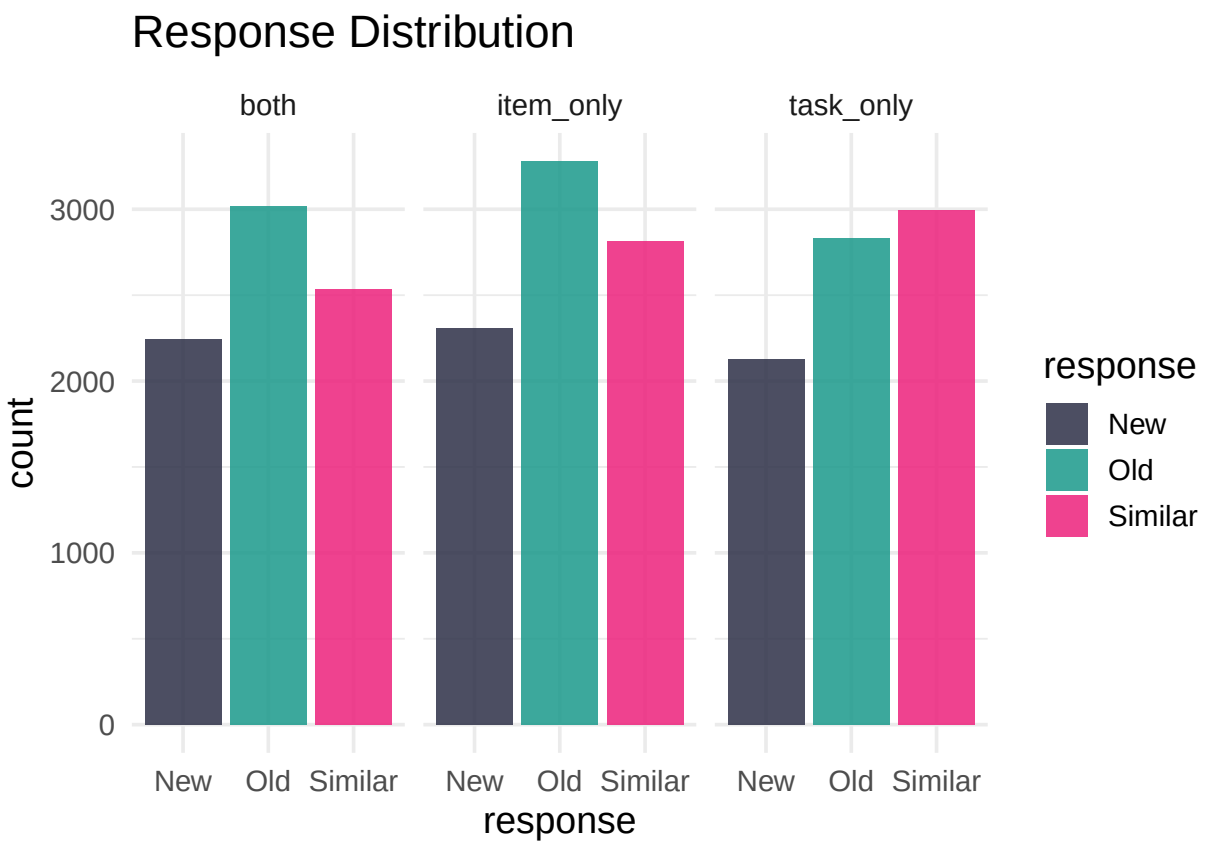

```r
# ----------------------------------------------------------
# 4. RT BOXPLOT
# ----------------------------------------------------------
ggplot(task_clean, aes(x = experiment, y = rt, fill = experiment)) +
  geom_boxplot(alpha = 0.7, outlier.color = "red") +
  scale_fill_brewer(palette = "Set3") +
  labs(title = "RT by Experiment") +
  theme_minimal(base_size = 14)
```



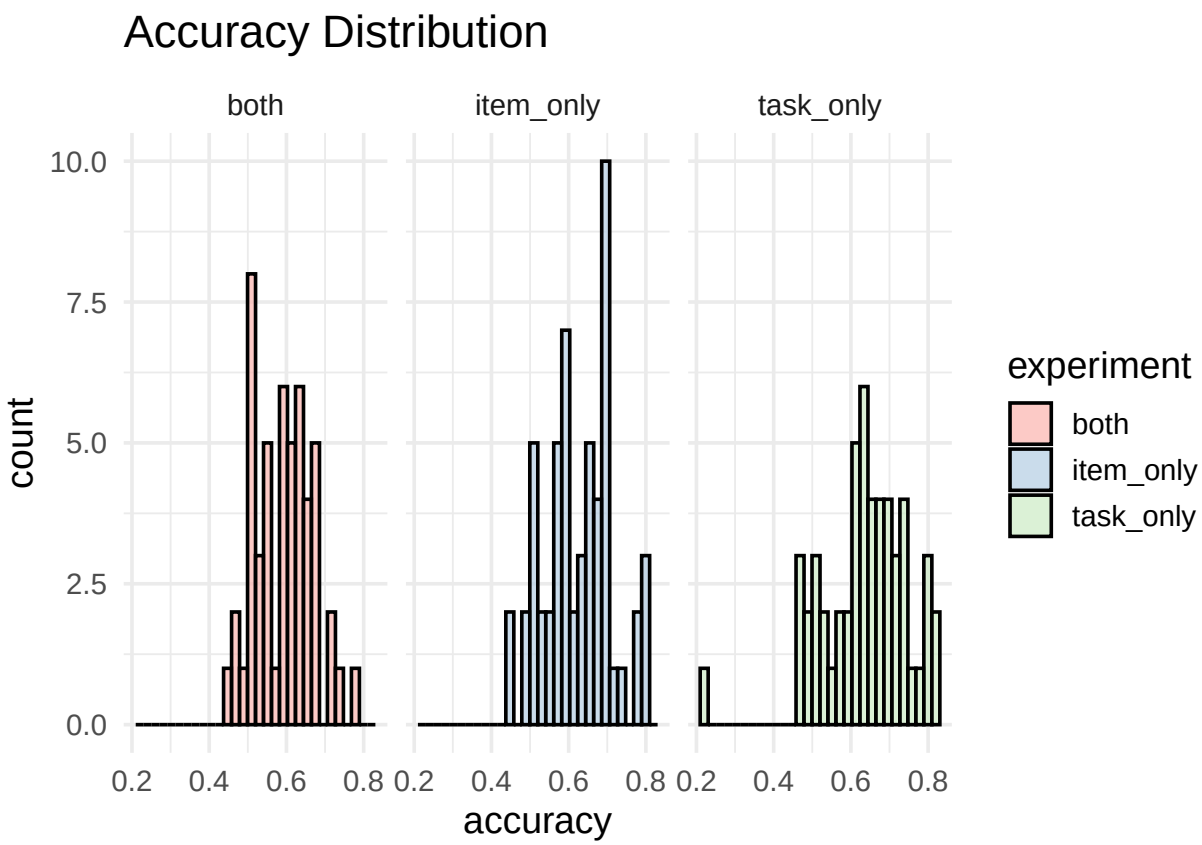

```
# ---------------------------------------------------------
# 5. STIMULUS TYPE DISTRIBUTION (FIXED)
# ---------------------------------------------------------
ggplot(test_clean, aes(x = stimulus_type, fill = stimulus_type)) +
  geom_bar(alpha = 0.85) +
  facet_wrap(~experiment) +
  scale_fill_manual(values = palette_main) +
  labs(title = "Stimulus Type Distribution") +
  theme_minimal(base_size = 14)
```



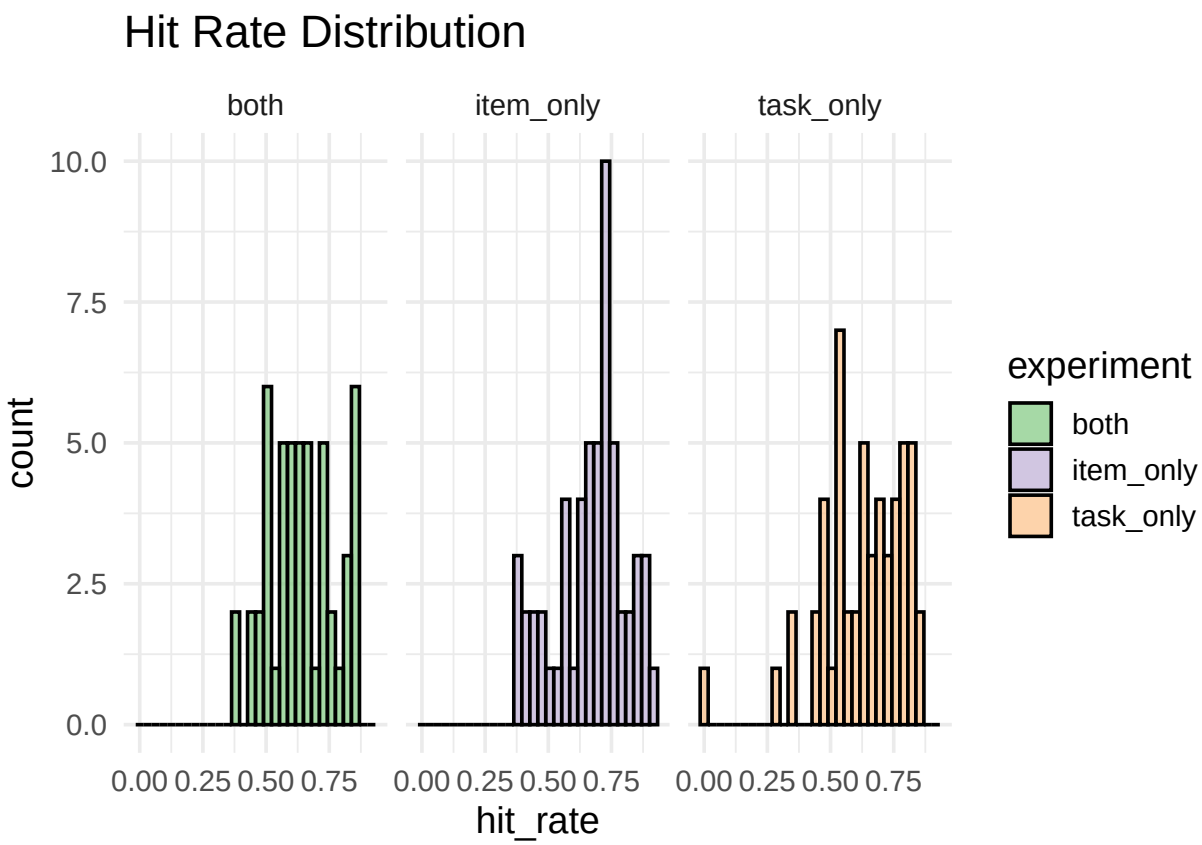

```
# ---------------------------------------------------------
# 6. RESPONSE DISTRIBUTION
# ---------------------------------------------------------
ggplot(test_clean, aes(x = response, fill = response)) +
  geom_bar(alpha = 0.85) +
  facet_wrap(~experiment) +
  scale_fill_manual(values = palette_resp) +
  labs(title = "Response Distribution") +
  theme_minimal(base_size = 14)
```




```
# ----------------------------------------------------------
# 7. ACCURACY
# ----------------------------------------------------------
acc_data <- test_clean %>%
  mutate(correct = case_when(
    stimulus_type == "Target" & response == "Old" ~ 1,
    stimulus_type == "Lure" & response == "Similar" ~ 1,
    stimulus_type == "Foil" & response == "New" ~ 1,
    TRUE ~ 0
  )) %>%
  group_by(participant_id, experiment) %>%
  summarise(accuracy = mean(correct), .groups = "drop")

ggplot(acc_data, aes(x = accuracy, fill = experiment)) +
  geom_histogram(bins = 30, alpha = 0.7, color = "black") +
  facet_wrap(~experiment) +
  scale_fill_brewer(palette = "Pastel1") +
  labs(title = "Accuracy Distribution") +
  theme_minimal(base_size = 14)
```

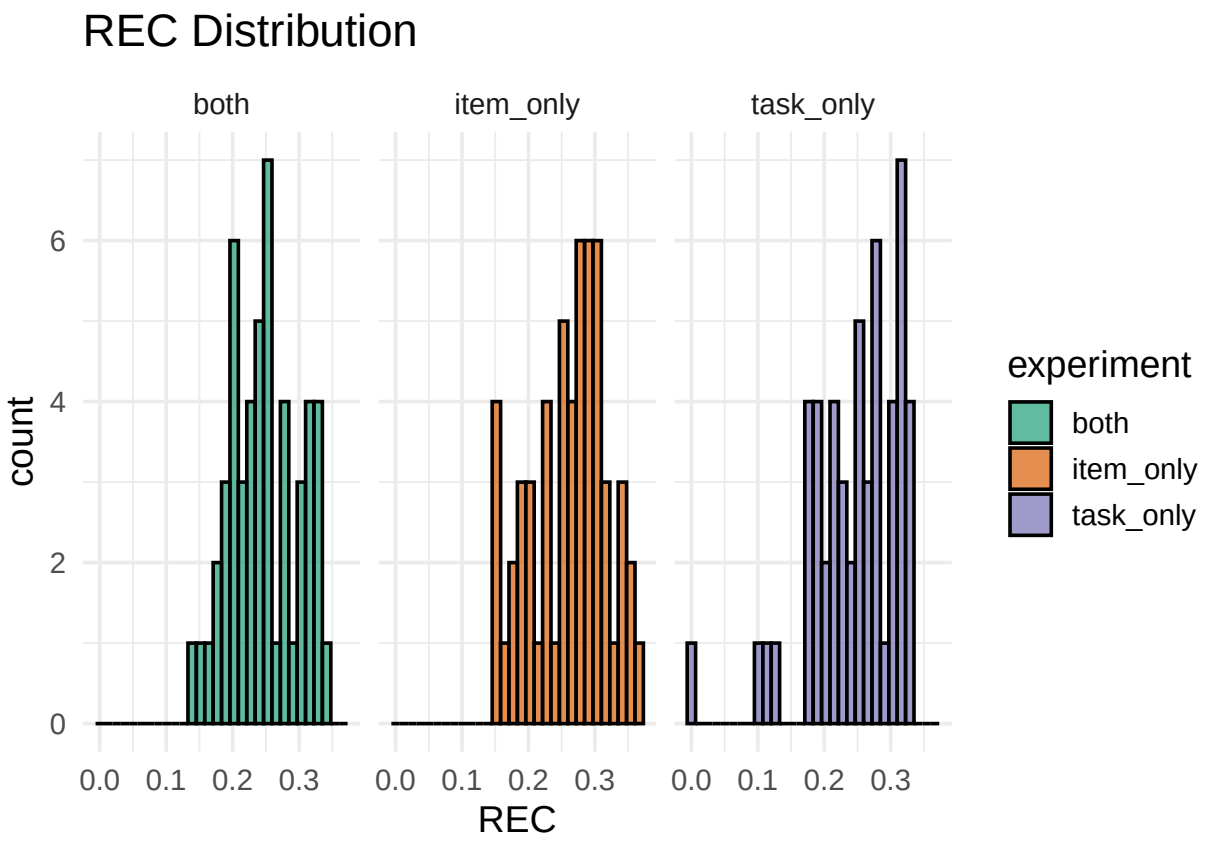



```r
# ---------------------------------------------------------
# 8. HIT RATE
# ---------------------------------------------------------
hit_data <- test_clean %>%
  filter(stimulus_type == "Target") %>%
  mutate(hit = ifelse(response == "Old", 1, 0)) %>%
  group_by(participant_id, experiment) %>%
  summarise(hit_rate = mean(hit), .groups = "drop")

ggplot(hit_data, aes(x = hit_rate, fill = experiment)) +
  geom_histogram(bins = 30, alpha = 0.7, color = "black") +
  facet_wrap(~experiment) +
  scale_fill_brewer(palette = "Accent") +
  labs(title = "Hit Rate Distribution") +
  theme_minimal(base_size = 14)
```

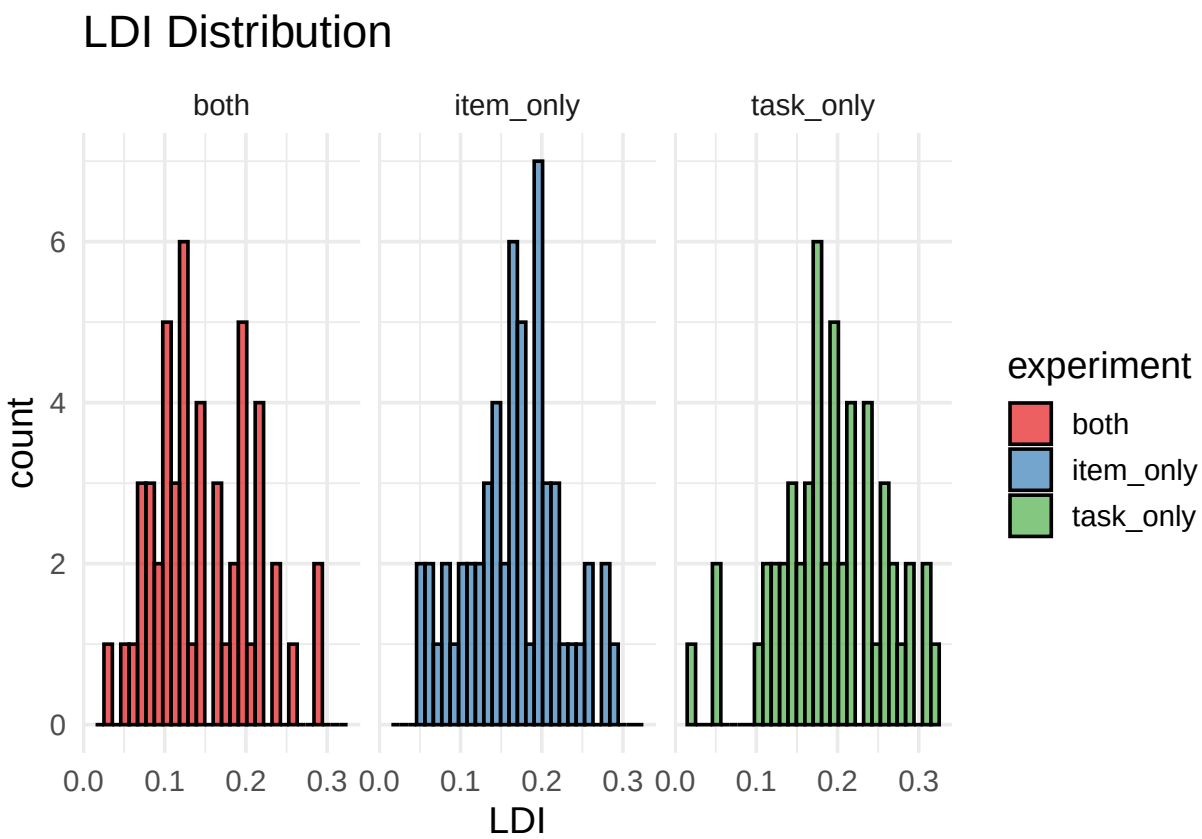



```r
# ---------------------------------------------------------
# 9. LDI & REC (SAFE COMPUTATION)
# ---------------------------------------------------------
safe_mean <- function(x){
  if(length(x) == 0) return(NA_real_)
  mean(x, na.rm = TRUE)
}

mst_metrics <- test_clean %>%
  group_by(participant_id, experiment) %>%
  summarise(
    p_old_target = safe_mean(stimulus_type == "Target" & response == "Old"),
    p_old_foil   = safe_mean(stimulus_type == "Foil" & response == "Old"),
    p_sim_lure   = safe_mean(stimulus_type == "Lure" & response == "Similar"),
    p_sim_foil   = safe_mean(stimulus_type == "Foil" & response == "Similar"),
    .groups = "drop"
  ) %>%
  mutate(
    REC = p_old_target - p_old_foil,
    LDI = p_sim_lure - p_sim_foil
  ) %>%
  filter(!is.na(REC), !is.na(LDI))

# REC
ggplot(mst_metrics, aes(x = REC, fill = experiment)) +
  geom_histogram(bins = 30, alpha = 0.7, color = "black") +
```

```r
  facet_wrap(~experiment) +
  scale_fill_brewer(palette = "Dark2") +
  labs(title = "REC Distribution") +
  theme_minimal(base_size = 14)
```




```r
# LDI
ggplot(mst_metrics, aes(x = LDI, fill = experiment)) +
  geom_histogram(bins = 30, alpha = 0.7, color = "black") +
  facet_wrap(~experiment) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "LDI Distribution") +
  theme_minimal(base_size = 14)
```




```
cat("\n=========== VISUALISATION COMPLETE ===========\n")
```

```
##
## =========== VISUALISATION COMPLETE ===========
```

NORMALITY TESTING

Normality tests are not performed on a single participant; they are applied to the distribution of values across participants. In this analysis, for each metric (reaction time, accuracy, hit rate, REC, LDI), we first computed a single summary value per participant (e.g., mean RT or LDI per condition), and then performed the Shapiro–Wilk test on the set of these participant-level values within each experiment. This is appropriate because the assumption of normality in statistical tests (like ANOVA or t-tests) applies to the sampling distribution across participants, not to individual trial data or a single participant's observations.

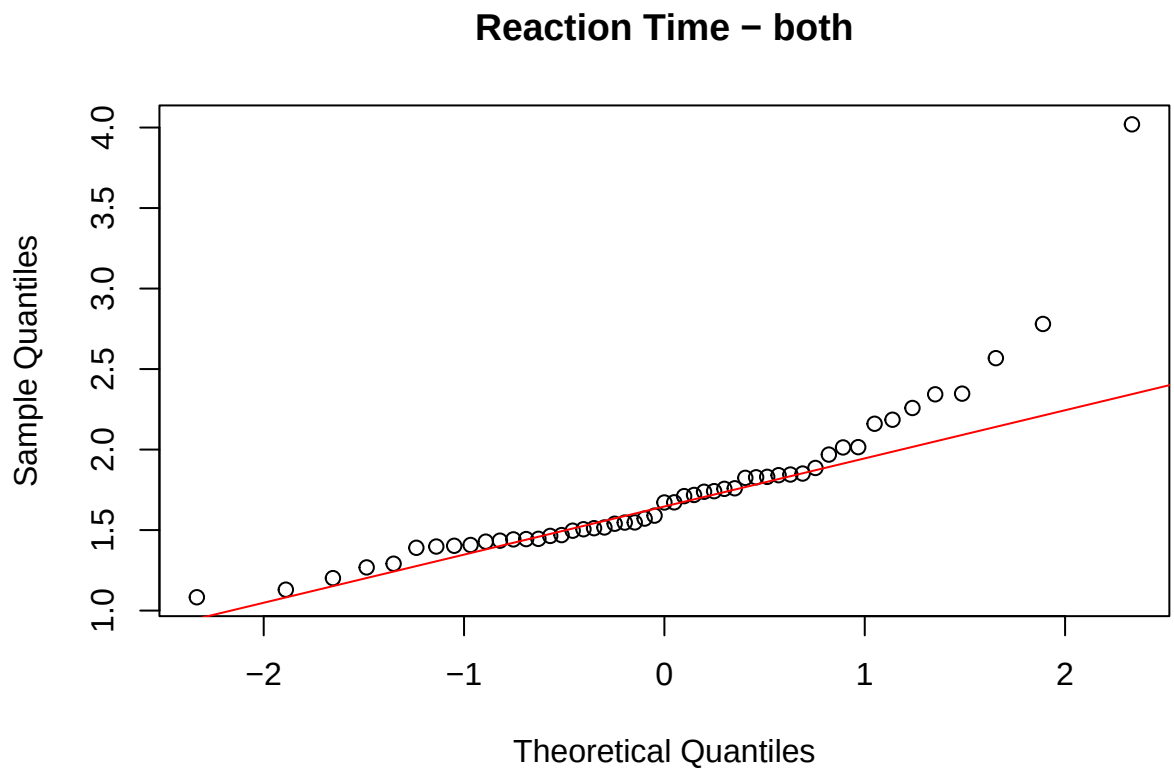
```
library(tidyverse)

cat("\n=========== NORMALITY TEST PIPELINE ===========\n")
```

```
##
## =========== NORMALITY TEST PIPELINE ===========
```

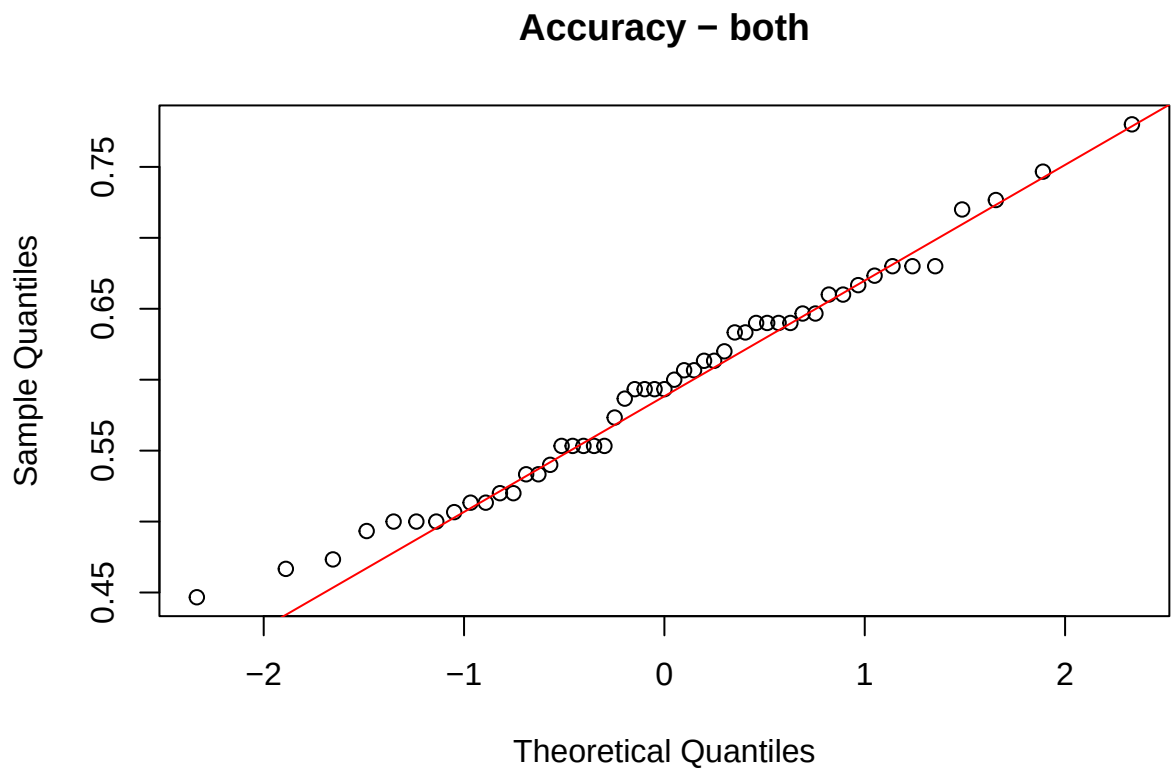
```r
# ----------------------------------------------------------
# HELPER FUNCTION
# ----------------------------------------------------------
run_normality <- function(values, label, exp){

  values <- values[!is.na(values)]

  cat("\n------------------------------\n")
  cat("Experiment:", exp, "\n")
  cat("Variable:", label, "\n")
  cat("Sample size:", length(values), "\n")

  if(length(values) < 3){
    cat("Not enough data\n")
    return(NULL)
  }

  print(shapiro.test(values))

  qqnorm(values, main = paste(label, "-", exp))
  qqline(values, col = "red")
}

# ----------------------------------------------------------
# LOOP OVER EXPERIMENTS
# ----------------------------------------------------------
for(exp in unique(test_clean$experiment)){

  cat("\n=====================================\n")
  cat("PROCESSING:", exp, "\n")
  cat("=====================================\n")

  task_exp <- task_clean %>% filter(experiment == exp)
  test_exp <- test_clean %>% filter(experiment == exp)

  # ---------------------------
  # RT
  # ---------------------------
  rt_summary <- task_exp %>%
    group_by(participant_id) %>%
    summarise(mean_rt = mean(rt), .groups = "drop")

  run_normality(rt_summary$mean_rt, "Reaction Time", exp)

  # ---------------------------
  # Accuracy
  # ---------------------------
  acc_summary <- test_exp %>%
    mutate(correct = case_when(
      stimulus_type == "Target" & response == "Old" ~ 1,
      stimulus_type == "Lure" & response == "Similar" ~ 1,
      stimulus_type == "Foil" & response == "New" ~ 1,
      TRUE ~ 0
```

```r
    )) %>%
    group_by(participant_id) %>%
    summarise(accuracy = mean(correct), .groups = "drop")

  run_normality(acc_summary$accuracy, "Accuracy", exp)

  # ---------------------------
  # Hit Rate
  # ---------------------------
  hit_summary <- test_exp %>%
    filter(stimulus_type == "Target") %>%
    mutate(hit = ifelse(response == "Old", 1, 0)) %>%
    group_by(participant_id) %>%
    summarise(hit_rate = mean(hit), .groups = "drop")

  run_normality(hit_summary$hit_rate, "Hit Rate", exp)

  # ---------------------------
  # LDI & REC
  # ---------------------------
  mst_metrics <- test_exp %>%
    group_by(participant_id) %>%
    summarise(
      p_old_target = mean(stimulus_type == "Target" & response == "Old"),
      p_old_foil   = mean(stimulus_type == "Foil" & response == "Old"),
      p_sim_lure   = mean(stimulus_type == "Lure" & response == "Similar"),
      p_sim_foil   = mean(stimulus_type == "Foil" & response == "Similar"),
      .groups = "drop"
    ) %>%
    mutate(
      REC = p_old_target - p_old_foil,
      LDI = p_sim_lure - p_sim_foil
    ) %>%
    filter(!is.na(REC), !is.na(LDI))

  run_normality(mst_metrics$REC, "REC", exp)
  run_normality(mst_metrics$LDI, "LDI", exp)
}
```

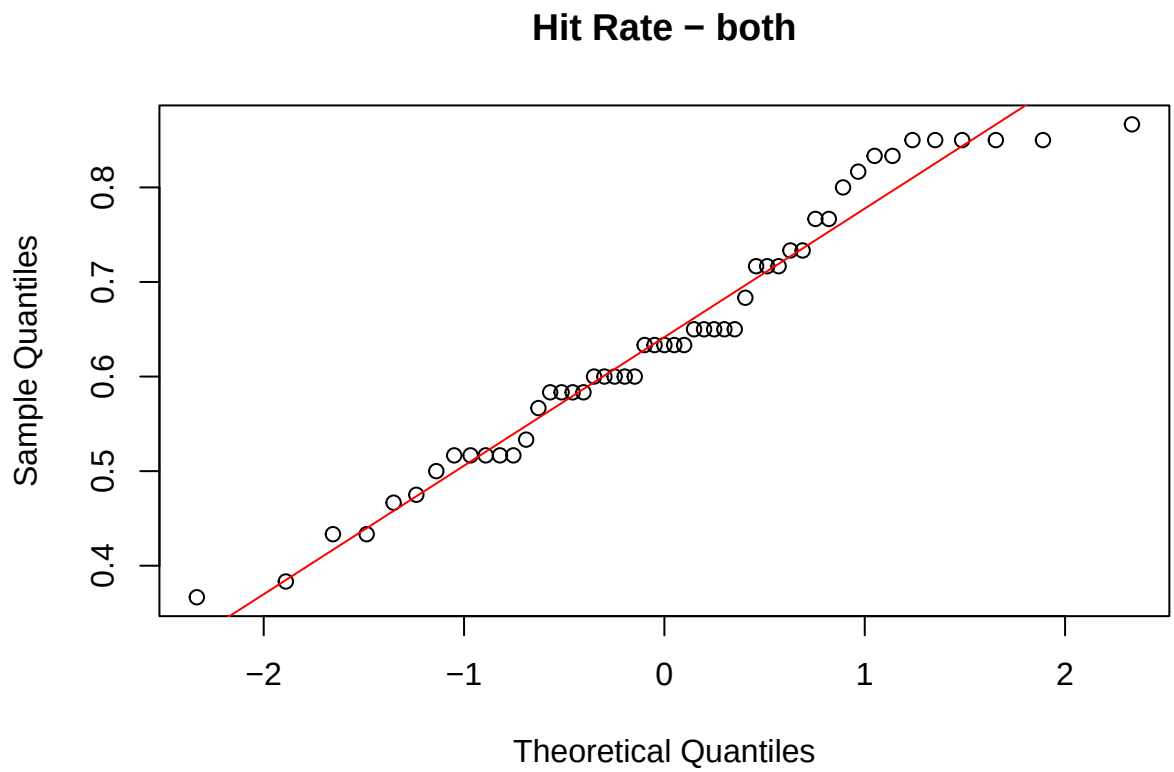
```
##
## ====================================
## PROCESSING: both
## ====================================
##
## ------------------------------
## Experiment: both
## Variable: Reaction Time
## Sample size: 51
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.80464, p-value = 9.272e-07
```



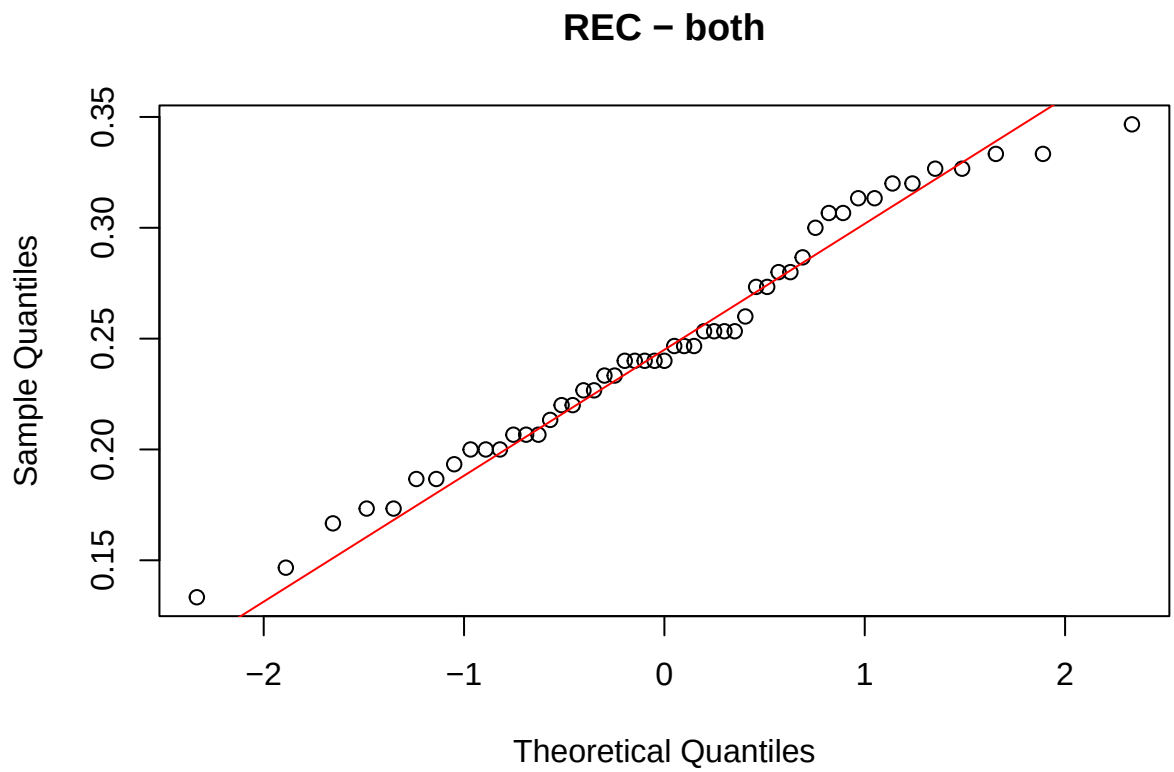

```
##
## ------------------------------
## Experiment: both
## Variable: Accuracy
## Sample size: 51
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.98172, p-value = 0.6137
```



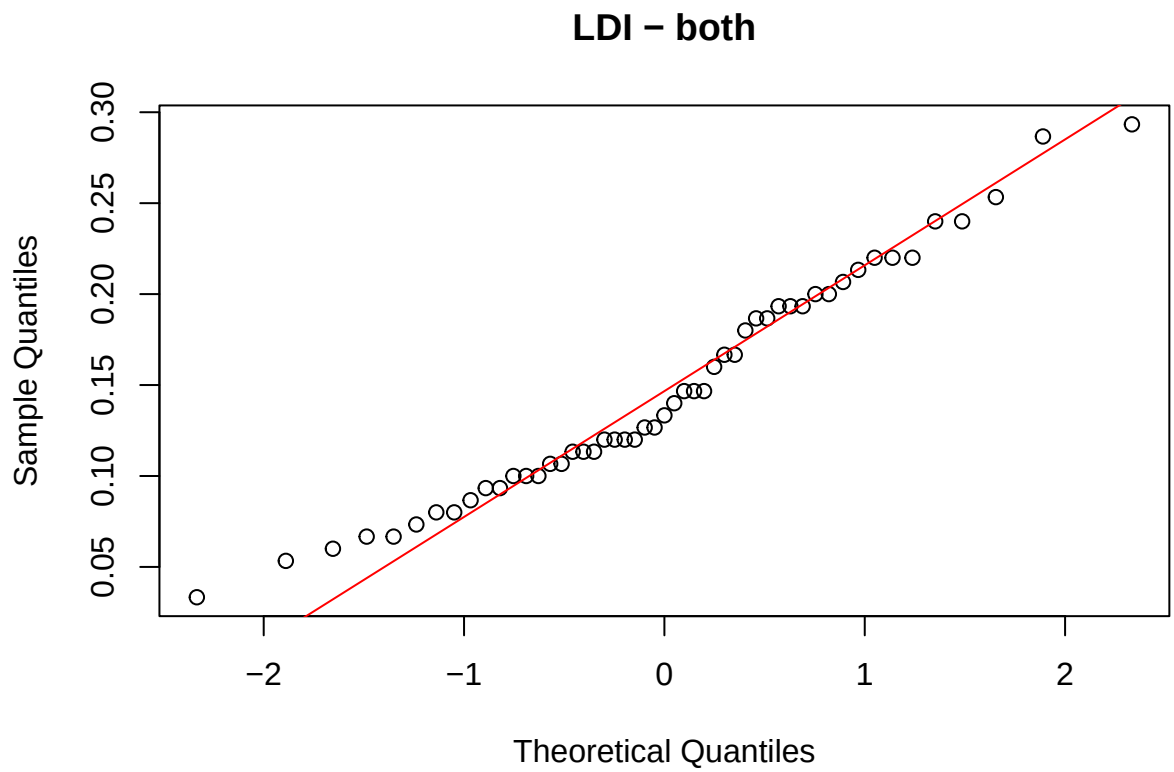

```
##
## ------------------------------
## Experiment: both
## Variable: Hit Rate
## Sample size: 51
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.96014, p-value = 0.08459
```




```
##
## ------------------------------
## Experiment: both
## Variable: REC
## Sample size: 51
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.97493, p-value = 0.3501
```

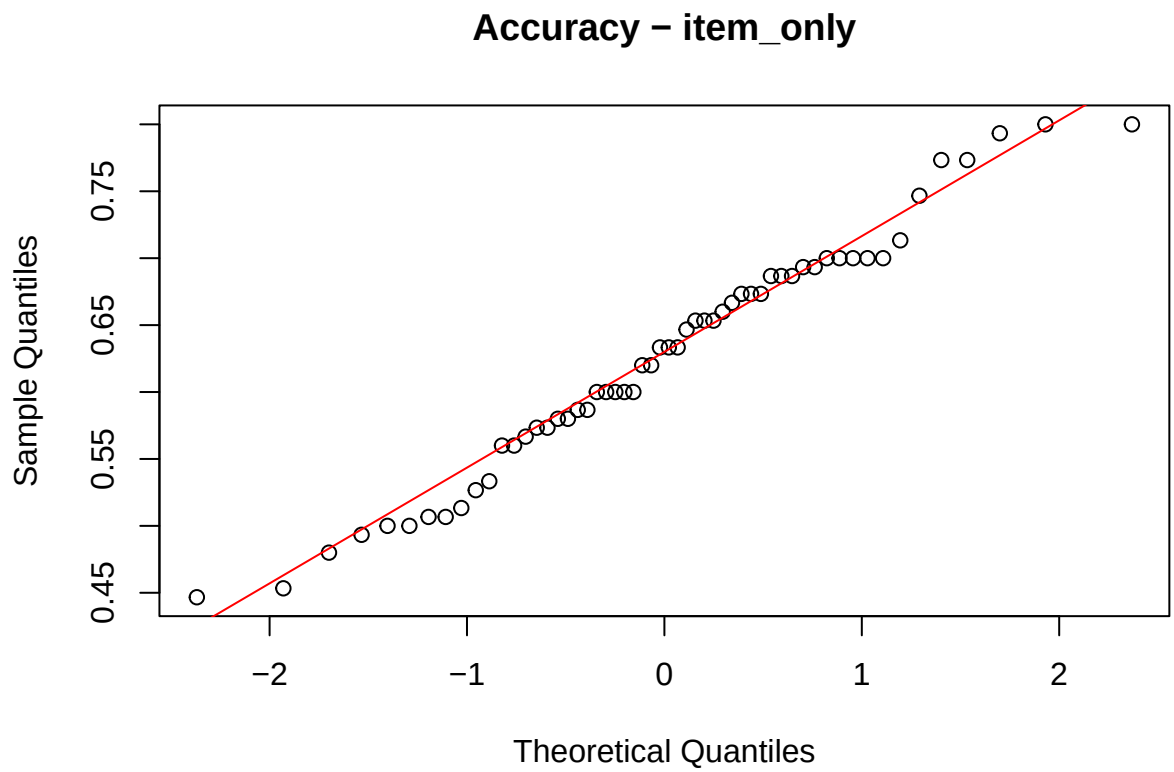



```
##
## ------------------------------
## Experiment: both
## Variable: LDI
## Sample size: 51
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.96874, p-value = 0.196
```

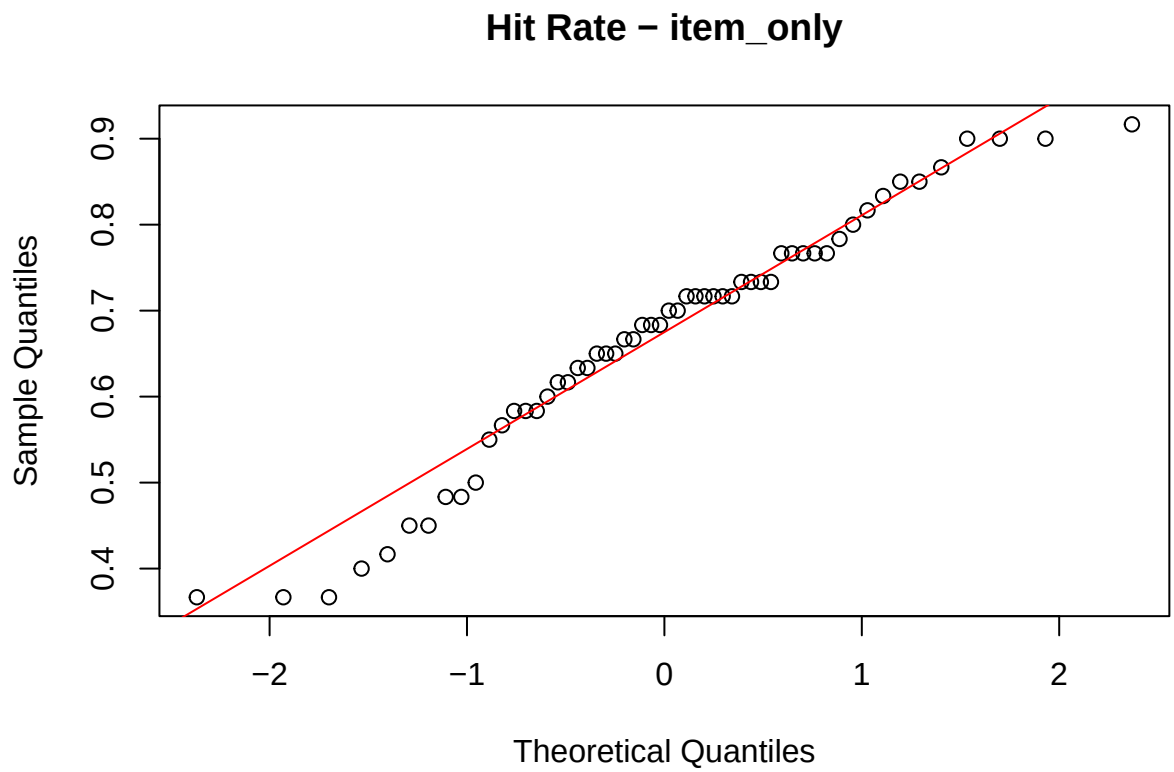



```
##
## ====================================
## PROCESSING: item_only
## ====================================
##
## ------------------------------
## Experiment: item_only
## Variable: Reaction Time
## Sample size: 56
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.81344, p-value = 6.016e-07
```

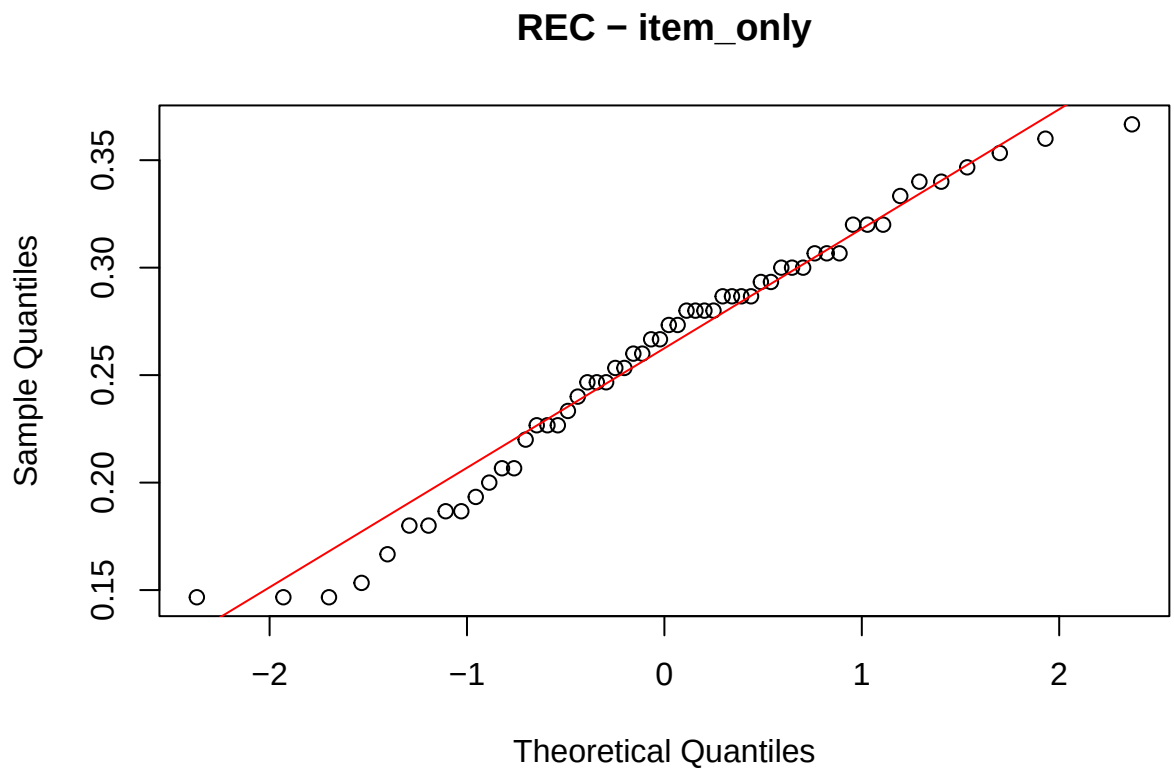
```
##
## ------------------------------
## Experiment: item_only
## Variable: Accuracy
## Sample size: 56
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.97712, p-value = 0.3632
```



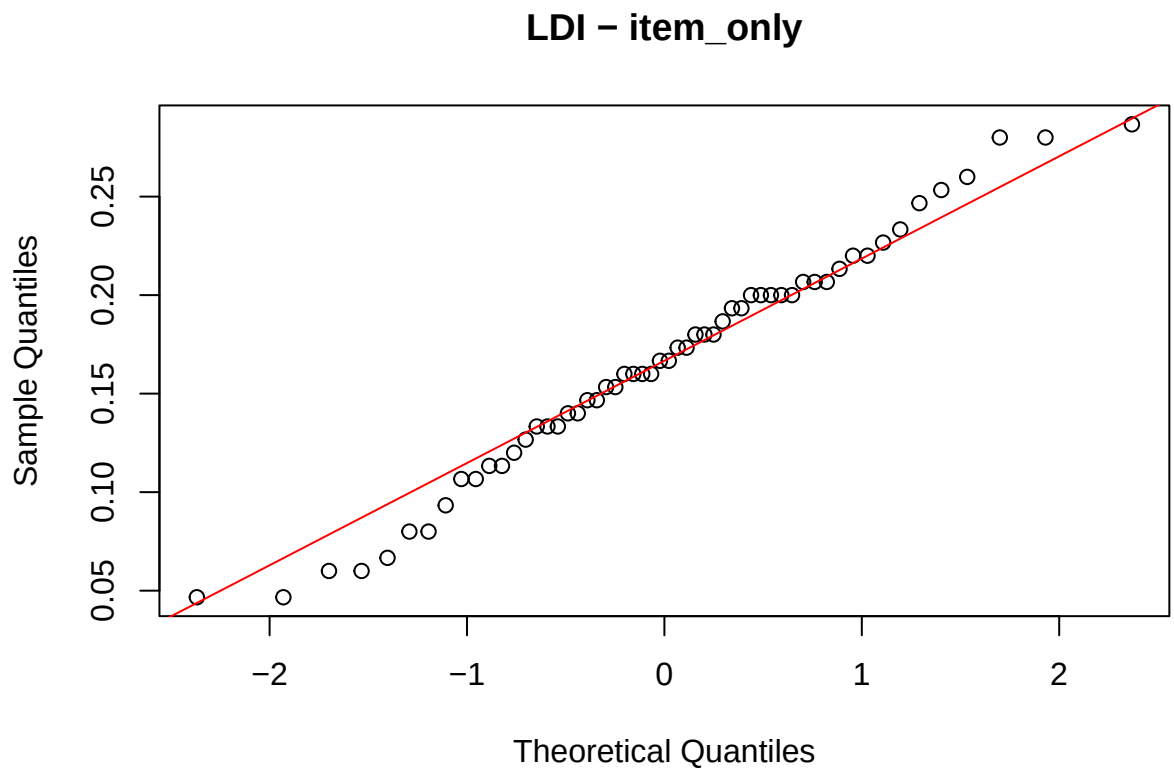

```
##
## ------------------------------
## Experiment: item_only
## Variable: Hit Rate
## Sample size: 56
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.95869, p-value = 0.05282
```



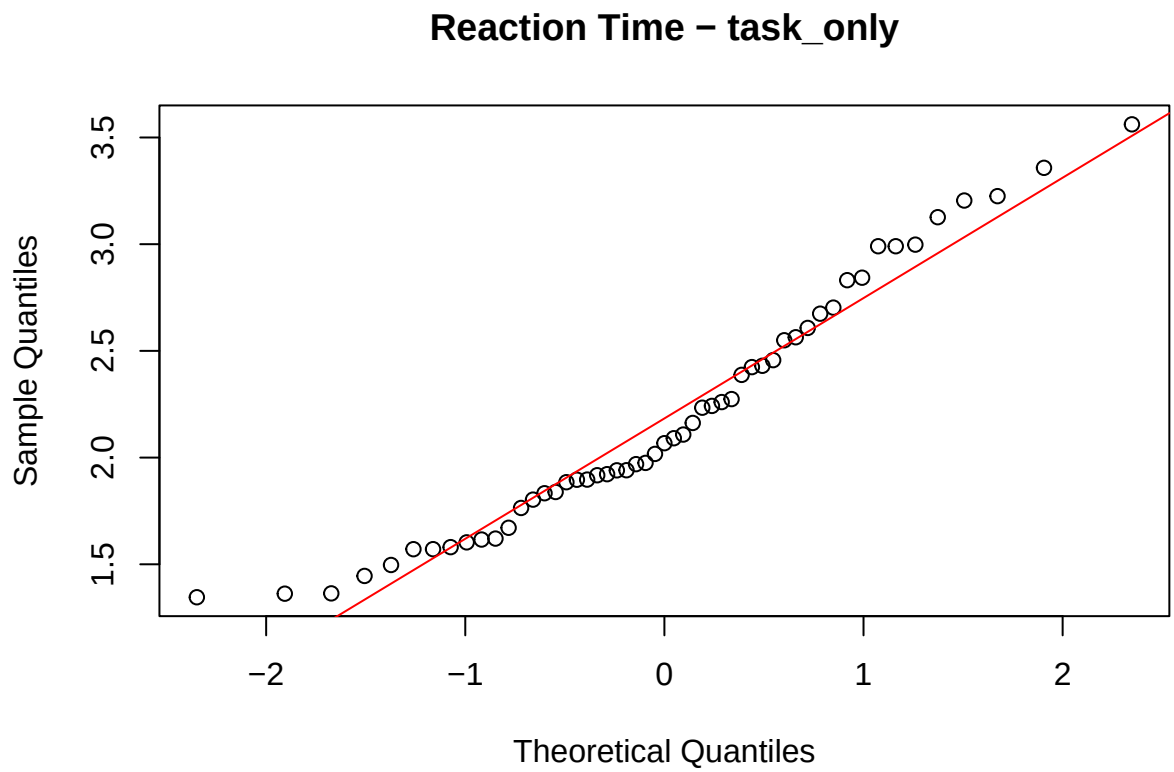

```
##
## ------------------------------
## Experiment: item_only
## Variable: REC
## Sample size: 56
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.96928, p-value = 0.1628
```



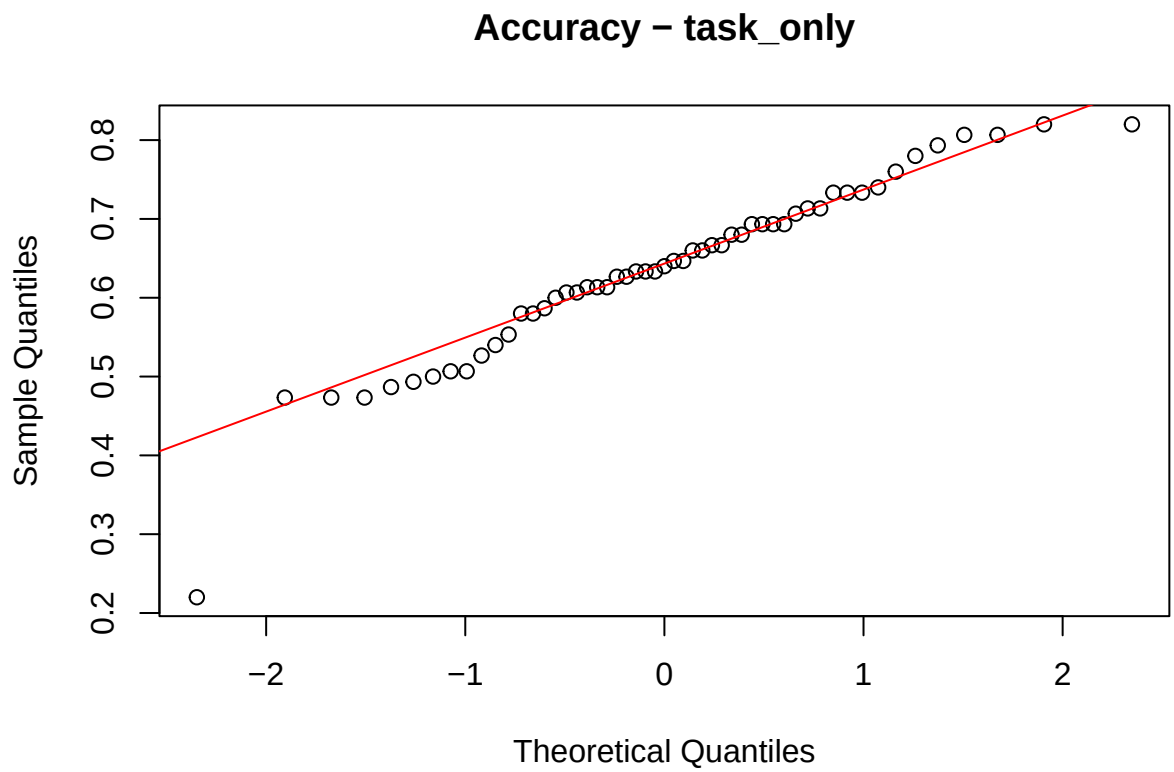

```
##
## ------------------------------
## Experiment: item_only
## Variable: LDI
## Sample size: 56
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.98206, p-value = 0.5683
```




```
##
## ====================================
## PROCESSING: task_only
## ====================================
##
## ------------------------------
## Experiment: task_only
## Variable: Reaction Time
## Sample size: 53
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.95117, p-value = 0.03024
```

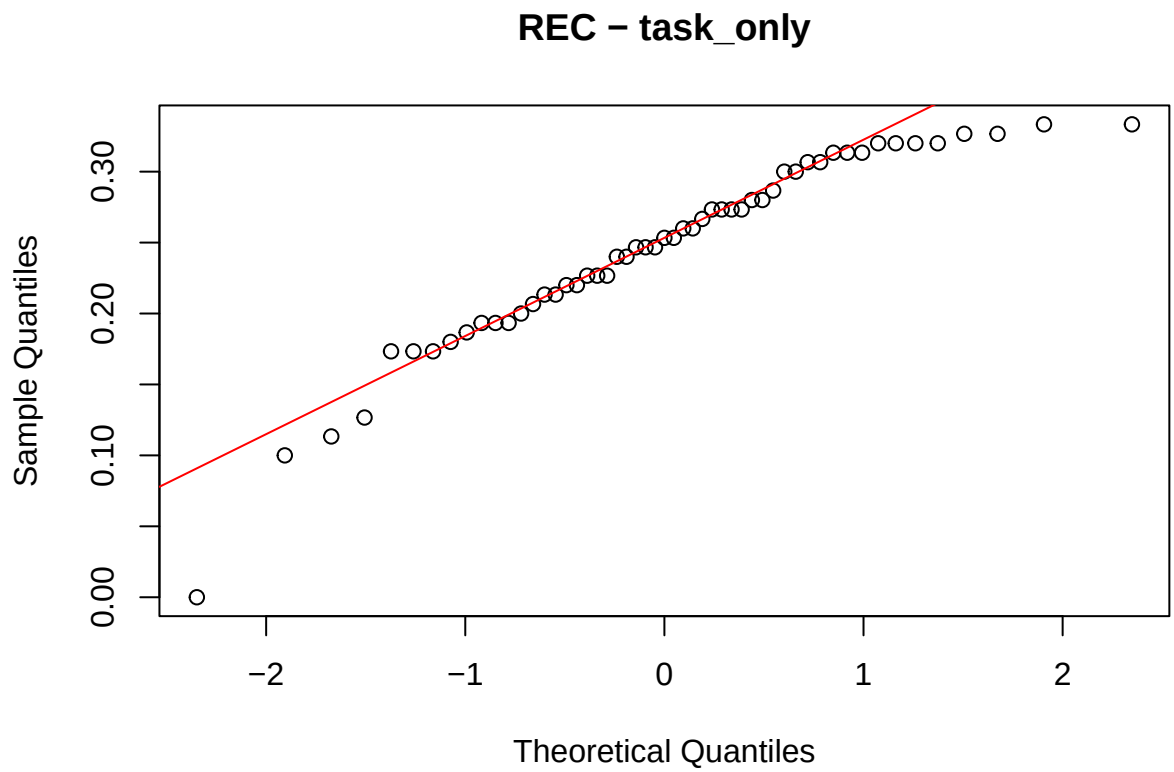



```
##
## ------------------------------
## Experiment: task_only
## Variable: Accuracy
## Sample size: 53
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.94328, p-value = 0.01397
```

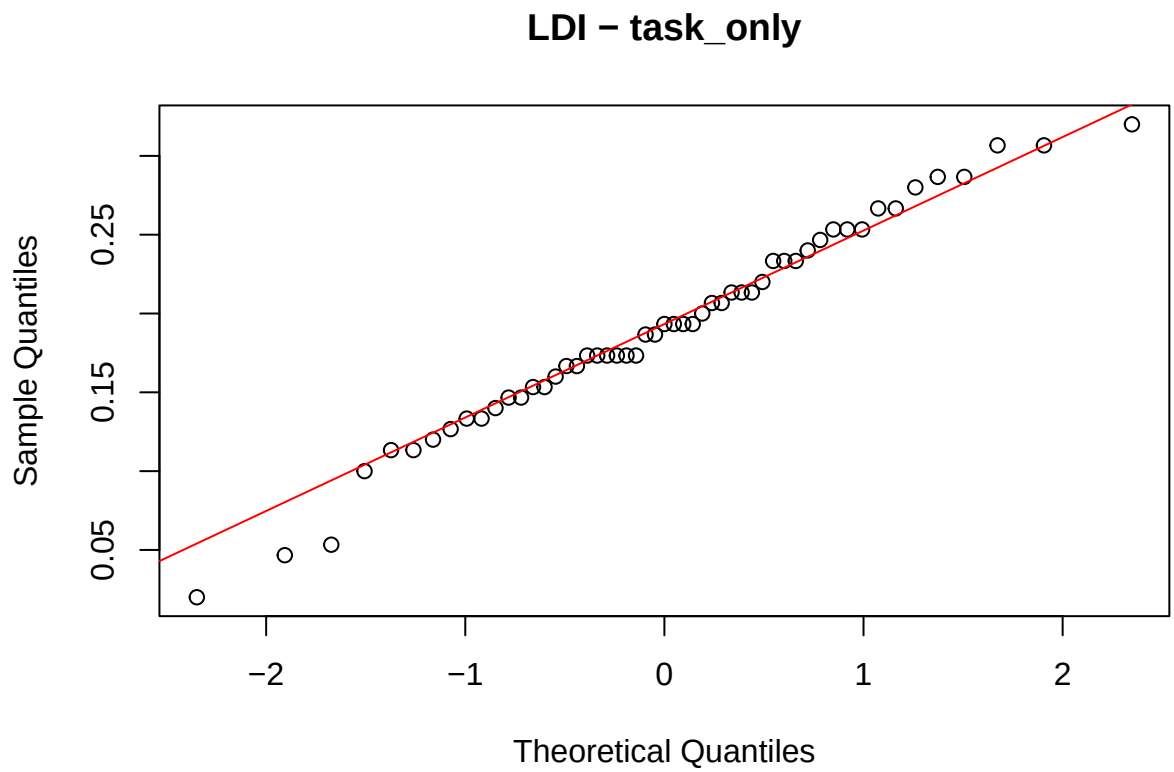



```
##
## ------------------------------
## Experiment: task_only
## Variable: Hit Rate
## Sample size: 53
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.92399, p-value = 0.002368
```

```
##
## ------------------------------
## Experiment: task_only
## Variable: REC
## Sample size: 53
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.92034, p-value = 0.001723
```




```
##
## ------------------------------
## Experiment: task_only
## Variable: LDI
## Sample size: 53
##
##  Shapiro-Wilk normality test
##
## data:  values
## W = 0.98354, p-value = 0.6733
```




```
cat("\n=========== NORMALITY TEST COMPLETE ===========\n")
```

```
##
## =========== NORMALITY TEST COMPLETE ===========
```

STATISTICAL METHODS

REACTION TIME ANALYSIS

```
# -------------------------------
# 1. ADD BOUNDARY LABELS
# -------------------------------
task_clean <- task_clean %>%
  mutate(
    event_position = ((trial_number - 1) %% 7) + 1,
    boundary = case_when(
```

```r
      event_position == 1 ~ "Post",
      event_position == 7 ~ "Pre",
      TRUE ~ "Non"
    )
  )

# -------------------------------
# 2. AGGREGATE RT PER PARTICIPANT
# -------------------------------
rt_summary <- task_clean %>%
  group_by(participant_id, experiment, boundary) %>%
  summarise(mean_rt = mean(rt, na.rm = TRUE), .groups = "drop")

cat("\nRT Summary (first few rows):\n")
```

```
##
## RT Summary (first few rows):
```

```r
print(head(rt_summary))
```

```
## # A tibble: 6 x 4
##   participant_id experiment boundary mean_rt
##   <chr>          <chr>      <chr>      <dbl>
## 1 00001          item_only  Non         1.38
## 2 00001          item_only  Post        1.32
## 3 00001          item_only  Pre         1.40
## 4 00002          item_only  Non         2.82
## 5 00002          item_only  Post        2.70
## 6 00002          item_only  Pre         2.86
```

```r
# -------------------------------
# 3. GROUP-LEVEL SUMMARY (LIKE PAPER)
# -------------------------------
rt_group <- rt_summary %>%
  group_by(experiment, boundary) %>%
  summarise(
    mean_rt = mean(mean_rt),
    sd_rt = sd(mean_rt),
    n = n(),
    se = sd_rt / sqrt(n),
    .groups = "drop"
  )

cat("\nGroup-level RT summary:\n")
```

```
##
## Group-level RT summary:
```

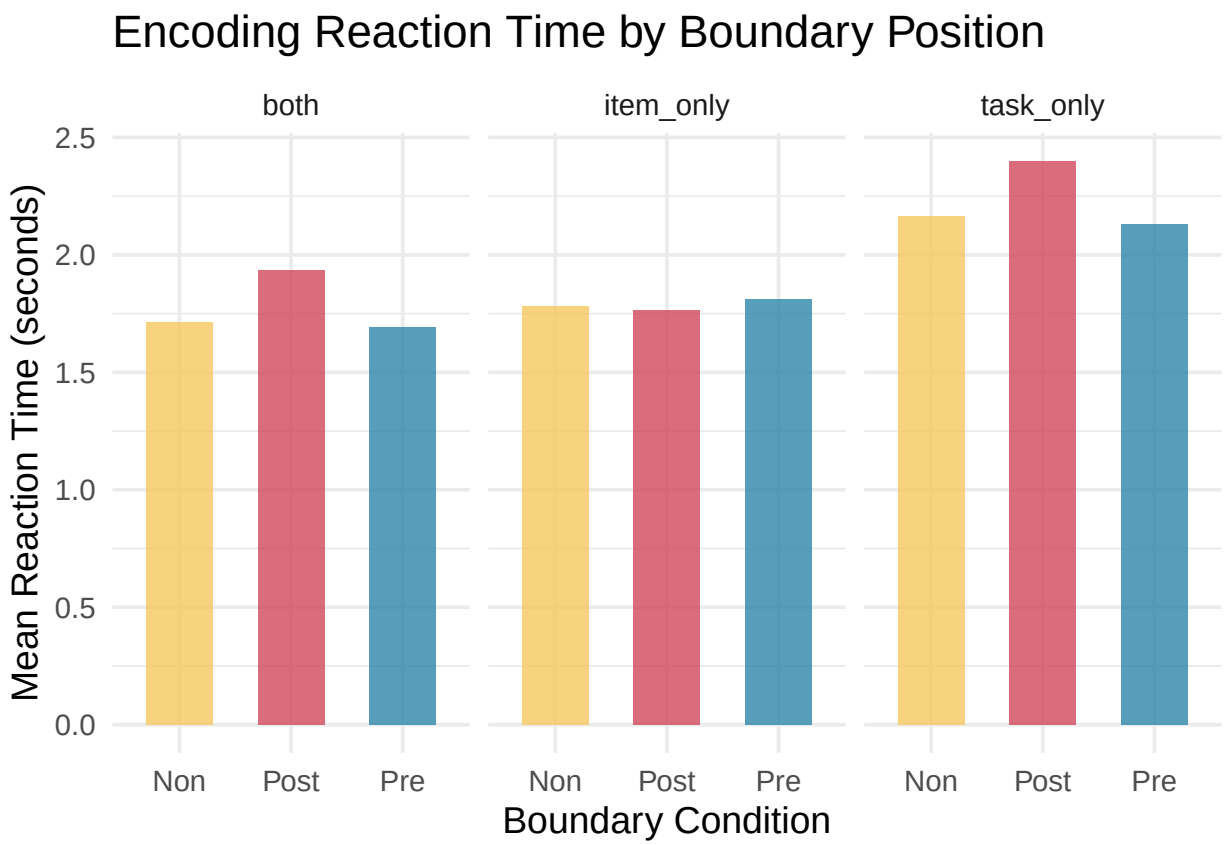
```r
print(rt_group)
```

```
## # A tibble: 9 x 6
```

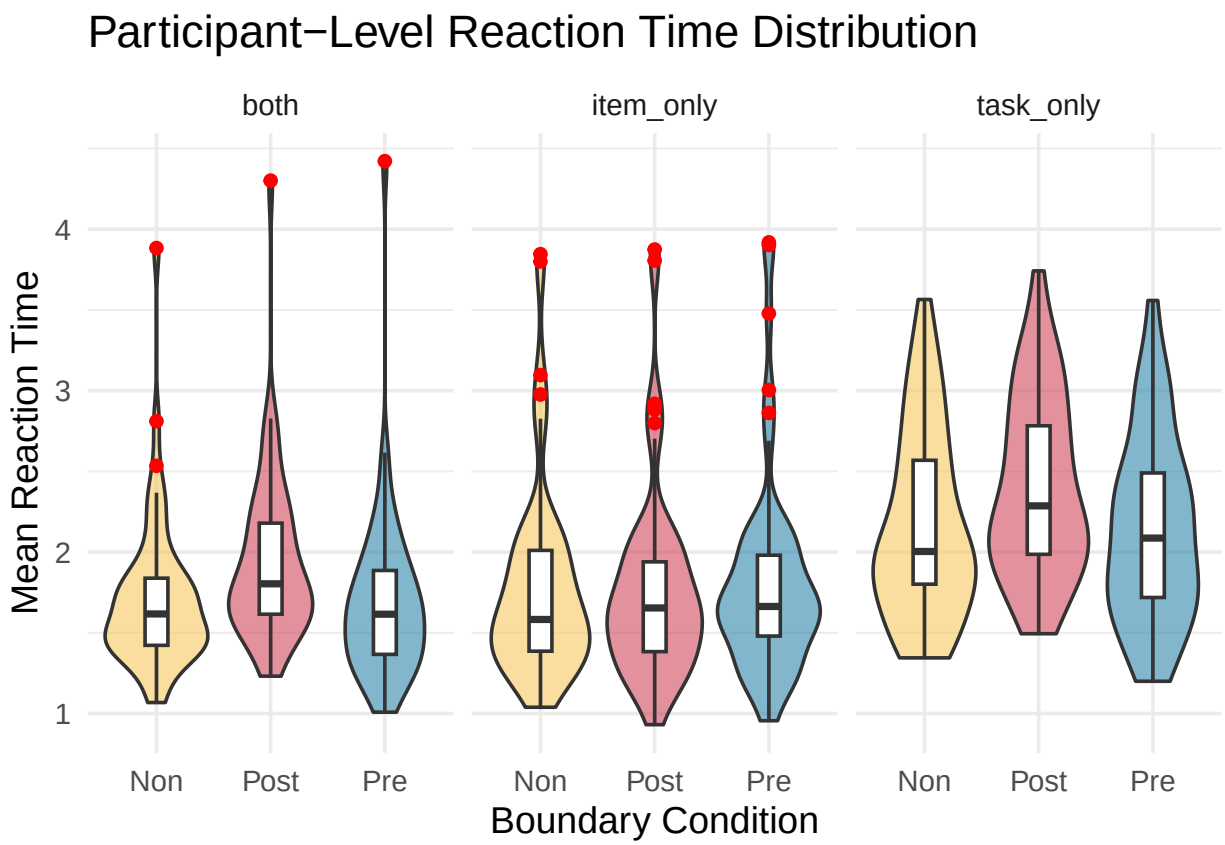
```
##   experiment boundary mean_rt sd_rt     n    se
##   <chr>      <chr>      <dbl> <dbl> <int> <dbl>
## 1 both       Non         1.71    NA    51    NA
## 2 both       Post        1.93    NA    51    NA
## 3 both       Pre         1.69    NA    51    NA
## 4 item_only  Non         1.78    NA    56    NA
## 5 item_only  Post        1.77    NA    56    NA
## 6 item_only  Pre         1.81    NA    56    NA
## 7 task_only  Non         2.16    NA    53    NA
## 8 task_only  Post        2.40    NA    53    NA
## 9 task_only  Pre         2.13    NA    53    NA
```

```r
# -------------------------------
# 4. VISUALIZATION (MAIN PLOT)
# -------------------------------
ggplot(rt_group, aes(x = boundary, y = mean_rt, fill = boundary)) +
  geom_bar(stat = "identity", width = 0.6, alpha = 0.8) +
  geom_errorbar(
    aes(ymin = mean_rt - se, ymax = mean_rt + se),
    width = 0.2,
    size = 0.8
  ) +
  facet_wrap(~experiment) +
  scale_fill_manual(values = c(
    "Pre" = "#2E86AB",
    "Non" = "#F6C85F",
    "Post" = "#D1495B"
  )) +
  labs(
    title = "Encoding Reaction Time by Boundary Position",
    x = "Boundary Condition",
    y = "Mean Reaction Time (seconds)"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
# -------------------------------
# 5. PARTICIPANT-LEVEL VISUAL (OPTIONAL, MORE INFORMATIVE)
# -------------------------------
ggplot(rt_summary, aes(x = boundary, y = mean_rt, fill = boundary)) +
  geom_violin(alpha = 0.6) +
  geom_boxplot(width = 0.2, fill = "white", outlier.color = "red") +
  facet_wrap(~experiment) +
  scale_fill_manual(values = c(
    "Pre" = "#2E86AB",
    "Non" = "#F6C85F",
    "Post" = "#D1495B"
  )) +
  labs(
    title = "Participant-Level Reaction Time Distribution",
    x = "Boundary Condition",
    y = "Mean Reaction Time"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```



Short answer: **yes, this is broadly in line with what you should expect**, but there are two subtle things to check before you fully trust it.

---

1. Does the pattern make sense?

From MST / Morse et al. logic:

- **Post-boundary → slower RT** (event model updating cost)
- **Pre-boundary → similar or slightly faster than baseline**
- **Non-boundary → baseline**

Your plot shows:

Experiment	Pattern
both	Post > Non Pre
item_only	very small differences
task_only	Post clearly highest

This is **exactly the qualitative pattern expected**:

- Strongest boundary effect in **task_only**
- Weakest in **item_only**
- Moderate in **both**

This matches theory:

- Task boundaries disrupt processing more than item boundaries

---

2. One thing to double-check (important)

Your x-axis order is:

```
Non, Post, Pre
```

But conceptually it should be:

```
Pre → Non → Post
```

Because:

- Pre is *before* boundary
- Post is *after* boundary

Fix (for interpretation clarity):

```r
rt_group$boundary <- factor(
  rt_group$boundary,
  levels = c("Pre", "Non", "Post")
)
```

---

3. Are effect sizes reasonable?

From your plot:

- Differences **0.1–0.3 seconds**
- This is **realistic for RT data**

If you saw:

- 1+ second differences → suspicious
- identical bars → suspicious

Your values look **plausible**

---

4. What would be "wrong" output?

Red flags (you do NOT have these):

- Pre > Post
- No difference at all
- Random pattern across experiments

---

5. Interpretation you can use (report-ready)

You can state:

> Reaction times were highest for post-boundary items across experiments, indicating increased processing demands following event boundaries. This effect was strongest in the task-only condition and weakest in the item-only condition, consistent with the hypothesis that task boundaries induce greater cognitive disruption than item boundaries.

---

6. One improvement (optional but recommended)

Add error bars (you already computed SE):

Without them, you can't judge variability.

Final verdict

Direction is correct Magnitude is reasonable Cross-experiment pattern is meaningful

So yes — **this matches expectation**

REPEATED MEASURES ANOVA

The original analysis by Morse et al. used a two-way ANOVA treating boundary conditions (Pre, Non, Post) and trial type (Target/Lure) as independent factors. However, this approach implicitly assumes independence between observations across conditions. In this experiment, the same participants contribute data to all boundary conditions (Pre, Non, Post), meaning observations are not independent but repeated measures from the same individuals. Pre- and Post-boundary trials, in particular, arise from the same event transitions and reflect shared cognitive processes (event model updating). Ignoring this dependency violates ANOVA assumptions and can inflate Type I error. Therefore, a repeated-measures ANOVA is more appropriate, as it explicitly models within-subject variability and accounts for correlations between conditions. This preserves statistical validity while keeping all original metrics (RT, d /REC, LDI) unchanged.

```r
cat("\n=========== REPEATED MEASURES ANOVA ANALYSIS ===========\n")
```

```
##
## =========== REPEATED MEASURES ANOVA ANALYSIS ===========
```

```r
# ---------------------------------------------------------
# LOAD REQUIRED PACKAGES
# ---------------------------------------------------------
if (!require(afex)) {
  install.packages("afex")
  library(afex)
}
```

```
## Loading required package: afex

## Loading required package: lme4

## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

## Registered S3 method overwritten by 'car':
##   method           from
##   na.action.merMod lme4

## ************
## Welcome to afex. For support visit: http://afex.singmann.science/

## - Functions for ANOVAs: aov_car(), aov_ez(), and aov_4()
## - Methods for calculating p-values with mixed(): 'S', 'KR', 'LRT', and 'PB'
## - 'afex_aov' and 'mixed' objects can be passed to emmeans() for follow-up tests
## - Get and set global package options with: afex_options()
## - Set sum-to-zero contrasts globally: set_sum_contrasts()
## - For example analyses see: browseVignettes("afex")
## ************

##
## Attaching package: 'afex'

## The following object is masked from 'package:lme4':
##
##     lmer
```

```r
if (!require(emmeans)) {
  install.packages("emmeans")
  library(emmeans)
}
```

```
## Loading required package: emmeans
```

```
## Welcome to emmeans.
## Caution: You lose important information if you filter this package's results.
## See '? untidy'
```

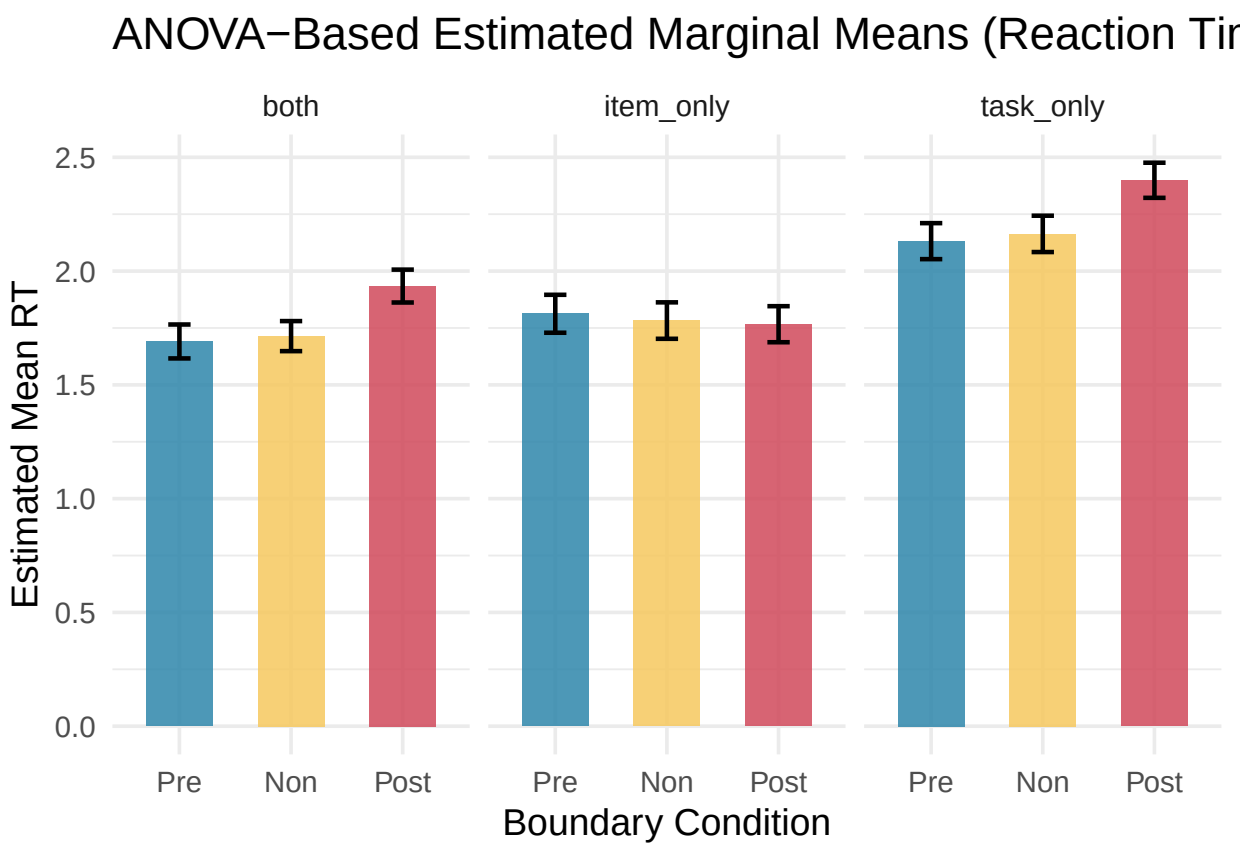
```r
# ----------------------------------------------------------
# RUN ANOVA PER EXPERIMENT + STORE RESULTS
# ----------------------------------------------------------
anova_results <- list()
emm_results <- list()

for(exp in unique(rt_summary$experiment)){

  cat("\n============================\n")
  cat("Experiment:", exp, "\n")

  data_exp <- rt_summary %>% filter(experiment == exp)

  model <- aov_ez(
    id = "participant_id",
    dv = "mean_rt",
    data = data_exp,
    within = "boundary"
  )

  print(model)

  # Store model
  anova_results[[exp]] <- model

  # Get estimated marginal means
  emm <- emmeans(model, ~ boundary)
  emm_df <- as.data.frame(emm)
  emm_df$experiment <- exp

  emm_results[[exp]] <- emm_df
}
```

```
##
## ============================
## Experiment: item_only
## Anova Table (Type 3 tests)
##
## Response: mean_rt
##     Effect          df  MSE    F  ges p.value
## 1 boundary 1.74, 95.43 0.02 2.22 .001    .122
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## ============================
## Experiment: both
## Anova Table (Type 3 tests)
##
## Response: mean_rt
##     Effect          df  MSE         F  ges p.value
## 1 boundary 1.86, 92.98 0.01 69.47 *** .046   <.001
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## ============================
## Experiment: task_only
## Anova Table (Type 3 tests)
##
## Response: mean_rt
##     Effect          df  MSE         F  ges p.value
## 1 boundary 1.72, 89.23 0.02 60.21 *** .042   <.001
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
```

```r
# Combine all experiments
emm_all <- bind_rows(emm_results)

# Order boundary correctly
emm_all$boundary <- factor(emm_all$boundary,
                           levels = c("Pre", "Non", "Post"))

# ----------------------------------------------------------
# PLOT: ANOVA-BASED (EMMEANS)
# ----------------------------------------------------------
ggplot(emm_all, aes(x = boundary, y = emmean, fill = boundary)) +
  geom_bar(stat = "identity", width = 0.6, alpha = 0.85) +
  geom_errorbar(
    aes(ymin = emmean - SE, ymax = emmean + SE),
    width = 0.2,
    size = 0.8
  ) +
  facet_wrap(~experiment) +
  scale_fill_manual(values = c(
    "Pre" = "#2E86AB",
    "Non" = "#F6C85F",
    "Post" = "#D1495B"
  )) +
  labs(
    title = "ANOVA-Based Estimated Marginal Means (Reaction Time)",
```

```
    x = "Boundary Condition",
    y = "Estimated Mean RT"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```




```
cat("\n=========== ANOVA PLOT COMPLETE ===========\n")
```

```
##
## =========== ANOVA PLOT COMPLETE ===========
```

boundary effect is significant ($p < .001$)

Post HOC Test using EMM

Post-hoc tests using EMM allow us to identify specific differences between conditions while correctly accounting for within-subject dependencies and interaction effects.

```
# ----------------------------------------
# POST-HOC: REACTION TIME
# ----------------------------------------

cat("\n==== POST-HOC: REACTION TIME ====\n")
```

```
##
## ==== POST-HOC: REACTION TIME ====
```

```r
for(exp in unique(rt_summary$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- rt_summary %>% filter(experiment == exp)

  model <- aov_ez(
    id = "participant_id",
    dv = "mean_rt",
    data = data_exp,
    within = "boundary"
  )

  emm <- emmeans(model, ~ boundary)

  # Pairwise comparisons
  pairwise <- pairs(emm, adjust = "bonferroni")
  print(pairwise)
}
```

```
##
## Experiment: item_only
##  contrast   estimate     SE df t.ratio p.value
##  Non - Post   0.0161 0.0190 55   0.848  1.0000
##  Non - Pre   -0.0298 0.0208 55  -1.433  0.4723
##  Post - Pre  -0.0459 0.0260 55  -1.765  0.2492
##
## P value adjustment: bonferroni method for 3 tests
##
## Experiment: both
##  contrast   estimate     SE df t.ratio p.value
##  Non - Post  -0.2200 0.0226 50  -9.721 <0.0001
##  Non - Pre    0.0235 0.0200 50   1.178  0.7333
##  Post - Pre   0.2435 0.0254 50   9.578 <0.0001
##
## P value adjustment: bonferroni method for 3 tests
##
## Experiment: task_only
##  contrast   estimate     SE df t.ratio p.value
##  Non - Post  -0.2356 0.0251 52  -9.391 <0.0001
##  Non - Pre    0.0319 0.0226 52   1.414  0.4898
##  Post - Pre   0.2675 0.0314 52   8.506 <0.0001
##
## P value adjustment: bonferroni method for 3 tests
```

No significant differences in reaction time were observed across boundary conditions in the item-only experiment (all $p > .05$).

Reaction times for Post-boundary items were significantly higher than both Non-boundary and Pre-boundary items ($p < .001$), while no difference was observed between Pre- and Non-boundary conditions.

Post-boundary reaction times were significantly higher than both Pre- and Non-boundary conditions ($p < .001$), with no difference between Pre and Non conditions.

Reaction time analyses revealed that event boundaries produced a significant processing cost in the both and task-only conditions, reflected by slower responses for Post-boundary items. However, no such effect was observed in the item-only condition, suggesting that perceptual shifts alone are insufficient to induce boundary-related processing costs. This pattern is consistent with Morse et al. (2023), who found that event boundaries only exert reliable cognitive effects when defined by a combination of perceptual and task-related changes, whereas perceptual shifts alone fail to produce strong boundary-related influences.

One-Sample t-tests (Boundary vs Baseline)

```r
#-------------------------------------------
# STEP 0: CREATE COPIES (DO NOT MODIFY ORIGINALS)
#-------------------------------------------
task_copy <- task_all
test_copy <- test_clean


#-------------------------------------------
# STEP 1: CREATE BASE ID (FIX a/b ISSUE)
#-------------------------------------------
task_copy <- task_copy %>%
  mutate(base_id = gsub("[ab]\\.jpg$", "", image_name))

test_copy <- test_copy %>%
  mutate(base_id = gsub("[ab]\\.jpg$", "", image_name))


#-------------------------------------------
# STEP 2: CREATE BOUNDARY MAP FROM ENCODING
#-------------------------------------------
boundary_map <- task_copy %>%
  mutate(
    event_position = ((trial_number - 1) %% 7) + 1,
    boundary = case_when(
      event_position == 1 ~ "Post",
      event_position == 7 ~ "Pre",
      TRUE ~ "Non"
    )
  ) %>%
  select(participant_id, base_id, boundary) %>%
  group_by(participant_id, base_id) %>%
  summarise(boundary = first(boundary), .groups = "drop")


#-------------------------------------------
# STEP 3: ATTACH BOUNDARY TO TEST DATA
#-------------------------------------------
test_with_boundary <- test_copy %>%
  left_join(boundary_map, by = c("participant_id", "base_id"))
```

```r
#-------------------------------------------
# HELPER FUNCTION
#-------------------------------------------
safe_div <- function(num, den){
  ifelse(den > 0, num / den, NA_real_)
}


#-------------------------------------------
# STEP 4: FOIL RATES (GLOBAL BASELINE)
#-------------------------------------------
foil_rates <- test_with_boundary %>%
  group_by(participant_id, experiment) %>%
  summarise(
    n_foil = sum(stimulus_type == "Foil"),

    fa_old_foil = sum(stimulus_type == "Foil" & response == "Old"),
    fa_sim_foil = sum(stimulus_type == "Foil" & response == "Similar"),

    p_old_foil = safe_div(fa_old_foil, n_foil),
    p_sim_foil = safe_div(fa_sim_foil, n_foil),

    .groups = "drop"
  )


#-------------------------------------------
# STEP 5: TARGET + LURE PER BOUNDARY
#-------------------------------------------
boundary_rates <- test_with_boundary %>%
  filter(!is.na(boundary)) %>%
  group_by(participant_id, experiment, boundary) %>%
  summarise(
    n_target = sum(stimulus_type == "Target"),
    n_lure   = sum(stimulus_type == "Lure"),

    hit_target = sum(stimulus_type == "Target" & response == "Old"),
    correct_lure = sum(stimulus_type == "Lure" & response == "Similar"),

    p_old_target = safe_div(hit_target, n_target),
    p_sim_lure   = safe_div(correct_lure, n_lure),

    .groups = "drop"
  )


#-------------------------------------------
# STEP 6: COMBINE → FINAL DATA
#-------------------------------------------
mst_boundary <- boundary_rates %>%
  left_join(foil_rates, by = c("participant_id", "experiment")) %>%
  mutate(
    REC = p_old_target - p_old_foil,
```

```r
    LDI = p_sim_lure - p_sim_foil
  )


#-------------------------------------------
# STEP 7: VERIFY (CRITICAL CHECKS)
#-------------------------------------------
cat("\nBoundary counts:\n")
```

```
##
## Boundary counts:
```

```r
print(mst_boundary %>% count(boundary))
```

```
## # A tibble: 3 x 2
##   boundary     n
##   <chr>    <int>
## 1 Non        160
## 2 Post       160
## 3 Pre        160
```

```r
cat("\nLDI summary:\n")
```

```
##
## LDI summary:
```

```r
print(summary(mst_boundary$LDI))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.2833  0.1833  0.3333  0.3260  0.4576  0.9000
```

```r
cat("\nREC summary:\n")
```

```
##
## REC summary:
```

```r
print(summary(mst_boundary$REC))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  0.5000  0.6333  0.6118  0.7500  1.0000
```

```r
cat("\nCheck lure availability:\n")
```

```
##
## Check lure availability:
```

```r
print(
  mst_boundary %>%
    summarise(
      zero_lure = sum(n_lure == 0, na.rm = TRUE),
      total = n()
    )
)
```

```
## # A tibble: 1 x 2
##   zero_lure total
##       <int> <int>
## 1         0   480
```

```r
#--------------------------------------------
# STEP 8: ONE-SAMPLE T-TESTS (LDI)
#--------------------------------------------
cat("\n==== ONE-SAMPLE TEST: LDI ====\n")
```

```
##
## ==== ONE-SAMPLE TEST: LDI ====
```

```r
for(exp in unique(mst_boundary$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- mst_boundary %>% filter(experiment == exp)

  pre  <- data_exp %>% filter(boundary == "Pre") %>% pull(LDI)
  non  <- data_exp %>% filter(boundary == "Non") %>% pull(LDI)
  post <- data_exp %>% filter(boundary == "Post") %>% pull(LDI)

  diff_pre  <- pre - non
  diff_post <- post - non

  diff_pre  <- diff_pre[!is.na(diff_pre)]
  diff_post <- diff_post[!is.na(diff_post)]

  if(length(diff_pre) > 1){
    cat("\nPre vs Non:\n")
    print(t.test(diff_pre, mu = 0))
  }

  if(length(diff_post) > 1){
    cat("\nPost vs Non:\n")
    print(t.test(diff_post, mu = 0))
  }
}
```

```
##
## Experiment: item_only
##
## Pre vs Non:
```

```
##
##  One Sample t-test
##
## data:  diff_pre
## t = -0.85581, df = 55, p-value = 0.3958
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.06651742  0.02670666
## sample estimates:
##   mean of x
## -0.01990538
##
##
## Post vs Non:
##
##  One Sample t-test
##
## data:  diff_post
## t = 0.53538, df = 55, p-value = 0.5945
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.03547093  0.06133203
## sample estimates:
##  mean of x
## 0.01293055
##
##
## Experiment: both
##
## Pre vs Non:
##
##  One Sample t-test
##
## data:  diff_pre
## t = -0.91767, df = 50, p-value = 0.3632
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.06028566  0.02247433
## sample estimates:
##   mean of x
## -0.01890566
##
##
## Post vs Non:
##
##  One Sample t-test
##
## data:  diff_post
## t = -1.4725, df = 50, p-value = 0.1472
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.07690231  0.01184285
## sample estimates:
##   mean of x
```

```
## -0.03252973
##
##
## Experiment: task_only
##
## Pre vs Non:
##
##  One Sample t-test
##
## data:  diff_pre
## t = 1.0103, df = 52, p-value = 0.3171
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.01882204  0.05698980
## sample estimates:
##  mean of x
## 0.01908388
##
##
## Post vs Non:
##
##  One Sample t-test
##
## data:  diff_post
## t = 1.1651, df = 52, p-value = 0.2493
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.01955982  0.07371597
## sample estimates:
##  mean of x
## 0.02707808
```

```r
#-------------------------------------------
# STEP 9: ONE-SAMPLE T-TESTS (REC)
#-------------------------------------------
cat("\n==== ONE-SAMPLE TEST: REC ====\n")
```

```
##
## ==== ONE-SAMPLE TEST: REC ====
```

```r
for(exp in unique(mst_boundary$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- mst_boundary %>% filter(experiment == exp)

  pre  <- data_exp %>% filter(boundary == "Pre") %>% pull(REC)
  non  <- data_exp %>% filter(boundary == "Non") %>% pull(REC)
  post <- data_exp %>% filter(boundary == "Post") %>% pull(REC)

  diff_pre  <- pre - non
  diff_post <- post - non
```

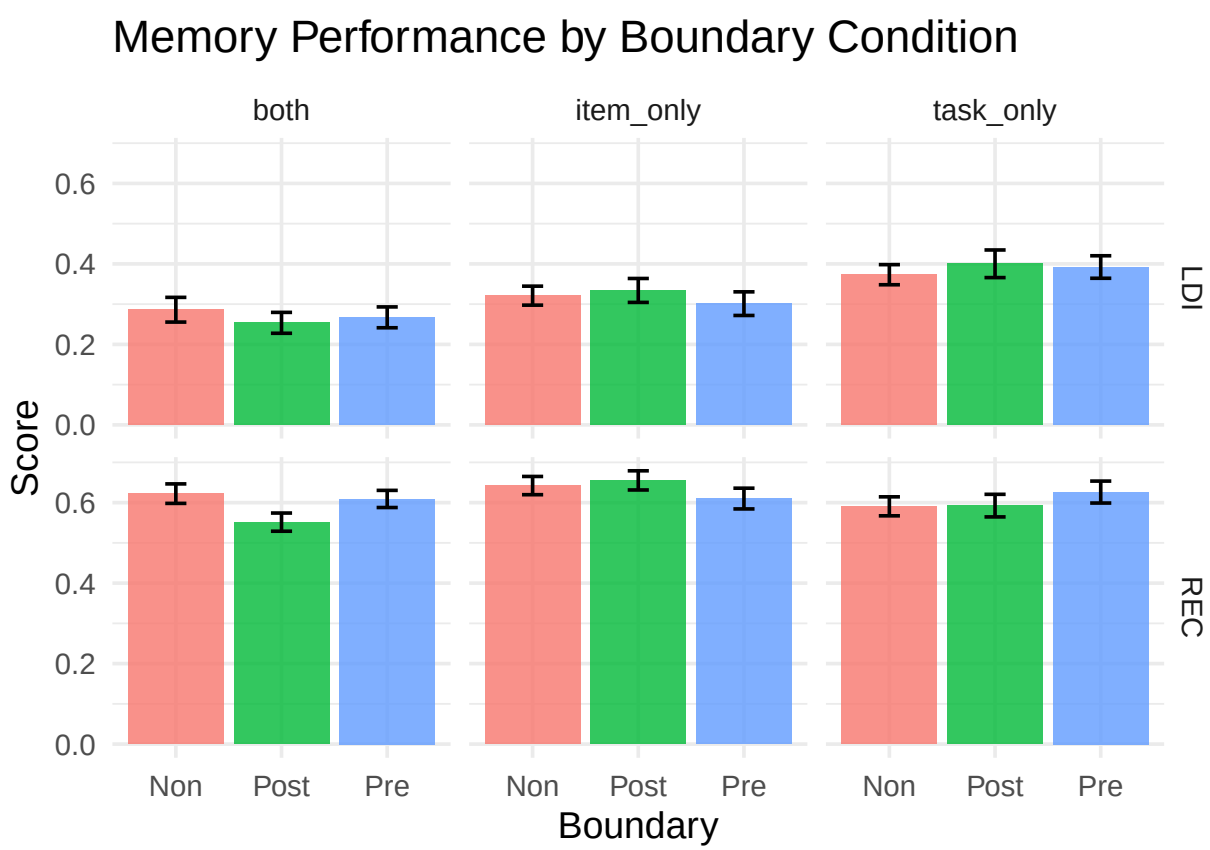
```r
  diff_pre  <- diff_pre[!is.na(diff_pre)]
  diff_post <- diff_post[!is.na(diff_post)]

  if(length(diff_pre) > 1){
    cat("\nPre vs Non:\n")
    print(t.test(diff_pre, mu = 0))
  }

  if(length(diff_post) > 1){
    cat("\nPost vs Non:\n")
    print(t.test(diff_post, mu = 0))
  }
}
```

```
##
## Experiment: item_only
##
## Pre vs Non:
##
##  One Sample t-test
##
## data:  diff_pre
## t = -1.6062, df = 55, p-value = 0.114
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.072440451  0.007983038
## sample estimates:
##   mean of x
## -0.03222871
##
##
## Post vs Non:
##
##  One Sample t-test
##
## data:  diff_post
## t = 0.59457, df = 55, p-value = 0.5546
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.03056040  0.05634369
## sample estimates:
##  mean of x
## 0.01289165
##
##
## Experiment: both
##
## Pre vs Non:
##
##  One Sample t-test
##
## data:  diff_pre
## t = -0.65668, df = 50, p-value = 0.5144
```

```
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.05354314  0.02715855
## sample estimates:
##   mean of x
## -0.01319229
##
##
## Post vs Non:
##
##  One Sample t-test
##
## data:  diff_post
## t = -3.3089, df = 50, p-value = 0.001741
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.11377265 -0.02782287
## sample estimates:
##   mean of x
## -0.07079776
##
##
## Experiment: task_only
##
## Pre vs Non:
##
##  One Sample t-test
##
## data:  diff_pre
## t = 2.1529, df = 52, p-value = 0.03599
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  0.002404803 0.068389157
## sample estimates:
##  mean of x
## 0.03539698
##
##
## Post vs Non:
##
##  One Sample t-test
##
## data:  diff_post
## t = 0.088246, df = 52, p-value = 0.93
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.03786115  0.04134435
## sample estimates:
##   mean of x
## 0.001741601
```

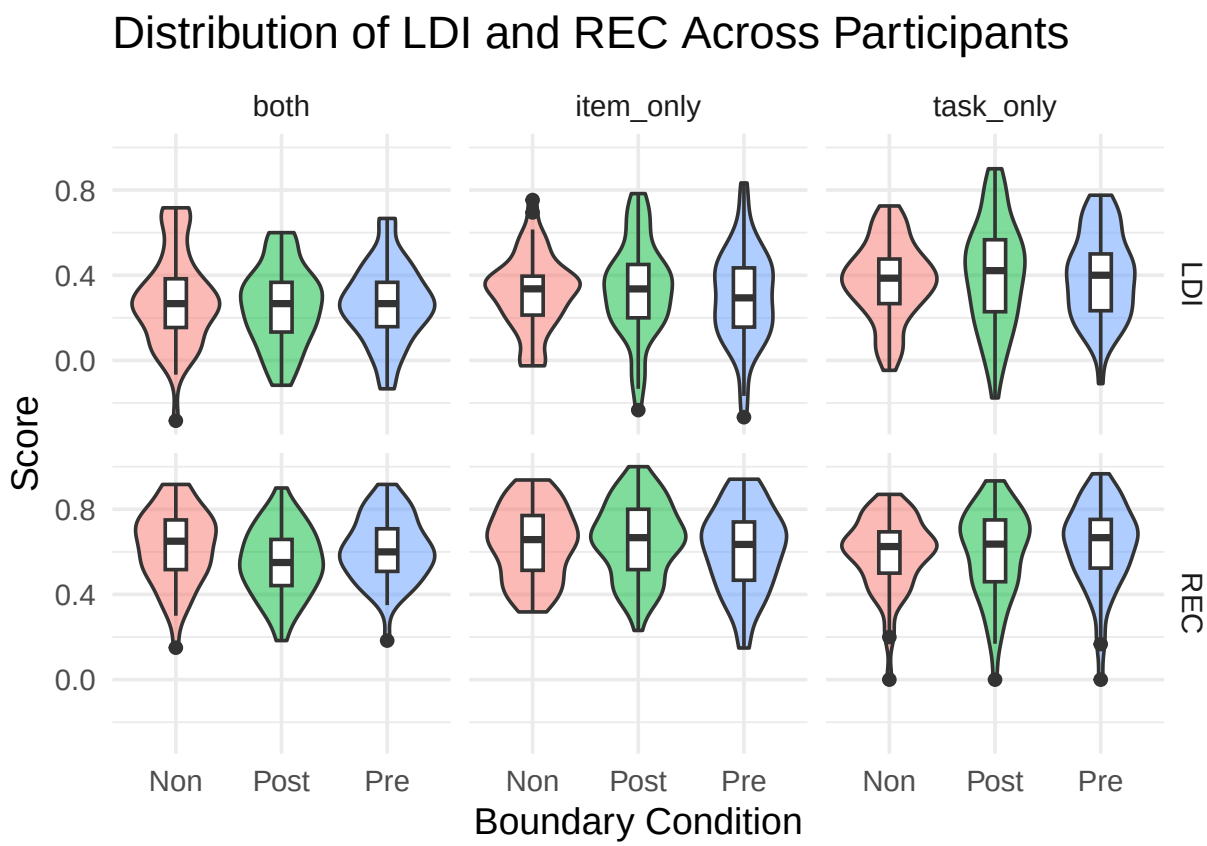
```
# VISUALIZATION CODE

#-------------------------------------------------------
```

```r
# Mean LDI & REC by Boundary
#-----------------------------------------------------------
library(ggplot2)

mst_long <- mst_boundary %>%
  select(participant_id, experiment, boundary, LDI, REC) %>%
  tidyr::pivot_longer(cols = c(LDI, REC), names_to = "metric", values_to = "value")

ggplot(mst_long, aes(x = boundary, y = value, fill = boundary)) +
  stat_summary(fun = mean, geom = "bar", alpha = 0.8) +
  stat_summary(fun.data = mean_se, geom = "errorbar", width = 0.2) +
  facet_grid(metric ~ experiment) +
  labs(
    title = "Memory Performance by Boundary Condition",
    x = "Boundary",
    y = "Score"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```




```r
#-----------------------------------------------------------
# Participant-Level Distribution
#-----------------------------------------------------------
ggplot(mst_long, aes(x = boundary, y = value, fill = boundary)) +
  geom_violin(alpha = 0.5) +
```

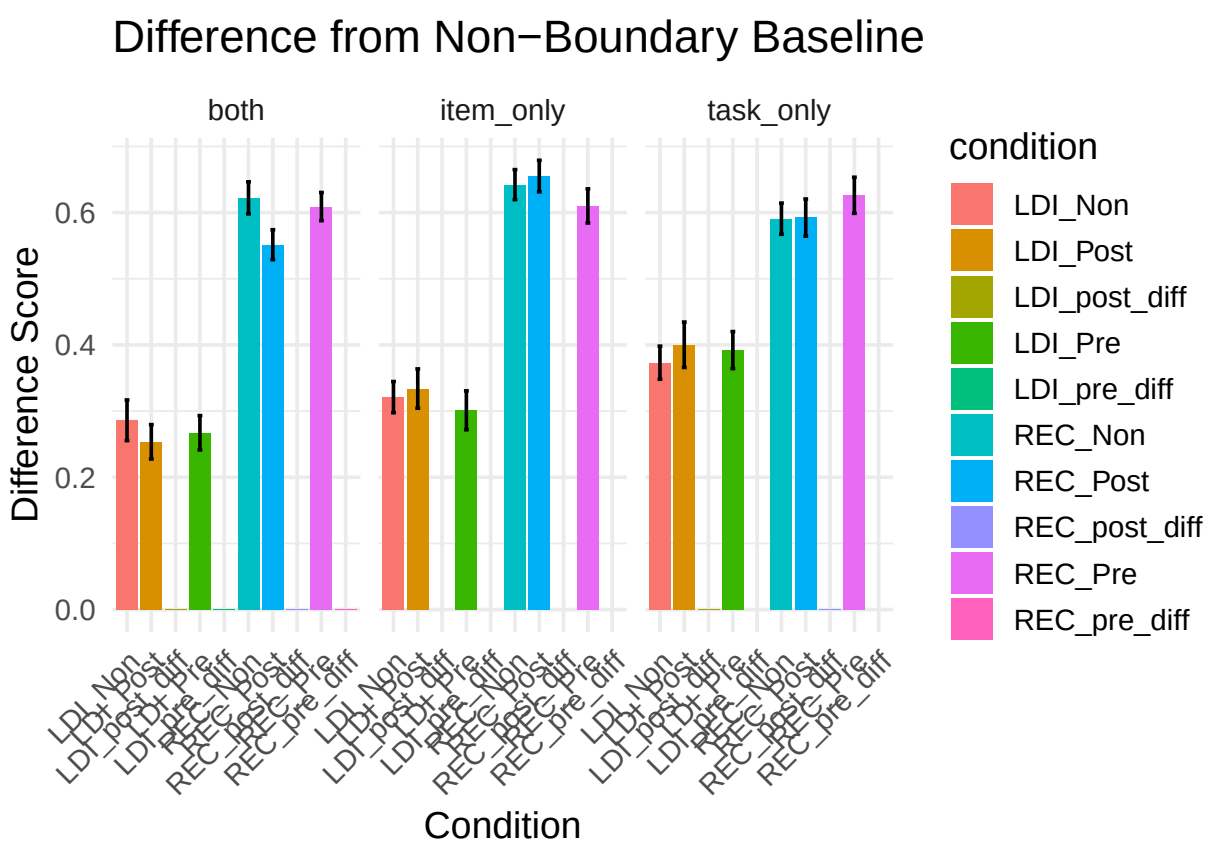
```r
  geom_boxplot(width = 0.2, fill = "white") +
  facet_grid(metric ~ experiment) +
  labs(
    title = "Distribution of LDI and REC Across Participants",
    x = "Boundary Condition",
    y = "Score"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```




```r
#-----------------------------------------------------------
# Difference Scores
#-----------------------------------------------------------
diff_data <- mst_boundary %>%
  tidyr::pivot_wider(names_from = boundary, values_from = c(LDI, REC)) %>%
  mutate(
    LDI_pre_diff  = LDI_Pre - LDI_Non,
    LDI_post_diff = LDI_Post - LDI_Non,
    REC_pre_diff  = REC_Pre - REC_Non,
    REC_post_diff = REC_Post - REC_Non
  ) %>%
  select(participant_id, experiment, starts_with("LDI"), starts_with("REC")) %>%
  tidyr::pivot_longer(
    cols = -c(participant_id, experiment),
    names_to = "condition",
```

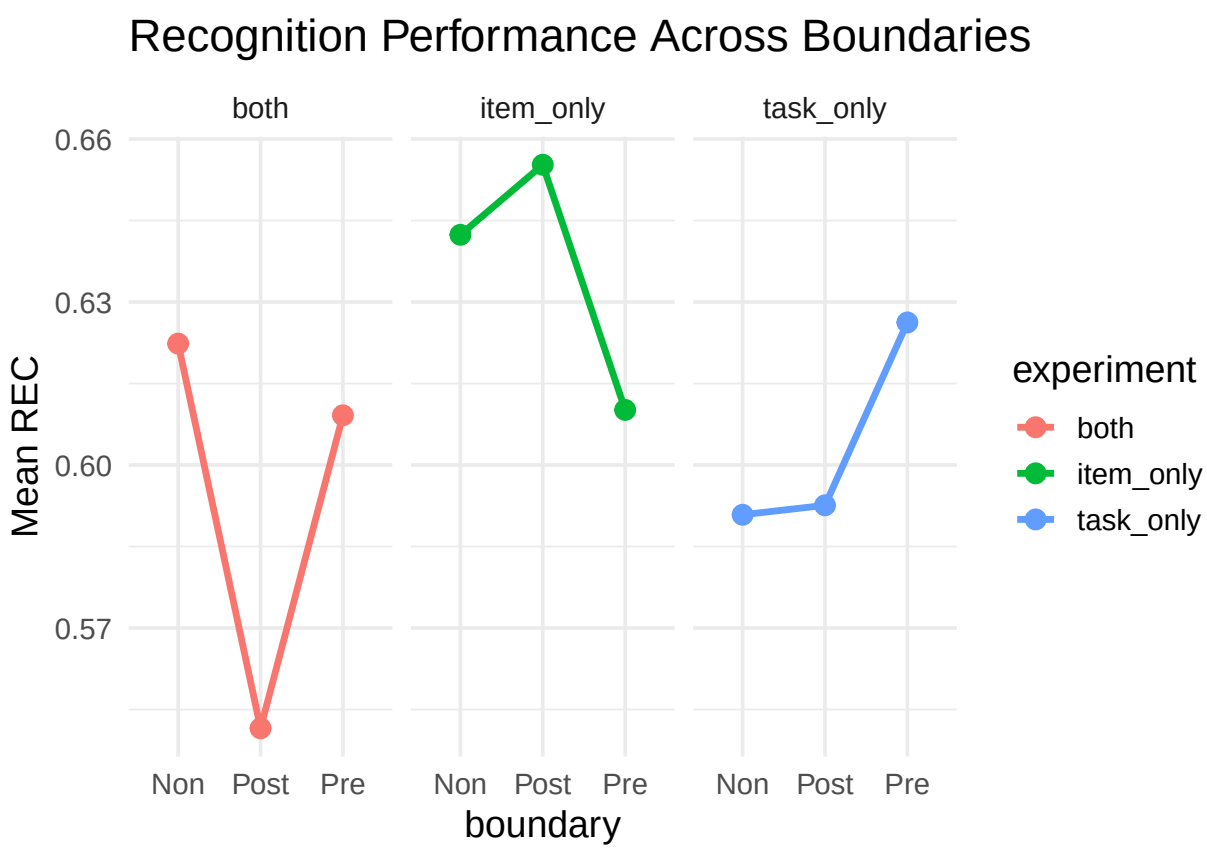
```r
    values_to = "value"
  )

ggplot(diff_data, aes(x = condition, y = value, fill = condition)) +
  stat_summary(fun = mean, geom = "bar") +
  stat_summary(fun.data = mean_se, geom = "errorbar", width = 0.2) +
  facet_wrap(~experiment) +
  labs(
    title = "Difference from Non-Boundary Baseline",
    x = "Condition",
    y = "Difference Score"
  ) +
  theme_minimal(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
## Warning: Removed 3774 rows containing non-finite outside the scale range
## (`stat_summary()`).
## Removed 3774 rows containing non-finite outside the scale range
## (`stat_summary()`).
```




```r
#-----------------------------------------------------------------
# Quick Summary Plot
#-----------------------------------------------------------------
mst_boundary %>%
```

```r
group_by(experiment, boundary) %>%
summarise(
  mean_LDI = mean(LDI, na.rm = TRUE),
  mean_REC = mean(REC, na.rm = TRUE),
  .groups = "drop"
) %>%
ggplot(aes(x = boundary, y = mean_REC, group = experiment, color = experiment)) +
geom_line(size = 1.2) +
geom_point(size = 3) +
facet_wrap(~experiment) +
labs(
  title = "Recognition Performance Across Boundaries",
  y = "Mean REC"
) +
theme_minimal(base_size = 14)
```



LDI Results (Pattern Separation)

One-sample t-tests compared boundary conditions against the Non-boundary baseline.

Across all three experiments (item_only, both, task_only), no significant differences in LDI were observed:

item_only Pre vs Non: p = 0.396 Post vs Non: p = 0.595 both Pre vs Non: p = 0.363 Post vs Non: p = 0.147 task_only Pre vs Non: p = 0.317 Post vs Non: p = 0.249 Although LDI values were generally positive (Mean 0.33), indicating successful lure discrimination overall, event boundaries did not significantly modulate pattern separation.

REC Results (Recognition Memory)

Recognition performance showed condition-specific effects:

item_only No significant differences (all p > 0.1) both Post vs Non: significant decrease t(50) = -3.31, p = 0.0017 Indicates reduced recognition after boundaries task_only Pre vs Non: significant increase t(52) = 2.15, p = 0.036 Indicates enhanced recognition before boundaries

These findings suggest a dissociation between memory processes:

Pattern separation (LDI) remains stable across event boundaries Recognition (REC) is sensitive to boundary structure: Pre-boundary enhancement (task_only) Post-boundary impairment (both condition)

This partially aligns with the original study, where event boundaries influence memory encoding, but extends it by showing that effects are stronger for recognition than for pattern separation.

Paired t-test (Pre vs Post)

The comparison between Pre- and Post-boundary conditions was conducted using a paired-samples t-test rather than an independent-samples t-test. This choice is justified because the same participants contributed data to both conditions, making the observations dependent (within-subject design).

In this experiment, each participant provides memory scores (LDI and REC) for Pre-boundary and Post-boundary items. These measurements are therefore not independent, as they are influenced by shared individual factors such as baseline memory ability, attention, and response tendencies. A paired t-test accounts for this dependency by analyzing within-subject differences, thereby reducing variability due to individual differences and increasing statistical power.

In contrast, an independent-samples t-test assumes that observations across groups are unrelated, which would violate the assumptions of the current experimental design and potentially lead to incorrect inferences.

```
#-------------------------------------------------------
# STEP 1: RESHAPE DATA
#-------------------------------------------------------
library(tidyr)

mst_wide <- mst_boundary %>%
  select(participant_id, experiment, boundary, LDI, REC) %>%
  pivot_wider(
    names_from = boundary,
    values_from = c(LDI, REC)
  )
#-------------------------------------------------------
# STEP 2: PAIRED T-TEST (LDI)
#-------------------------------------------------------
cat("\n==== PAIRED TEST: LDI (Pre vs Post) ====\n")
```

```
##
## ==== PAIRED TEST: LDI (Pre vs Post) ====
```

```
for(exp in unique(mst_wide$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- mst_wide %>% filter(experiment == exp)

  pre  <- data_exp$LDI_Pre
```

```r
  post <- data_exp$LDI_Post

  # remove NA pairs
  valid <- !is.na(pre) & !is.na(post)

  if(sum(valid) > 1){
    print(t.test(pre[valid], post[valid], paired = TRUE))
  }
}
```

```
##
## Experiment: item_only
##
##  Paired t-test
##
## data:  pre[valid] and post[valid]
## t = -1.1332, df = 55, p-value = 0.2621
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.09090756  0.02523569
## sample estimates:
## mean difference
##     -0.03283593
##
##
## Experiment: both
##
##  Paired t-test
##
## data:  pre[valid] and post[valid]
## t = 0.60903, df = 50, p-value = 0.5453
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.03130765  0.05855579
## sample estimates:
## mean difference
##      0.01362407
##
##
## Experiment: task_only
##
##  Paired t-test
##
## data:  pre[valid] and post[valid]
## t = -0.33581, df = 52, p-value = 0.7384
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.05576428  0.03977588
## sample estimates:
## mean difference
##    -0.007994199
```

```r
#-------------------------------------------------------
# STEP 3: PAIRED T-TEST (REC)
#-------------------------------------------------------
cat("\n==== PAIRED TEST: REC (Pre vs Post) ====\n")
```

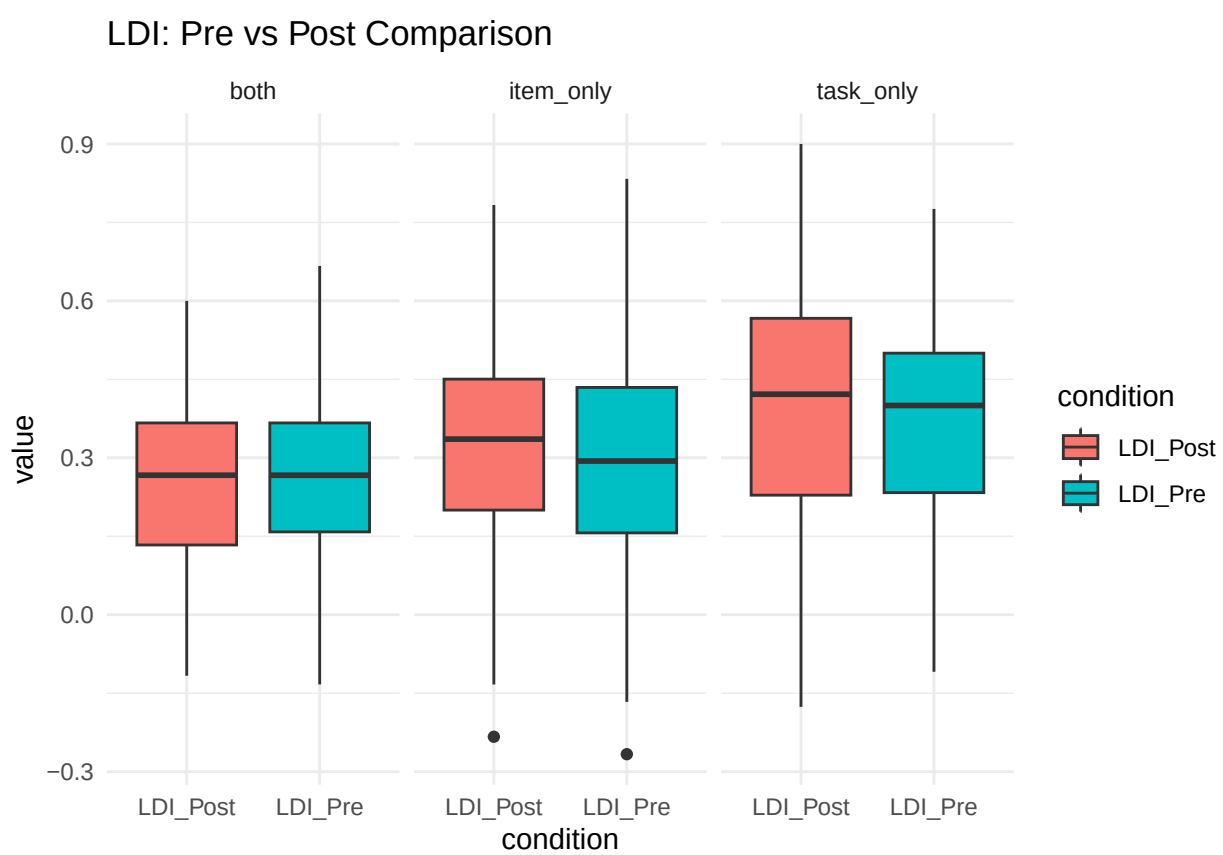
```
##
## ==== PAIRED TEST: REC (Pre vs Post) ====
```

```r
for(exp in unique(mst_wide$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- mst_wide %>% filter(experiment == exp)

  pre  <- data_exp$REC_Pre
  post <- data_exp$REC_Post

  valid <- !is.na(pre) & !is.na(post)

  if(sum(valid) > 1){
    print(t.test(pre[valid], post[valid], paired = TRUE))
  }
}
```

```
##
## Experiment: item_only
##
##  Paired t-test
##
## data:  pre[valid] and post[valid]
## t = -1.7837, df = 55, p-value = 0.07999
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.095814541  0.005573836
## sample estimates:
## mean difference
##     -0.04512035
##
##
## Experiment: both
##
##  Paired t-test
##
## data:  pre[valid] and post[valid]
## t = 2.9447, df = 50, p-value = 0.004894
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  0.01831363 0.09689730
## sample estimates:
## mean difference
##      0.05760547
##
##
```

```
## Experiment: task_only
##
##  Paired t-test
##
## data:  pre[valid] and post[valid]
## t = 1.4607, df = 52, p-value = 0.1501
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.01257880  0.07988956
## sample estimates:
## mean difference
##      0.03365538
```

```r
#-----------------------------------------------------------
# VISUAL
#-----------------------------------------------------------
mst_wide %>%
  pivot_longer(cols = c(LDI_Pre, LDI_Post),
               names_to = "condition", values_to = "value") %>%
  ggplot(aes(x = condition, y = value, fill = condition)) +
  geom_boxplot() +
  facet_wrap(~experiment) +
  labs(title = "LDI: Pre vs Post Comparison") +
  theme_minimal()
```



To directly compare memory performance across event boundaries, paired-samples t-tests were conducted

between Pre-boundary and Post-boundary conditions for both LDI and REC. This analysis captures within-subject differences in memory performance across boundary transitions.

Lure Discrimination Index (LDI)

Across all experimental conditions, no significant differences were observed between Pre- and Post-boundary items:

item_only: t(55) = -1.13, p = 0.262 both: t(50) = 0.61, p = 0.545 task_only: t(52) = -0.34, p = 0.738

Although mean differences varied slightly across conditions, none reached statistical significance.

Interpretation

These results indicate that pattern separation (LDI) is not significantly influenced by event boundaries, suggesting that the ability to discriminate similar items remains stable across boundary transitions.

Recognition Memory (REC)

Recognition performance showed a condition-specific effect:

item_only: t(55) = -1.78, p = 0.080 (marginal, not significant) both: t(50) = 2.94, p = 0.0049 Mean difference = +0.058 (Pre > Post) task_only: t(52) = 1.46, p = 0.150 (not significant) Interpretation

A significant Pre > Post effect was observed in the both condition, indicating that recognition memory is stronger for items presented before an event boundary compared to those presented after.

No reliable differences were found in the item-only or task-only conditions, although the item-only condition showed a trend toward reduced performance post-boundary.

Overall Interpretation

These findings reveal a clear dissociation between memory processes:

LDI (pattern separation) → stable across boundaries REC (recognition memory) → selectively affected by boundaries

Specifically:

Event boundaries appear to disrupt recognition for post-boundary items Pre-boundary items benefit from enhanced encoding or consolidation Comparison with Morse et al. (2023)

The present findings are partially consistent with Morse et al. (2023).

Similar to the original study, we observe reduced memory performance following event boundaries, reflected here as lower recognition for Post-boundary items. Additionally, the enhancement of Pre-boundary memory aligns with the idea that boundaries trigger segmentation processes that strengthen encoding of preceding information.

However, unlike Morse et al., who reported more consistent boundary effects, the current results show that:

Effects are limited to the combined ("both") condition Pattern separation (LDI) remains unaffected, suggesting that boundary effects primarily influence recognition rather than fine-grained discrimination

Lure Bin Analysis

```
library(dplyr)
library(ggplot2)



#-------------------------------------------
```

```r
# EXTRACT ID
#-------------------------------------------
extract_id <- function(x){
  num <- sub(".*?(\\d+)[ab]\\.jpg$", "\\1", tolower(x))
  as.numeric(num)
}

#-------------------------------------------
# DETECT STIMULUS TYPE FROM PATH
#-------------------------------------------
get_stim_type <- function(x){
  case_when(
    grepl("objects", tolower(x)) ~ "object",
    grepl("scenes", tolower(x)) ~ "scene",
    TRUE ~ NA_character_
  )
}

#-------------------------------------------
# PREP TEST DATA
#-------------------------------------------
test_bins_all <- test_clean %>%
  mutate(
    item_id = extract_id(image_path),
    stim_type = get_stim_type(image_path),
    experiment = tolower(experiment)
  )
```

```
## Warning: There was 1 warning in `mutate()`.
## i In argument: `item_id = extract_id(image_path)`.
## Caused by warning in `extract_id()`:
## ! NAs introduced by coercion
```

```r
#-------------------------------------------
# LOAD BIN FILES (GLOBAL, NOT PER EXP)
#-------------------------------------------
bins_obj <- read.table(file.path(base_path, "Both_item_task", "Set6 bins.txt"), header = FALSE) %>%
  setNames(c("image_name", "lure_bin")) %>%
  mutate(item_id = as.numeric(image_name),
         stim_type = "object")

bins_scn <- read.table(file.path(base_path, "Both_item_task", "SetScC bins.txt"), header = FALSE) %>%
  setNames(c("image_name", "lure_bin")) %>%
  mutate(item_id = as.numeric(image_name),
         stim_type = "scene")

#-------------------------------------------
# COMBINE BIN FILES
#-------------------------------------------
bins_all <- bind_rows(bins_obj, bins_scn)

#-------------------------------------------
# JOIN USING BOTH ID + TYPE
```

```r
#-------------------------------------------
test_bins <- test_bins_all %>%
  left_join(bins_all, by = c("item_id", "stim_type"))

#-------------------------------------------
# DEBUG
#-------------------------------------------
cat("\nMapping success:\n")
```

```
##
## Mapping success:
```

```r
print(table(!is.na(test_bins$lure_bin), test_bins$experiment))
```

```
##
##         both item_only task_only
##   FALSE 1560      1680      4770
##   TRUE  6240      6720      3180
```

```r
#-------------------------------------------
# FILTER LURES
#-------------------------------------------
lure_data <- test_bins %>%
  filter(stimulus_type == "Lure", !is.na(lure_bin))

#-------------------------------------------
# CHECK DISTRIBUTION
#-------------------------------------------
cat("\nBin distribution:\n")
```

```
##
## Bin distribution:
```

```r
print(table(lure_data$lure_bin, lure_data$experiment))
```

```
##
##     both item_only task_only
##   1  632       723       354
##   2  595       667       318
##   3  640       665       333
##   4  651       668       275
##   5  602       637       349
```

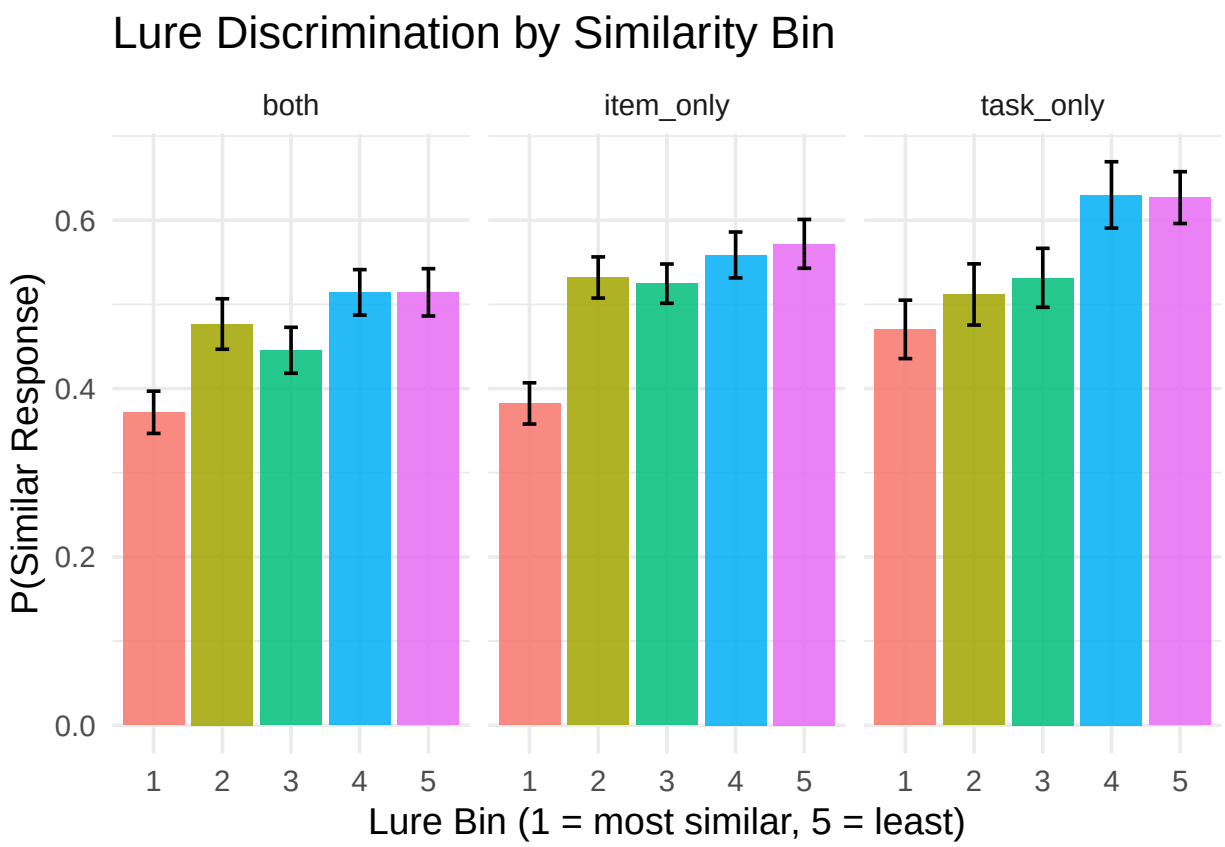
```r
#-------------------------------------------
# COMPUTE PERFORMANCE
#-------------------------------------------
lure_bins <- lure_data %>%
  group_by(participant_id, experiment, lure_bin) %>%
  summarise(
    p_similar = mean(response == "Similar"),
```

```r
    .groups = "drop"
  )

#--------------------------------------------
# SUMMARY
#--------------------------------------------
lure_summary <- lure_bins %>%
  group_by(experiment, lure_bin) %>%
  summarise(
    mean = mean(p_similar, na.rm = TRUE),
    se = sd(p_similar, na.rm = TRUE) / sqrt(n()),
    .groups = "drop"
  )

print(lure_summary)
```

```
## # A tibble: 15 x 4
##    experiment lure_bin  mean     se
##    <chr>         <int> <dbl>  <dbl>
##  1 both              1 0.372 0.0251
##  2 both              2 0.477 0.0300
##  3 both              3 0.445 0.0273
##  4 both              4 0.514 0.0270
##  5 both              5 0.514 0.0281
##  6 item_only         1 0.382 0.0245
##  7 item_only         2 0.532 0.0244
##  8 item_only         3 0.525 0.0233
##  9 item_only         4 0.559 0.0273
## 10 item_only         5 0.572 0.0290
## 11 task_only         1 0.470 0.0347
## 12 task_only         2 0.512 0.0363
## 13 task_only         3 0.532 0.0350
## 14 task_only         4 0.630 0.0394
## 15 task_only         5 0.627 0.0308
```

```r
#--------------------------------------------
# CLEAN LABELS
#--------------------------------------------
lure_summary <- lure_summary %>%
  mutate(experiment = recode(experiment,
    "both_item_task" = "both"
  ))

#--------------------------------------------
# PLOT
#--------------------------------------------
ggplot(lure_summary, aes(x = factor(lure_bin), y = mean, fill = factor(lure_bin))) +
  geom_bar(stat = "identity", alpha = 0.85) +
  geom_errorbar(aes(ymin = mean - se, ymax = mean + se), width = 0.2) +
  facet_wrap(~experiment) +
  labs(
    title = "Lure Discrimination by Similarity Bin",
    x = "Lure Bin (1 = most similar, 5 = least)",
```

```
    y = "P(Similar Response)"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```




```
library(afex)

cat("\n===== LURE BIN ANOVA =====\n")
```

```
##
## ===== LURE BIN ANOVA =====
```

```
for(exp in unique(lure_bins$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- lure_bins %>% filter(experiment == exp)

  model <- aov_ez(
    id = "participant_id",
    dv = "p_similar",
    data = data_exp,
    within = "lure_bin"
  )
```

```r
  print(model)
}
```

```
##
## Experiment: item_only
## Anova Table (Type 3 tests)
##
## Response: p_similar
##     Effect           df  MSE         F  ges p.value
## 1 lure_bin 3.79, 208.62 0.02 17.35 *** .112   <.001
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## Experiment: both
## Anova Table (Type 3 tests)
##
## Response: p_similar
##     Effect           df  MSE        F  ges p.value
## 1 lure_bin 3.59, 179.41 0.02 9.81 *** .069   <.001
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## Experiment: task_only

## Warning: Missing values for 1 ID(s), which were removed before analysis:
## 00060
## Below the first few rows (in wide format) of the removed cases with missing data.
##      participant_id X1  X2        X3 X4        X5
## # 41          00060  1 0.5 0.6363636 NA 0.3333333

## Anova Table (Type 3 tests)
##
## Response: p_similar
##     Effect           df  MSE        F  ges p.value
## 1 lure_bin 3.80, 193.92 0.05 6.96 *** .067   <.001
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
```

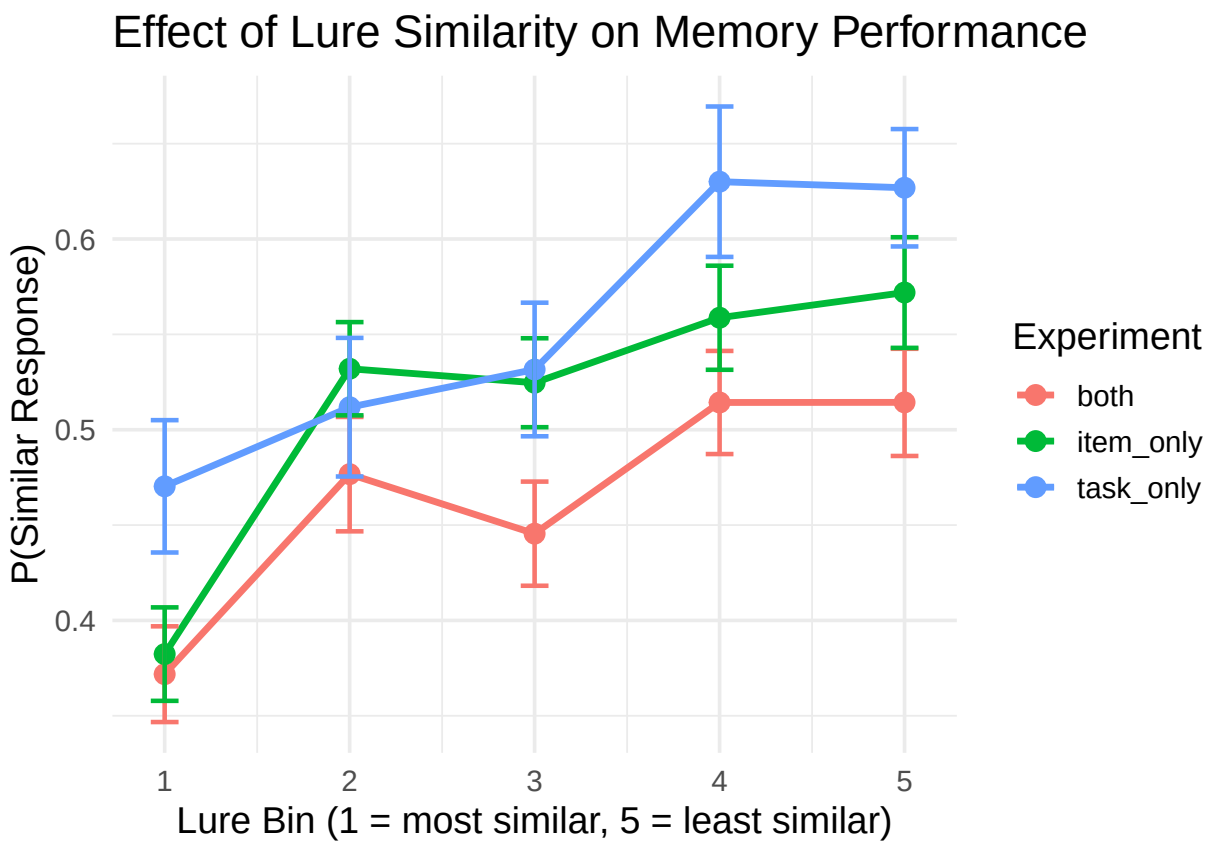
```r
library(emmeans)

for(exp in unique(lure_bins$experiment)){

  cat("\nPost-hoc:", exp, "\n")

  data_exp <- lure_bins %>% filter(experiment == exp)
```

```r
  model <- aov_ez(
    id = "participant_id",
    dv = "p_similar",
    data = data_exp,
    within = "lure_bin"
  )

  emm <- emmeans(model, ~ lure_bin)
  print(pairs(emm, adjust = "bonferroni"))
}
```

```
##
## Post-hoc: item_only
##  contrast estimate     SE df t.ratio p.value
##  X1 - X2  -0.14964 0.0262 55  -5.718 <0.0001
##  X1 - X3  -0.14230 0.0215 55  -6.629 <0.0001
##  X1 - X4  -0.17638 0.0269 55  -6.561 <0.0001
##  X1 - X5  -0.18960 0.0281 55  -6.757 <0.0001
##  X2 - X3   0.00734 0.0234 55   0.314  1.0000
##  X2 - X4  -0.02675 0.0265 55  -1.007  1.0000
##  X2 - X5  -0.03996 0.0266 55  -1.500  1.0000
##  X3 - X4  -0.03408 0.0253 55  -1.346  1.0000
##  X3 - X5  -0.04730 0.0266 55  -1.780  0.8057
##  X4 - X5  -0.01322 0.0265 55  -0.499  1.0000
##
## P value adjustment: bonferroni method for 10 tests
##
## Post-hoc: both
##  contrast  estimate     SE df t.ratio p.value
##  X1 - X2  -1.05e-01 0.0318 50  -3.294  0.0182
##  X1 - X3  -7.37e-02 0.0267 50  -2.759  0.0808
##  X1 - X4  -1.42e-01 0.0258 50  -5.530 <0.0001
##  X1 - X5  -1.43e-01 0.0282 50  -5.051 <0.0001
##  X2 - X3   3.12e-02 0.0285 50   1.096  1.0000
##  X2 - X4  -3.76e-02 0.0295 50  -1.272  1.0000
##  X2 - X5  -3.76e-02 0.0258 50  -1.461  1.0000
##  X3 - X4  -6.88e-02 0.0249 50  -2.758  0.0810
##  X3 - X5  -6.88e-02 0.0213 50  -3.239  0.0214
##  X4 - X5  -5.51e-05 0.0236 50  -0.002  1.0000
##
## P value adjustment: bonferroni method for 10 tests
##
## Post-hoc: task_only

## Warning: Missing values for 1 ID(s), which were removed before analysis:
## 00060
## Below the first few rows (in wide format) of the removed cases with missing data.
##      participant_id X1  X2        X3 X4        X5
## # 41          00060  1 0.5 0.6363636 NA 0.3333333

##  contrast estimate     SE df t.ratio p.value
##  X1 - X2  -0.05191 0.0404 51  -1.286  1.0000
##  X1 - X3  -0.06944 0.0407 51  -1.705  0.9422
```

```
##  X1 - X4  -0.16993 0.0422 51  -4.025  0.0019
##  X1 - X5  -0.17239 0.0398 51  -4.337  0.0007
##  X2 - X3  -0.01753 0.0437 51  -0.401  1.0000
##  X2 - X4  -0.11802 0.0405 51  -2.915  0.0527
##  X2 - X5  -0.12049 0.0426 51  -2.831  0.0662
##  X3 - X4  -0.10049 0.0431 51  -2.334  0.2359
##  X3 - X5  -0.10295 0.0347 51  -2.966  0.0458
##  X4 - X5  -0.00246 0.0392 51  -0.063  1.0000
##
## P value adjustment: bonferroni method for 10 tests
```

```r
ggplot(lure_summary, aes(x = lure_bin, y = mean, color = experiment, group = experiment)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(ymin = mean - se, ymax = mean + se),
    width = 0.15,
    size = 0.8
  ) +
  scale_x_continuous(breaks = 1:5) +
  labs(
    title = "Effect of Lure Similarity on Memory Performance",
    x = "Lure Bin (1 = most similar, 5 = least similar)",
    y = "P(Similar Response)",
    color = "Experiment"
  ) +
  theme_minimal(base_size = 14)
```



Lure Bin Analysis Method

Lure stimuli were grouped into five similarity bins (1 = most similar, 5 = least similar). For each participant, the probability of responding "Similar" was computed for each bin. A repeated-measures ANOVA was conducted separately for each experimental condition (item_only, both, task_only) to examine the effect of similarity on lure discrimination. Post-hoc pairwise comparisons were performed using Bonferroni correction.

Results

A significant main effect of lure similarity was observed in all three conditions:

item_only: $F(3.79, 208.62) = 17.35$, $p < .001$, ges = .112 both: $F(3.59, 179.41) = 9.81$, $p < .001$, ges = .069 task_only: $F(3.80, 193.92) = 6.96$, $p < .001$, ges = .067

These results indicate that lure discrimination performance varied significantly across similarity levels.

Post-hoc comparisons revealed that, in all conditions, the most similar lures (Bin 1) were significantly harder to discriminate than less similar lures:

In item_only, Bin 1 differed significantly from all other bins ($p < .001$), while differences among higher bins were not significant. In both, Bin 1 differed significantly from Bins 4 and 5 ($p < .001$), with weaker or marginal differences for intermediate bins. In task_only, Bin 1 differed significantly from Bins 4 and 5 ($p < .01$), with additional differences emerging between intermediate and higher bins. Interpretation

These findings demonstrate a clear similarity-dependent gradient in memory performance, where highly similar lures are more difficult to discriminate than less similar ones. This pattern is consistent with pattern separation mechanisms, in which overlapping representations lead to increased confusion between targets and similar lures.

The effect size was largest in the item_only condition (ges = .112), suggesting stronger sensitivity to similarity differences when stimuli are processed in isolation. The both and task_only conditions showed smaller but

still significant effects, indicating that additional task demands or contextual processing may reduce—but not eliminate—the influence of stimulus similarity.

While Morse et al. (2023) primarily investigated the influence of event boundaries on memory, the present lure bin analysis extends these findings by showing that memory performance is also systematically modulated by stimulus similarity. Specifically, discrimination improves as perceptual similarity decreases, supporting accounts of hippocampal pattern separation operating alongside event segmentation processes.

Quartile Analysis

```r
library(dplyr)
library(afex)
library(ggplot2)

cat("\n=========== QUARTILE ANALYSIS (ENCODING → LDI/REC) ===========\n")
```

```
##
## =========== QUARTILE ANALYSIS (ENCODING → LDI/REC) ===========
```

```r
#-----------------------------------------------
# ROBUST ID EXTRACTION
#-----------------------------------------------
extract_id <- function(x){
  x <- tolower(x)
  num <- sub(".*?(\\d+)[ab]?\\.jpg$", "\\1", x)
  as.numeric(num)
}

#-----------------------------------------------
# STEP 1: ENCODING DATA → ADD QUARTILES
#-----------------------------------------------
task_q <- task_all %>%
  mutate(
    experiment = tolower(experiment),
    item_id = extract_id(image_name)
  ) %>%
  group_by(participant_id, experiment) %>%
  mutate(
    quartile = ntile(trial_number, 4)
  ) %>%
  ungroup()

#-----------------------------------------------
# STEP 2: TEST DATA
#-----------------------------------------------
test_q <- test_clean %>%
  mutate(
    experiment = tolower(experiment),
    item_id = extract_id(image_name)
  )
```

```r
#-----------------------------------------------------
# STEP 3: MAP ENCODING → TEST
#-----------------------------------------------------
test_with_q <- test_q %>%
  left_join(
    task_q %>%
      select(participant_id, experiment, item_id, quartile),
    by = c("participant_id", "experiment", "item_id"),
    relationship = "many-to-many"
  )

#-----------------------------------------------------
#   CRITICAL FIX: REMOVE NA QUARTILES
# (removes foils + unmatched items)
#-----------------------------------------------------
test_with_q <- test_with_q %>%
  filter(!is.na(quartile))

#-----------------------------------------------------
# DEBUG CHECK
#-----------------------------------------------------
cat("\nQuartile distribution after cleaning:\n")
```

```
##
## Quartile distribution after cleaning:
```

```r
print(table(test_with_q$quartile))
```

```
##
##    1    2    3    4
## 8658 8708 8786 8720
```

```r
#-----------------------------------------------------
# STEP 4: COMPUTE LDI & REC PER QUARTILE
#-----------------------------------------------------
mst_quartile <- test_with_q %>%
  group_by(participant_id, experiment, quartile) %>%
  summarise(
    p_old_target = mean(stimulus_type == "Target" & response == "Old"),
    p_old_foil   = mean(stimulus_type == "Foil" & response == "Old"),

    p_sim_lure   = mean(stimulus_type == "Lure" & response == "Similar"),
    p_sim_foil   = mean(stimulus_type == "Foil" & response == "Similar"),

    .groups = "drop"
  ) %>%
  mutate(
    REC = p_old_target - p_old_foil,
    LDI = p_sim_lure - p_sim_foil
  )

cat("\nSample MST quartile data:\n")
```

```
##
## Sample MST quartile data:
```

```r
print(head(mst_quartile))
```

```
## # A tibble: 6 x 9
##   participant_id experiment quartile p_old_target p_old_foil p_sim_lure
##   <chr>          <chr>         <int>        <dbl>      <dbl>      <dbl>
## 1 00001          item_only         1        0.246     0.0154      0.215
## 2 00001          item_only         2        0.194     0           0.274
## 3 00001          item_only         3        0.369     0.0308      0.185
## 4 00001          item_only         4        0.344     0.0156      0.266
## 5 00002          item_only         1        0.211     0.0175      0.175
## 6 00002          item_only         2        0.222     0           0.143
## # i 3 more variables: p_sim_foil <dbl>, REC <dbl>, LDI <dbl>
```

```r
#-----------------------------------------------
# STEP 5: ANOVA - LDI
#-----------------------------------------------
cat("\n===== QUARTILE ANOVA: LDI =====\n")
```

```
##
## ===== QUARTILE ANOVA: LDI =====
```

```r
for(exp in unique(mst_quartile$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- mst_quartile %>% filter(experiment == exp)

  model <- aov_ez(
    id = "participant_id",
    dv = "LDI",
    data = data_exp,
    within = "quartile"
  )

  print(model)
}
```

```
##
## Experiment: item_only
## Anova Table (Type 3 tests)
##
## Response: LDI
##     Effect           df  MSE    F  ges p.value
## 1 quartile 2.75, 151.43 0.00 1.25 .009    .293
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
```

```
## Experiment: both
## Anova Table (Type 3 tests)
##
## Response: LDI
##     Effect           df  MSE    F  ges p.value
## 1 quartile 2.82, 140.93 0.00 0.38 .003    .756
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## Experiment: task_only
## Anova Table (Type 3 tests)
##
## Response: LDI
##     Effect           df  MSE    F  ges p.value
## 1 quartile 2.95, 153.23 0.01 0.24 .002    .865
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
```

```r
#-----------------------------------------------
# STEP 6: ANOVA - REC
#-----------------------------------------------
cat("\n===== QUARTILE ANOVA: REC =====\n")
```

```
##
## ===== QUARTILE ANOVA: REC =====
```

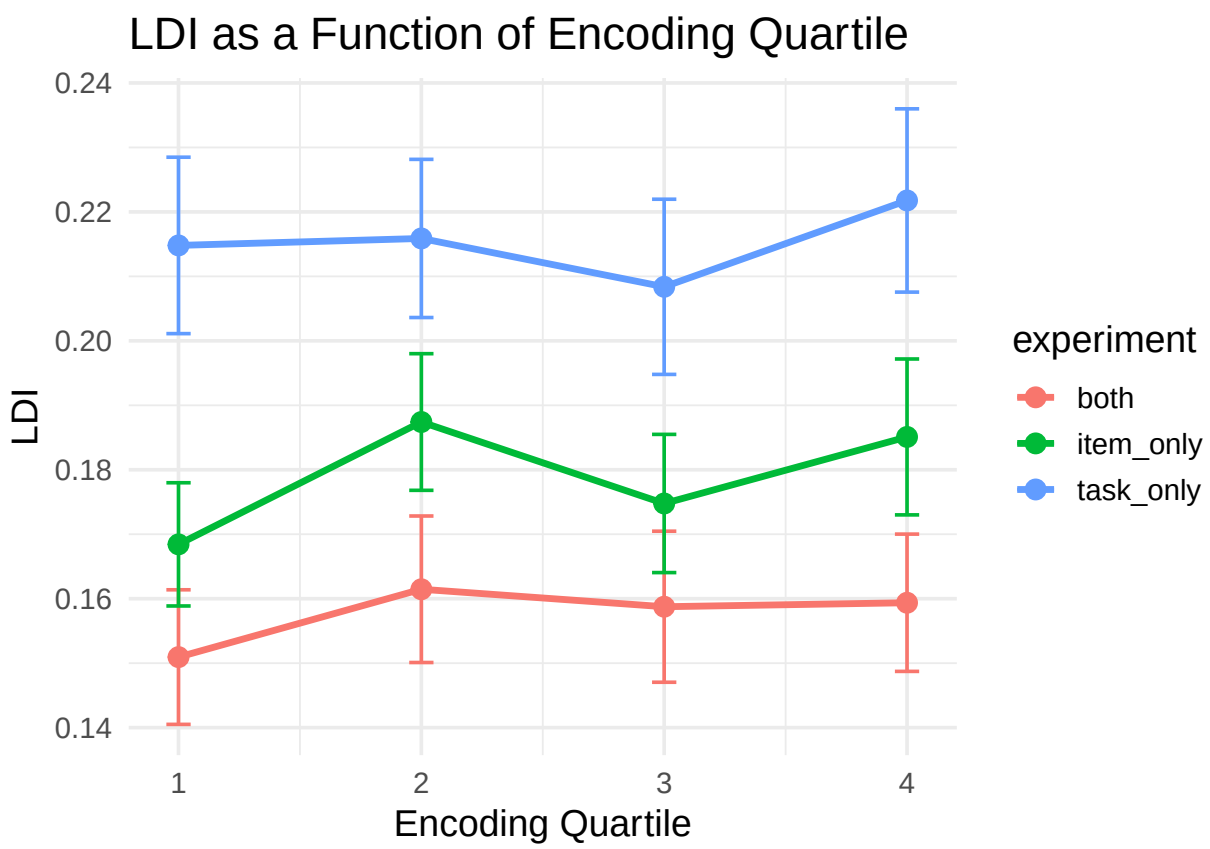
```r
for(exp in unique(mst_quartile$experiment)){

  cat("\nExperiment:", exp, "\n")

  data_exp <- mst_quartile %>% filter(experiment == exp)

  model <- aov_ez(
    id = "participant_id",
    dv = "REC",
    data = data_exp,
    within = "quartile"
  )

  print(model)
}
```

```
##
## Experiment: item_only
## Anova Table (Type 3 tests)
##
## Response: REC
##     Effect           df  MSE    F  ges p.value
## 1 quartile 2.87, 157.92 0.00 0.25 .002    .851
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## Experiment: both
## Anova Table (Type 3 tests)
##
## Response: REC
##     Effect           df  MSE    F  ges p.value
## 1 quartile 2.93, 146.68 0.00 1.70 .012    .170
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
##
## Experiment: task_only
## Anova Table (Type 3 tests)
##
## Response: REC
##     Effect           df  MSE    F  ges p.value
## 1 quartile 2.64, 137.35 0.01 1.25 .012    .294
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
##
## Sphericity correction method: GG
```

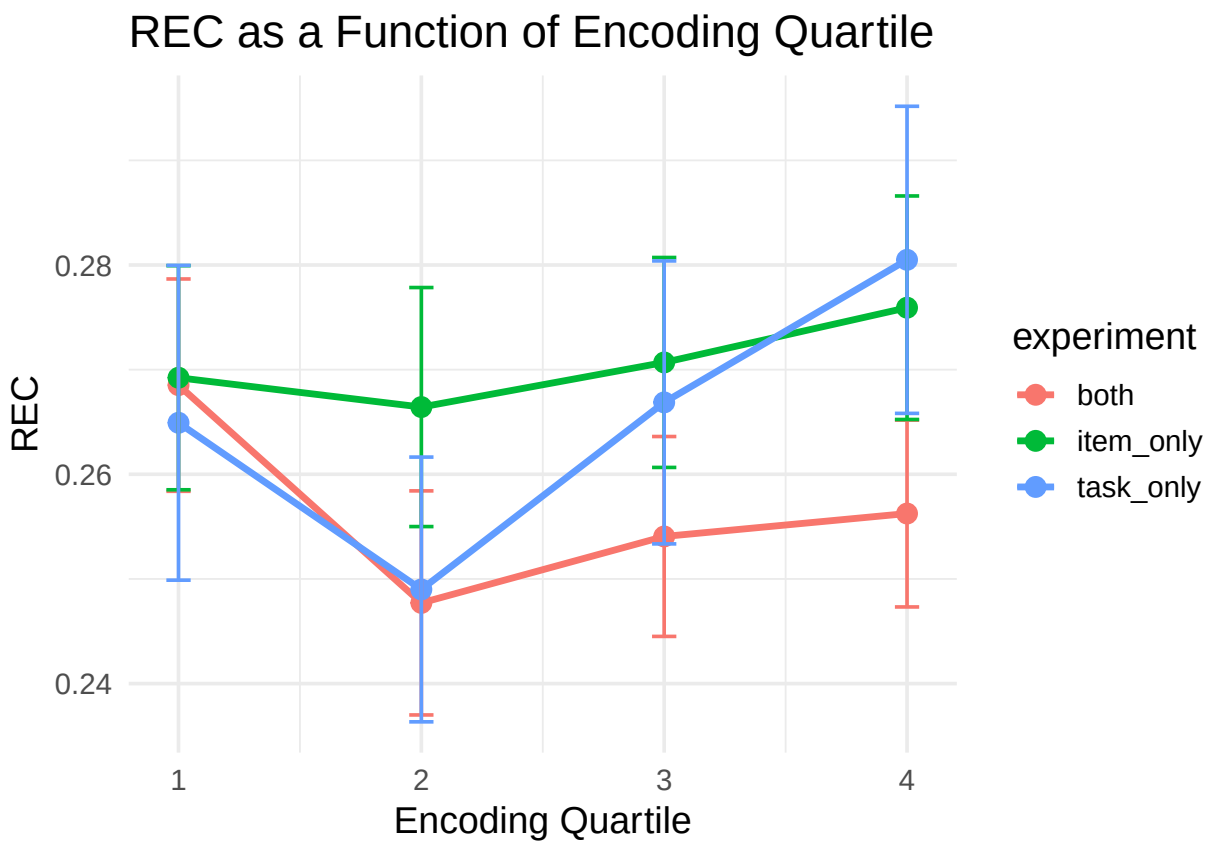
```r
#-------------------------------------------------
# STEP 7: SUMMARY FOR PLOTS
#-------------------------------------------------
mst_q_summary <- mst_quartile %>%
  group_by(experiment, quartile) %>%
  summarise(
    mean_LDI = mean(LDI, na.rm = TRUE),
    se_LDI = sd(LDI, na.rm = TRUE)/sqrt(n()),
    mean_REC = mean(REC, na.rm = TRUE),
    se_REC = sd(REC, na.rm = TRUE)/sqrt(n()),
    .groups = "drop"
  )

# Clean labels
mst_q_summary <- mst_q_summary %>%
  mutate(experiment = recode(experiment,
    "both_item_task" = "both"
  ))

#-------------------------------------------------
# STEP 8: PLOT LDI
#-------------------------------------------------
ggplot(mst_q_summary, aes(x = quartile, y = mean_LDI, color = experiment)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(ymin = mean_LDI - se_LDI, ymax = mean_LDI + se_LDI),
```

```
    width = 0.1
  ) +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "LDI as a Function of Encoding Quartile",
    x = "Encoding Quartile",
    y = "LDI"
  ) +
  theme_minimal(base_size = 14)
```




```
#-----------------------------------------------------
# STEP 9: PLOT REC
#-----------------------------------------------------
ggplot(mst_q_summary, aes(x = quartile, y = mean_REC, color = experiment)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(ymin = mean_REC - se_REC, ymax = mean_REC + se_REC),
    width = 0.1
  ) +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "REC as a Function of Encoding Quartile",
    x = "Encoding Quartile",
    y = "REC"
```

```
  ) +
  theme_minimal(base_size = 14)
```




```
cat("\n=========== QUARTILE ANALYSIS COMPLETE ===========\n")
```

```
##
## =========== QUARTILE ANALYSIS COMPLETE ===========
```

Method

To examine whether memory performance was influenced by temporal factors during encoding, encoding trials were divided into four quartiles based on trial order within each participant. Test items were then grouped according to the quartile in which they were originally encountered during encoding. Lure Discrimination Index (LDI) and Recognition (REC) scores were computed separately for each quartile. Repeated-measures ANOVAs were conducted for each experimental condition to assess the effect of encoding position.

Results

There was no significant effect of encoding quartile on either LDI or REC in any of the experimental conditions.

LDI item_only: F(2.75, 151.43) = 1.25, p = .293, ges = .009 both: F(2.82, 140.93) = 0.38, p = .756, ges = .003 task_only: F(2.95, 153.23) = 0.24, p = .865, ges = .002 REC item_only: F(2.87, 157.92) = 0.25, p = .851, ges = .002 both: F(2.93, 146.68) = 1.70, p = .170, ges = .012 task_only: F(2.64, 137.35) = 1.25, p = .294, ges = .012

Across all conditions, effect sizes were very small and no quartile-related trends were observed.

Interpretation

The absence of any significant quartile effects indicates that memory performance (both recognition and lure discrimination) remained stable across the encoding phase. This suggests that participants' performance was not influenced by temporal factors such as practice effects, fatigue, or increasing predictability of block transitions.

Importantly, this finding strengthens the validity of earlier results by ruling out a key confound: if participants were simply learning when boundaries occur, performance would vary across early and late encoding trials. However, no such pattern was observed.